

本节内容

二叉排序树 (BST)

知识总览

二叉排序树

二叉排序树的定义

查找操作

插入操作

删除操作

查找效率分析

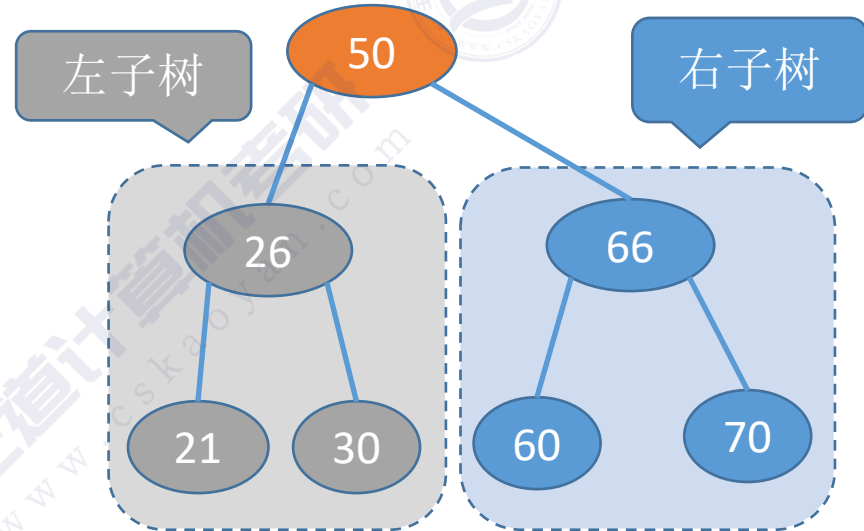
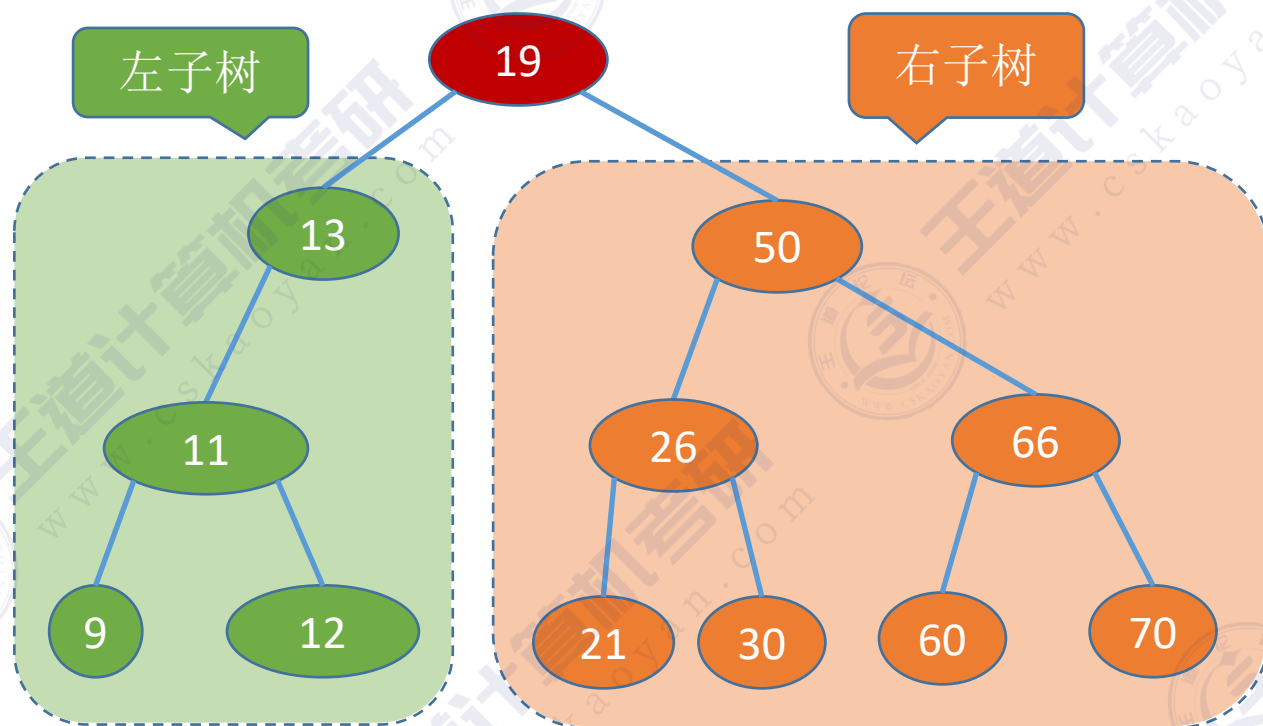
二叉排序树的定义

二叉排序树可用于元素的有序组织、搜索

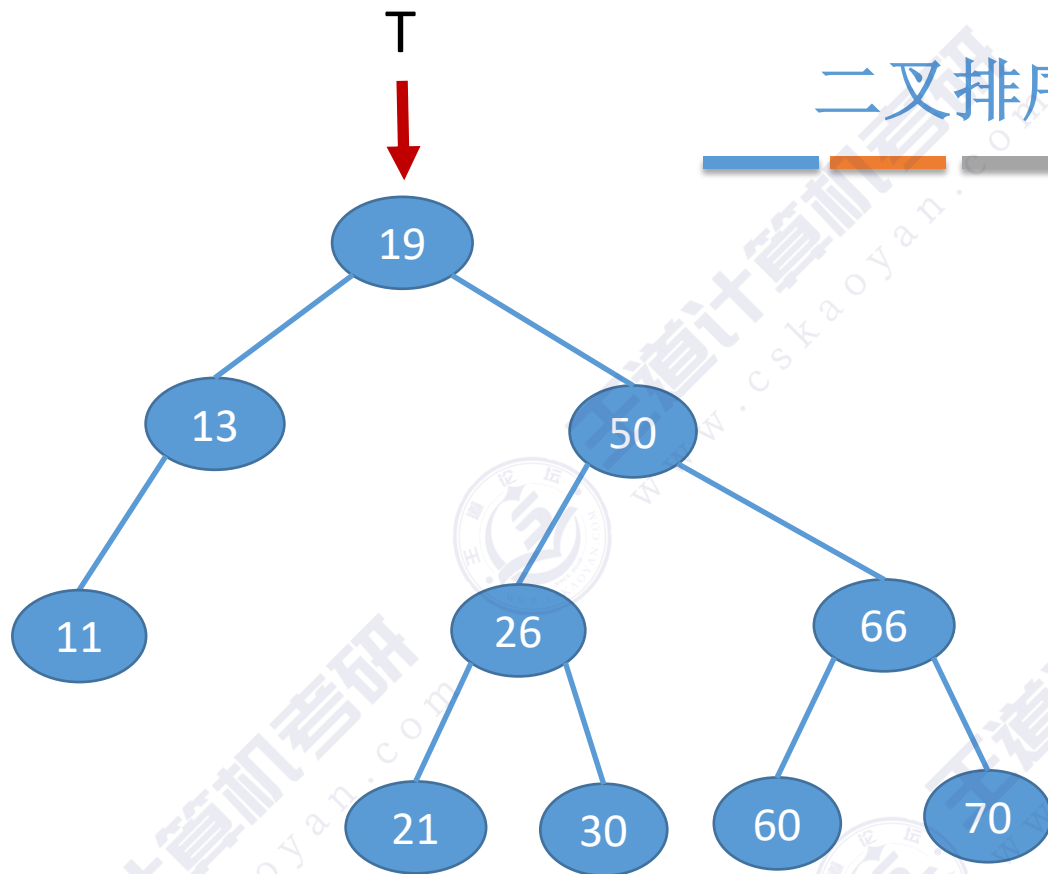
二叉排序树，又称二叉查找树（BST，Binary Search Tree）
一棵二叉树或者是空二叉树，或者是具有如下性质的二叉树：
左子树上所有结点的关键字均小于根结点的关键字；
右子树上所有结点的关键字均大于根结点的关键字。
左子树和右子树又各是一棵二叉排序树。

左子树结点值 < 根结点值 < 右子树结点值

进行中序遍历，可以得到一个递增的有序序列



二叉排序树的查找



左子树结点值 < 根结点值 < 右子树结点值

若树非空，目标值与根结点的值比较：
若相等，则查找成功；
若小于根结点，则在左子树上查找，否则在右子树上查找。

查找成功，返回结点指针；查找失败返回NULL

例1：查找关键字为30的结点

//二叉排序树结点

```
typedef struct BSTNode{  
    int key;  
    struct BSTNode *lchild,*rchild;  
}BSTNode,*BSTree;
```

//在二叉排序树中查找值为 key 的结点

```
BSTNode *BST_Search(BSTree T,int key){
```

```
    while(T!=NULL&&key!=T->key){
```

```
        if(key<T->key) T=T->lchild;
```

```
        else T=T->rchild;
```

```
    }
```

```
    return T;
```

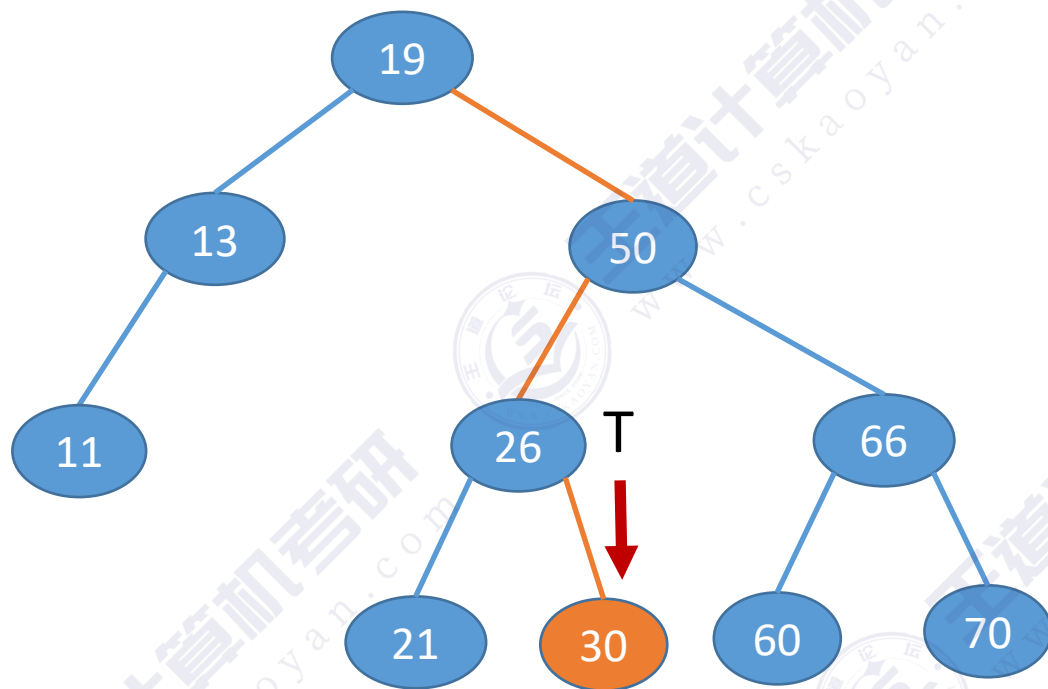
```
}
```

//若树空或等于根结点值，则结束循环

//小于，则在左子树上查找

//大于，则在右子树上查找

二叉排序树的查找



左子树结点值 < 根结点值 < 右子树结点值

若树非空，目标值与根结点的值比较：
若相等，则查找成功；
若小于根结点，则在左子树上查找，否则在右子树上查找。

查找成功，返回结点指针；查找失败返回NULL

例1：查找关键字为30的结点

//二叉排序树结点

```
typedef struct BSTNode{  
    int key;  
    struct BSTNode *lchild,*rchild;  
}BSTNode,*BSTree;
```

//在二叉排序树中查找值为 key 的结点

```
BSTNode *BST_Search(BSTree T,int key){
```

```
    while(T!=NULL&&key!=T->key){
```

```
        if(key<T->key) T=T->lchild;
```

```
        else T=T->rchild;
```

```
    }
```

```
    return T;
```

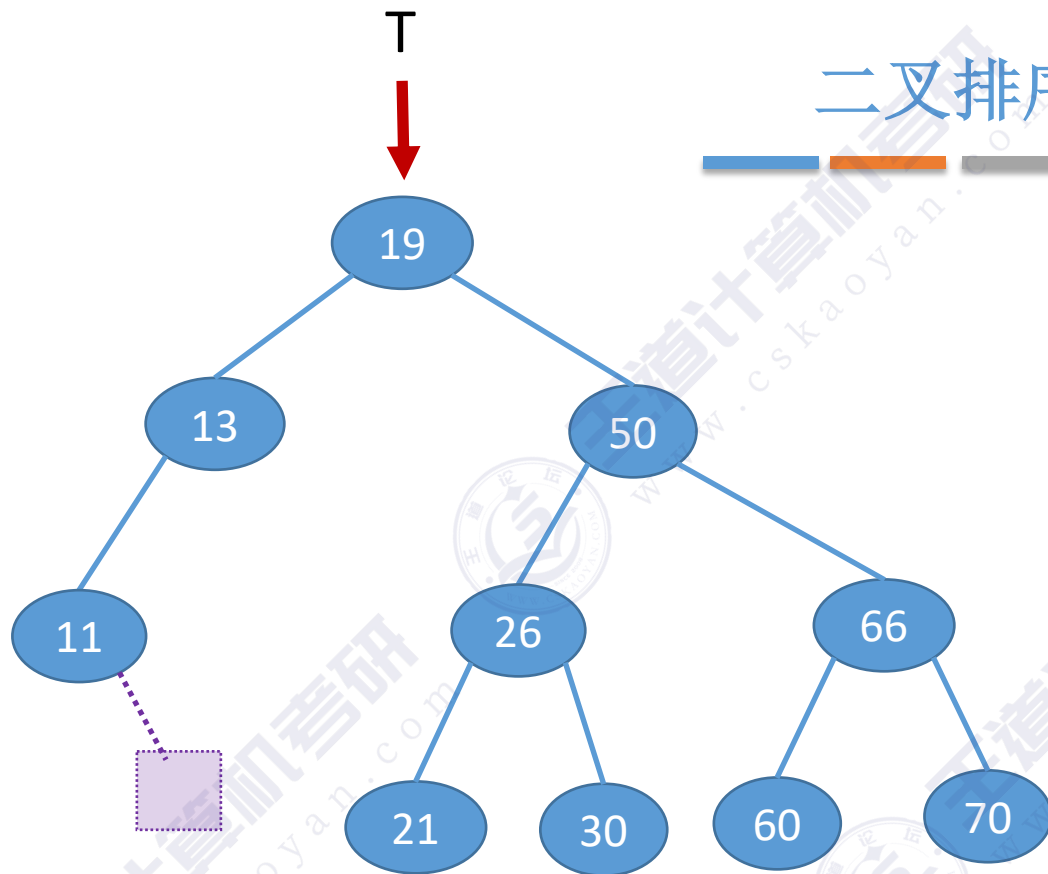
```
}
```

//若树空或等于根结点值，则结束循环

//小于，则在左子树上查找

//大于，则在右子树上查找

二叉排序树的查找



左子树结点值 < 根结点值 < 右子树结点值

若树非空，目标值与根结点的值比较：

若相等，则查找成功；

若小于根结点，则在左子树上查找，否则在右子树上查找。

查找成功，返回结点指针；查找失败返回NULL

例2：查找关键字为12的结点

//二叉排序树结点

```
typedef struct BSTNode{
    int key;
    struct BSTNode *lchild,*rchild;
}BSTNode,*BSTree;
```

//在二叉排序树中查找值为 key 的结点

```
BSTNode *BST_Search(BSTree T,int key){
```

```
    while(T!=NULL&&key!=T->key){
```

```
        if(key<T->key) T=T->lchild;
```

```
        else T=T->rchild;
```

```
    }
```

```
    return T;
```

```
}
```

//若树空或等于根结点值，则结束循环

//小于，则在左子树上查找

//大于，则在右子树上查找

二叉排序树的查找

//在二叉排序树中查找值为 key 的结点

```
BSTNode *BST_Search(BSTree T,int key){
```

```
    while(T!=NULL&&key!=T->key){
```

```
        if(key<T->key) T=T->lchild;
```

```
        else T=T->rchild;
```

```
    }
```

```
    return T;
```

```
}
```

//在二叉排序树中查找值为 key 的结点 (递归实现)

```
BSTNode *BSTSearch(BSTree T,int key){
```

```
    if (T==NULL)
```

```
        return NULL;    //查找失败
```

```
    if (key==T->key)
```

```
        return T;    //查找成功
```

```
    else if (key < T->key)
```

```
        return BSTSearch(T->lchild, key);    //在左子树中找
```

```
    else
```

```
        return BSTSearch(T->rchild, key);    //在右子树中找
```

```
}
```

//若树空或等于根结点值，则结束循环

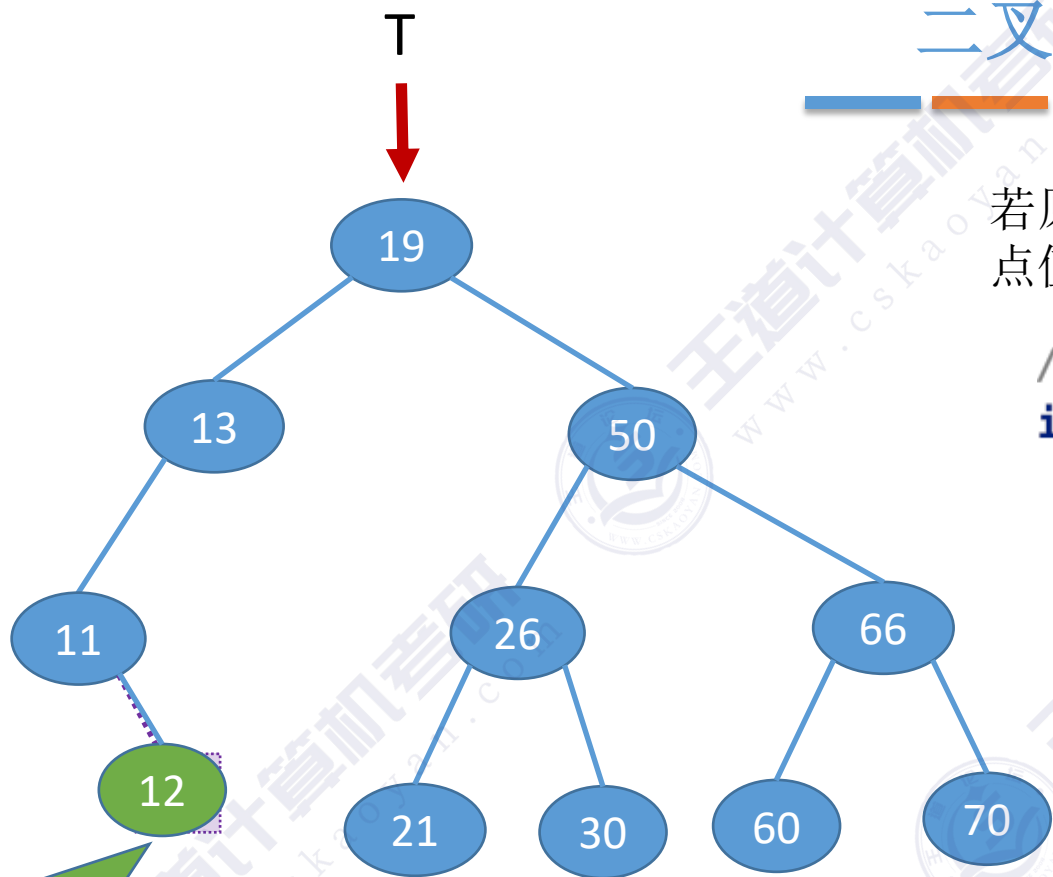
//小于，则在左子树上查找

//大于，则在右子树上查找

最坏空间复杂度 $O(1)$

最坏空间复杂度 $O(h)$

二叉排序树的插入



新插入的结点
一定是叶子

例：插入关键字为12的结点



嗨嗨，醒醒，
敲代码了！

练习：实现非
递归插入

若原二叉排序树为空，则直接插入结点；否则，若关键字 k 小于根结点值，则插入到左子树，若关键字 k 大于根结点值，则插入到右子树

//在二叉排序树插入关键字为 k 的新结点（递归实现）

最坏空间复杂度 $O(h)$

```
int BST_Insert(BSTree &T, int k){
```

```
    if(T==NULL){
```

//原树为空，新插入的结点为根结点

```
        T=(BSTree)malloc(sizeof(BSTNode));
```

```
        T->key=k;
```

```
        T->lchild=T->rchild=NULL;
```

```
        return 1;
```

//返回1，插入成功

```
    }
```

```
    else if(k==T->key)
```

//树中存在相同关键字的结点，插入失败

```
        return 0;
```

```
    else if(k<T->key)
```

//插入到 T 的左子树

```
        return BST_Insert(T->lchild,k);
```

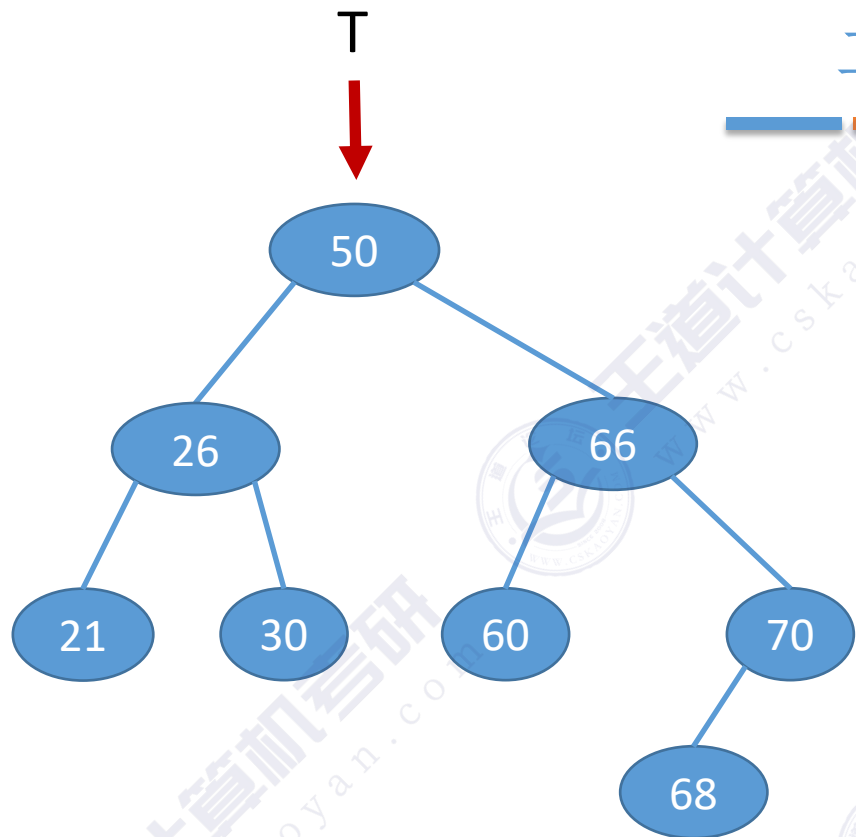
```
    else
```

//插入到 T 的右子树

```
        return BST_Insert(T->rchild,k);
```

```
}
```


二叉排序树的构造



//按照 `str[]` 中的关键字序列建立二叉排序树

```
void Creat_BST(BSTree &T,int str[],int n){
```

```
    T=NULL;           //初始时T为空树
```

```
    int i=0;
```

```
    while(i<n){           //依次将每个关键字插入到二叉排序树中
```

```
        BST_Insert(T,str[i]);
```

```
        i++;
```

```
    }
```

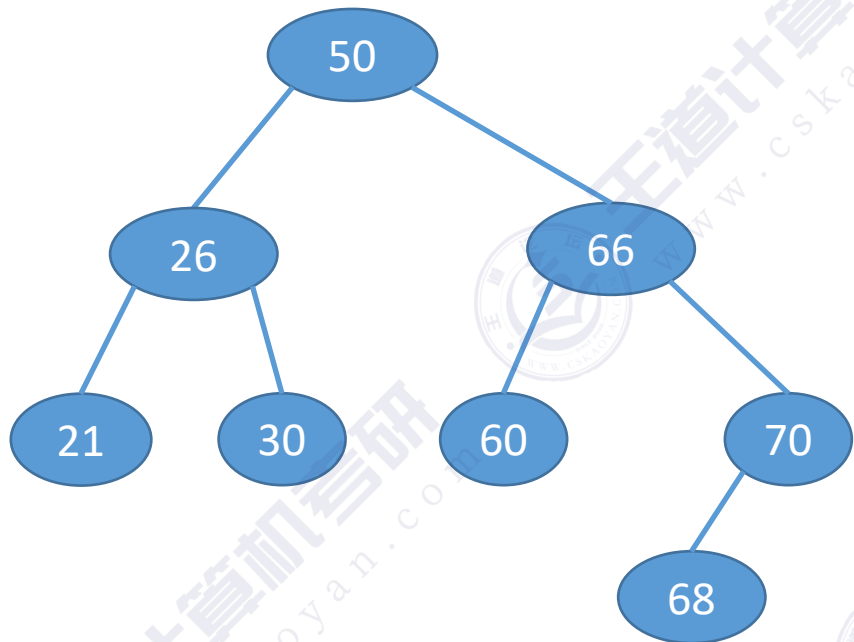
```
}
```

例1: 按照序列`str={50, 66, 60, 26, 21, 30, 70, 68}`建立BST

例2: 按照序列`str={50, 26, 21, 30, 66, 60, 70, 68}`建立BST

不同的关键字序列可能
得到同款二叉排序树

二叉排序树的构造



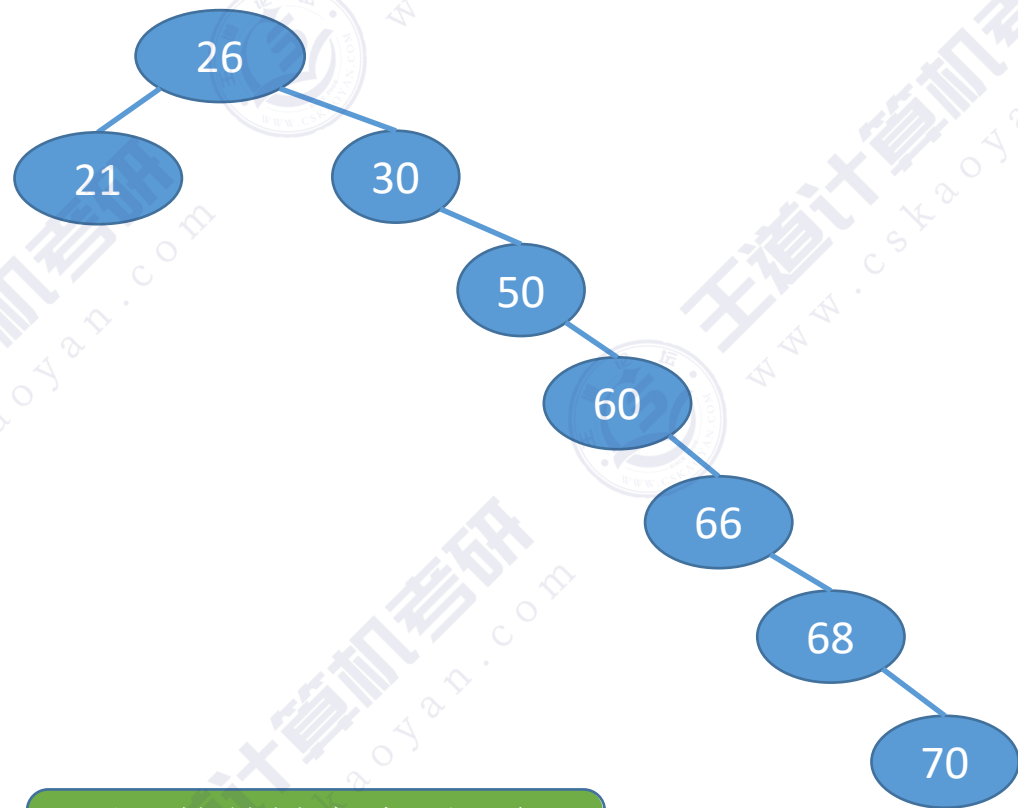
例1: 按照序列 $str=\{50, 66, 60, 26, 21, 30, 70, 68\}$ 建立BST

例2: 按照序列 $str=\{50, 26, 21, 30, 66, 60, 70, 68\}$ 建立BST

例3: 按照序列 $str=\{26, 21, 30, 50, 60, 66, 68, 70\}$ 建立BST

不同的关键字序列可能
得到同款二叉排序树

也可能得到不同款二叉排序树

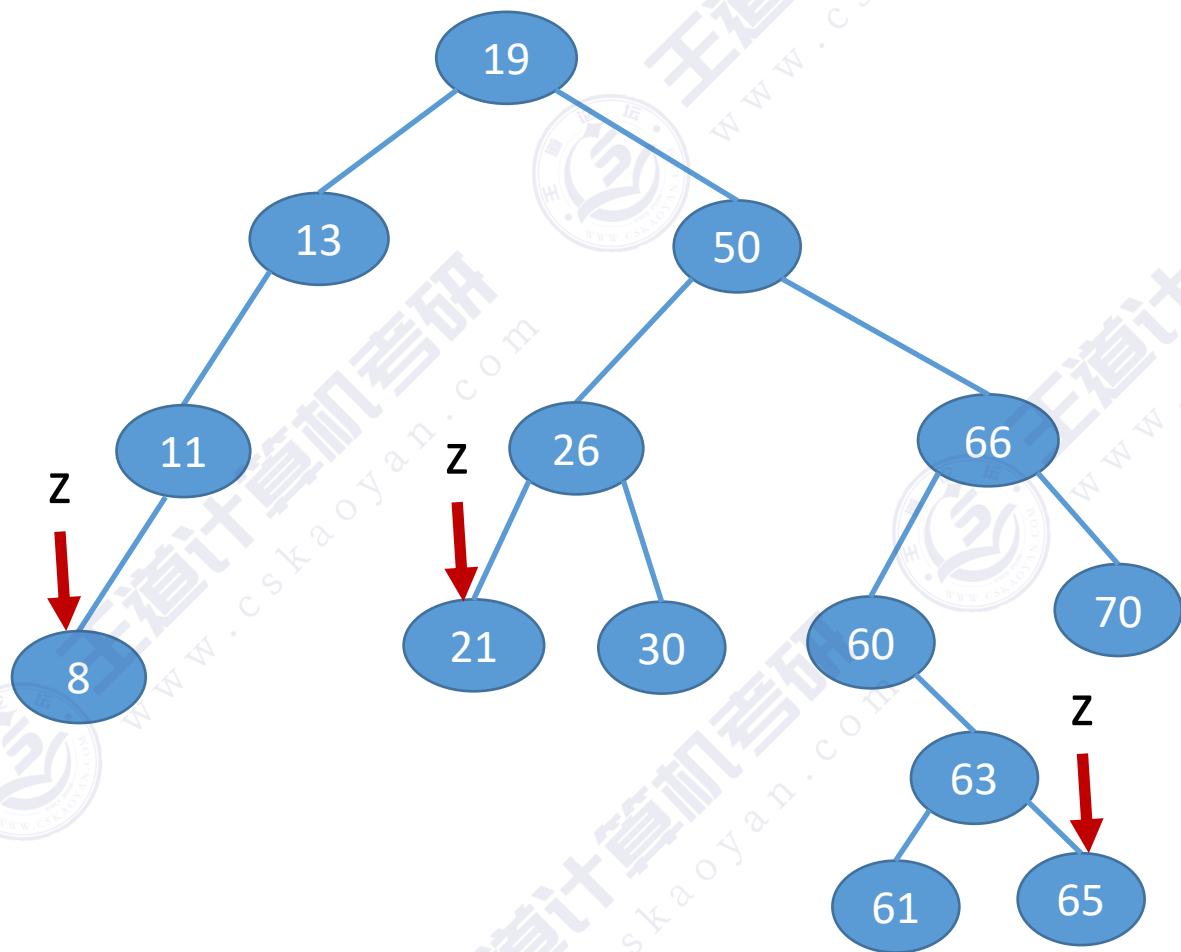


二叉排序树的删除

先搜索找到目标结点：

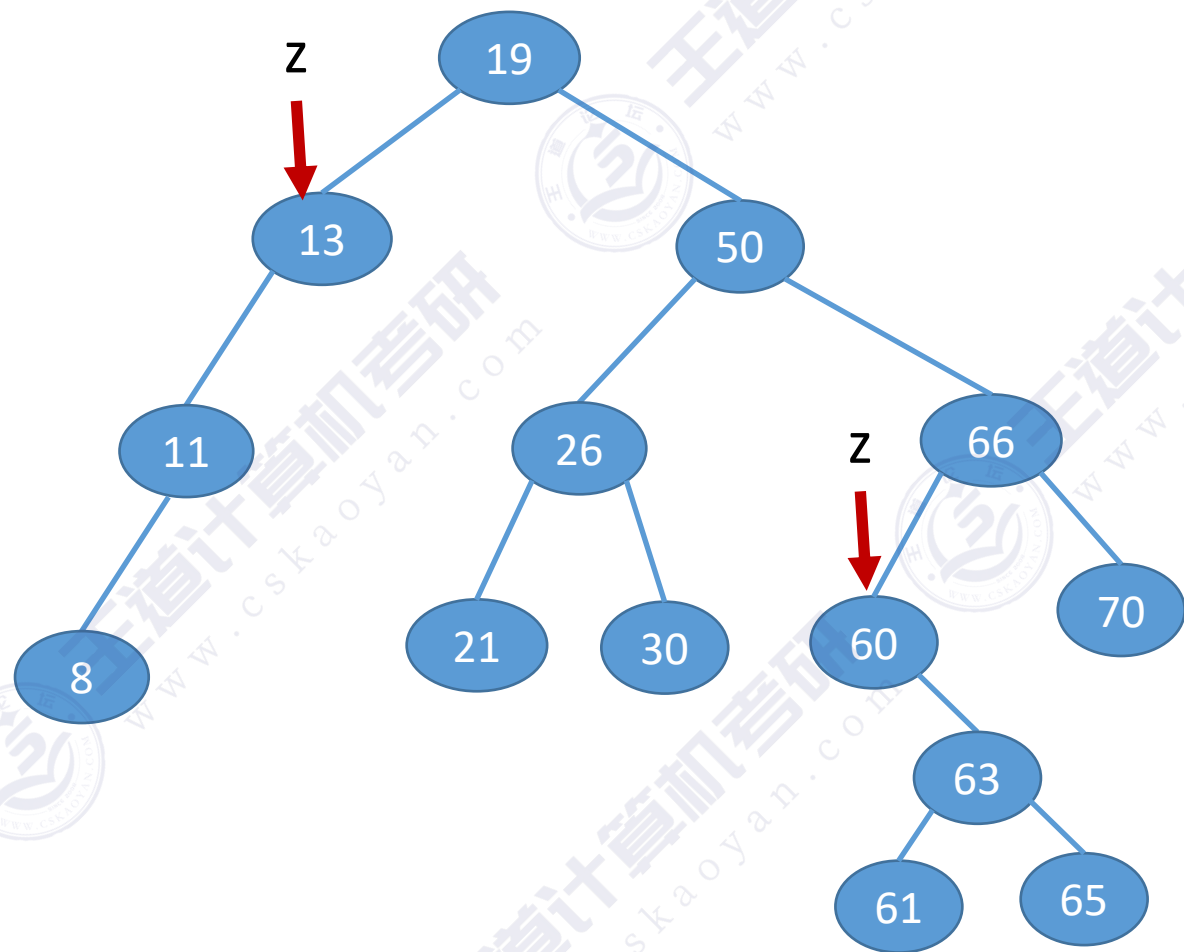
① 若被删除结点 z 是叶结点，则直接删除，不会破坏二叉排序树的性质。

左子树结点值 < 根结点值 < 右子树结点值



二叉排序树的删除

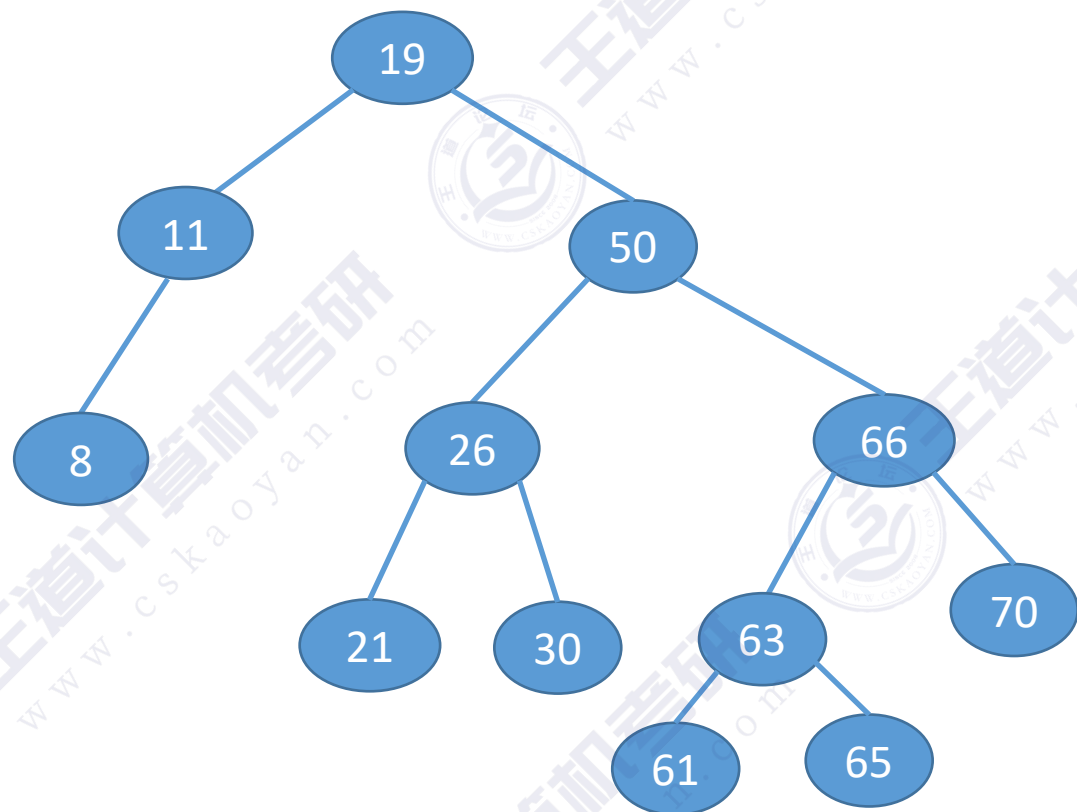
② 若结点 z 只有一棵左子树或右子树，则让 z 的子树成为 z 父结点的子树，替代 z 的位置。



左子树结点值 < 根结点值 < 右子树结点值

二叉排序树的删除

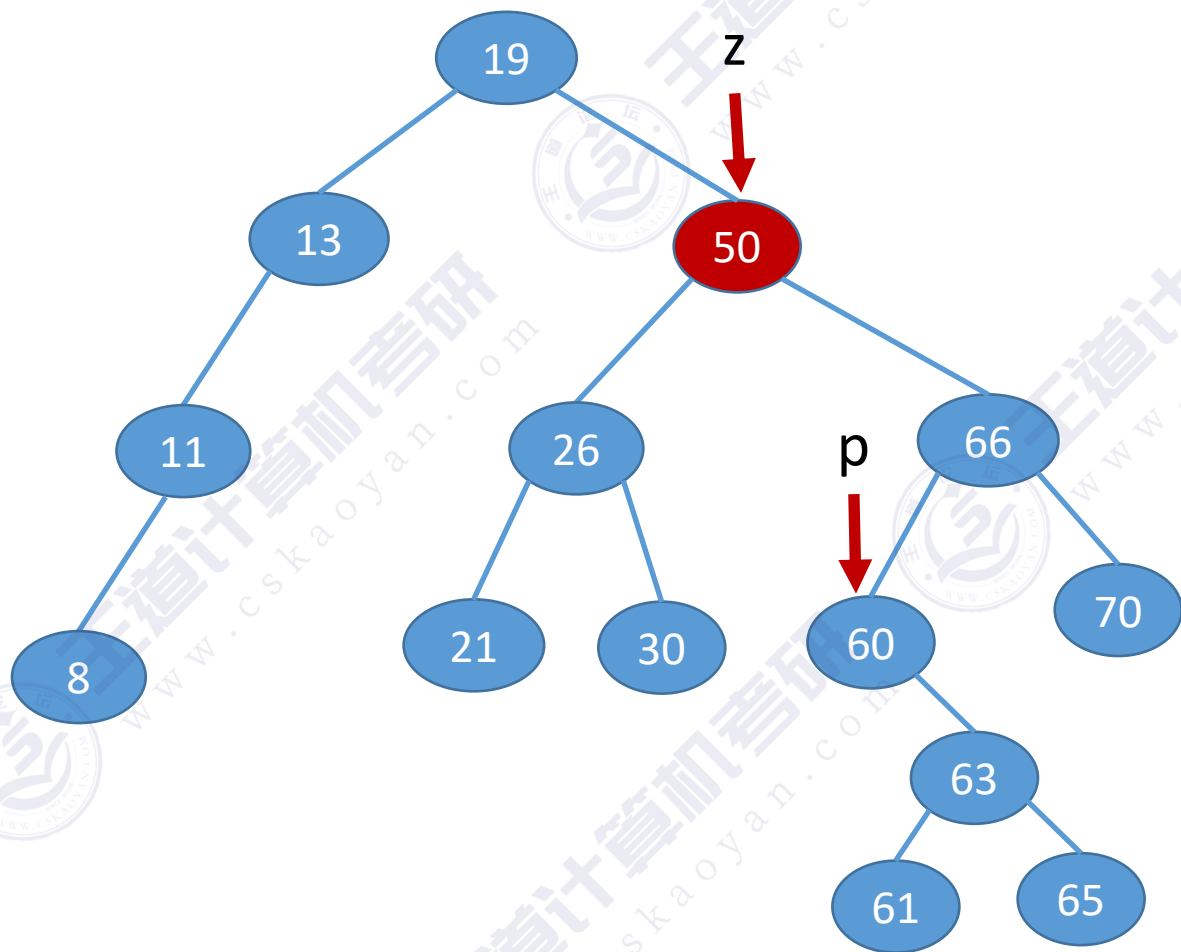
② 若结点 z 只有一棵左子树或右子树，则让 z 的子树成为 z 父结点的子树，替代 z 的位置。



左子树结点值 < 根结点值 < 右子树结点值

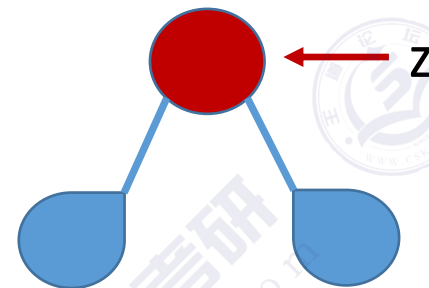
二叉排序树的删除

③ 若结点 z 有左、右两棵子树，则令 z 的直接后继（或直接前驱）替代 z ，然后从二叉排序树中删去这个直接后继（或直接前驱），这样就转换成了第一或第二种情况。



左子树结点值 < 根结点值 < 右子树结点值

进行中序遍历，可以得到一个递增的有序序列



中序遍历——左 根 右

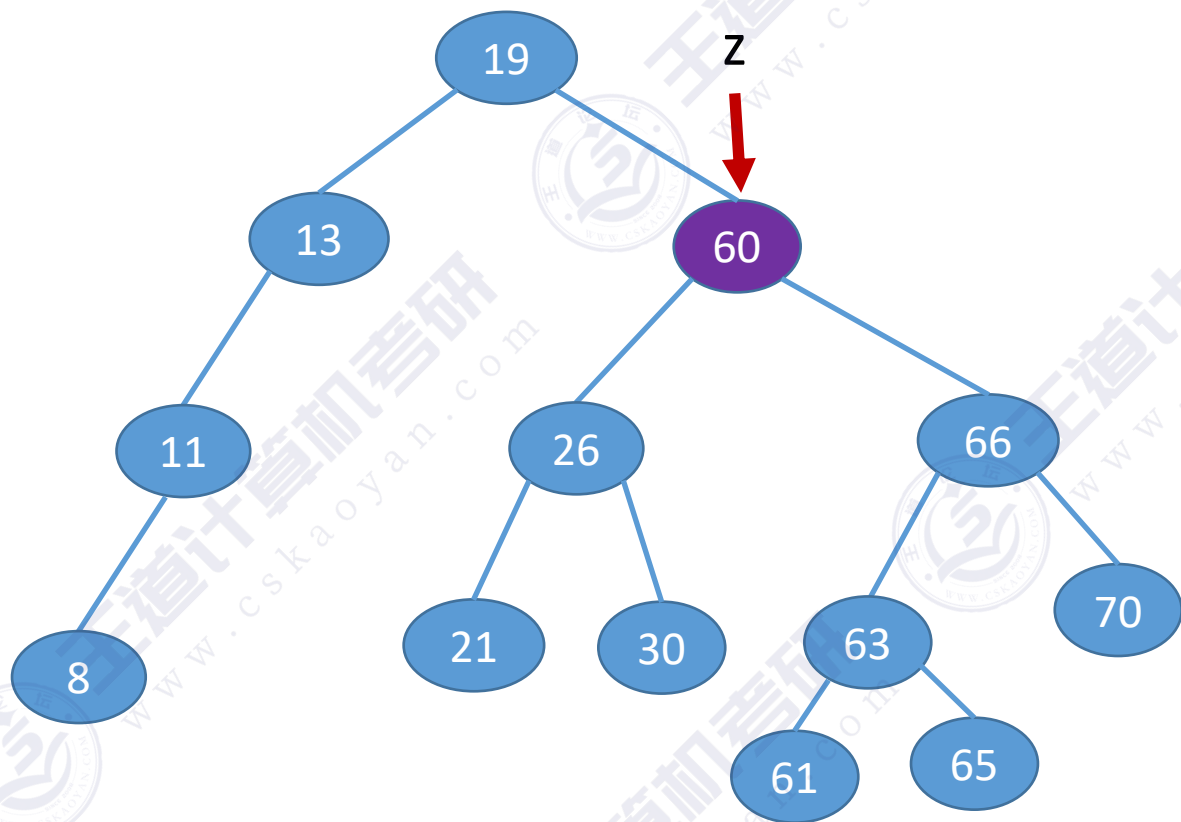
左 根 (左 根 右)

左 根 ((左 根 右) 根 右)

z 的后继： z 的右子树中最左下结点（该结点一定没有左子树）

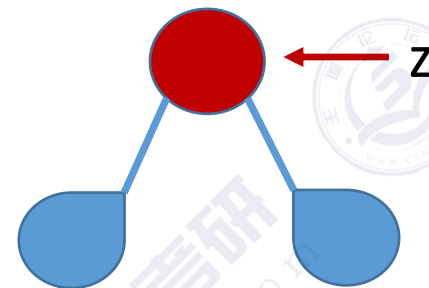
二叉排序树的删除

③ 若结点 z 有左、右两棵子树，则令 z 的直接后继（或直接前驱）替代 z ，然后从二叉排序树中删去这个直接后继（或直接前驱），这样就转换成了第一或第二种情况。



左子树结点值 < 根结点值 < 右子树结点值

进行中序遍历，可以得到一个递增的有序序列



中序遍历——左 根 右

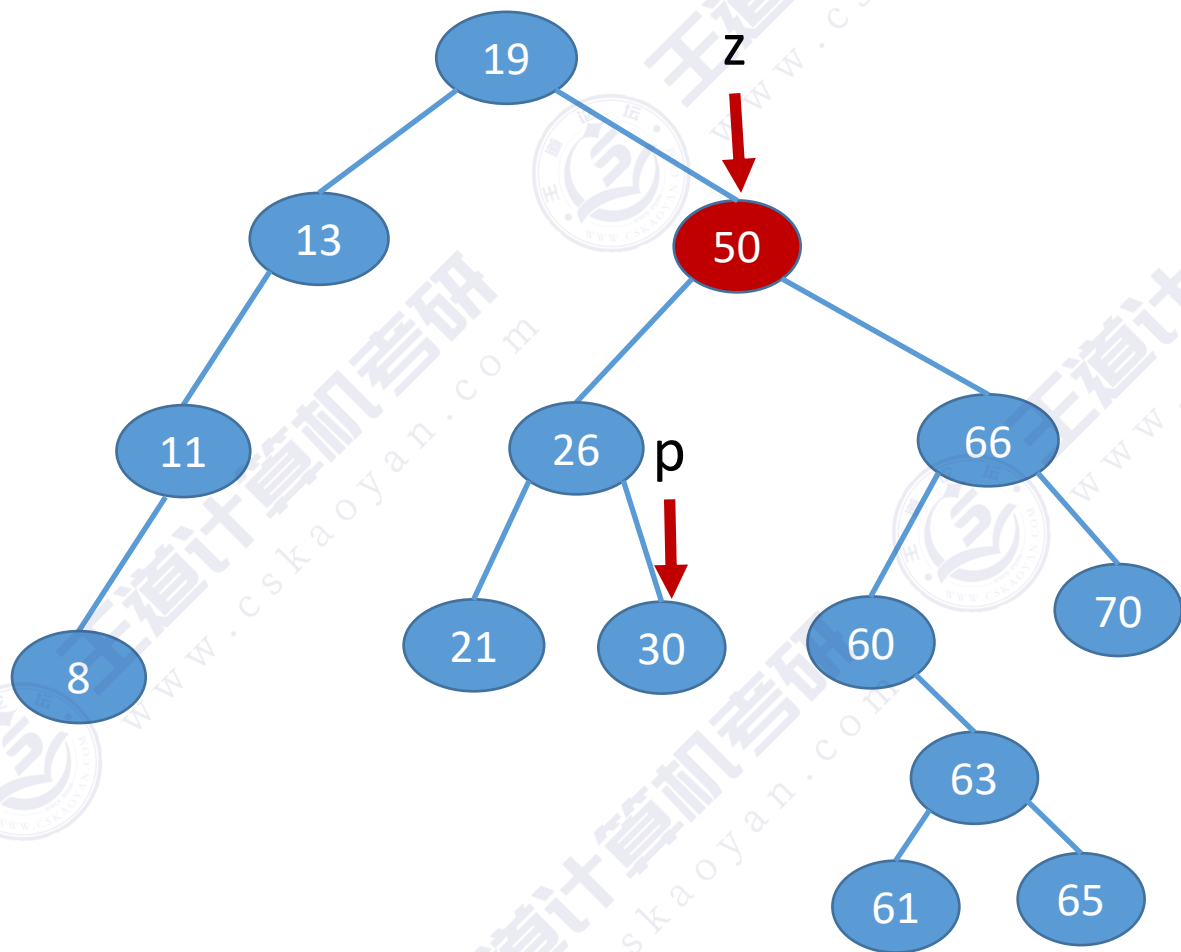
左 根 (左 根 右)

左 根 ((左 根 右) 根 右)

z 的后继： z 的右子树中最左下结点（该结点一定没有左子树）

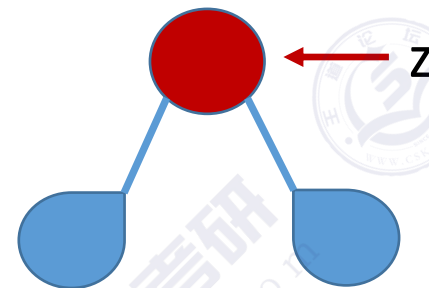
二叉排序树的删除

③ 若结点 z 有左、右两棵子树，则令 z 的直接后继（或直接前驱）替代 z ，然后从二叉排序树中删去这个直接后继（或直接前驱），这样就转换成了第一或第二种情况。



左子树结点值 < 根结点值 < 右子树结点值

进行中序遍历，可以得到一个递增的有序序列



中序遍历——左 根 右

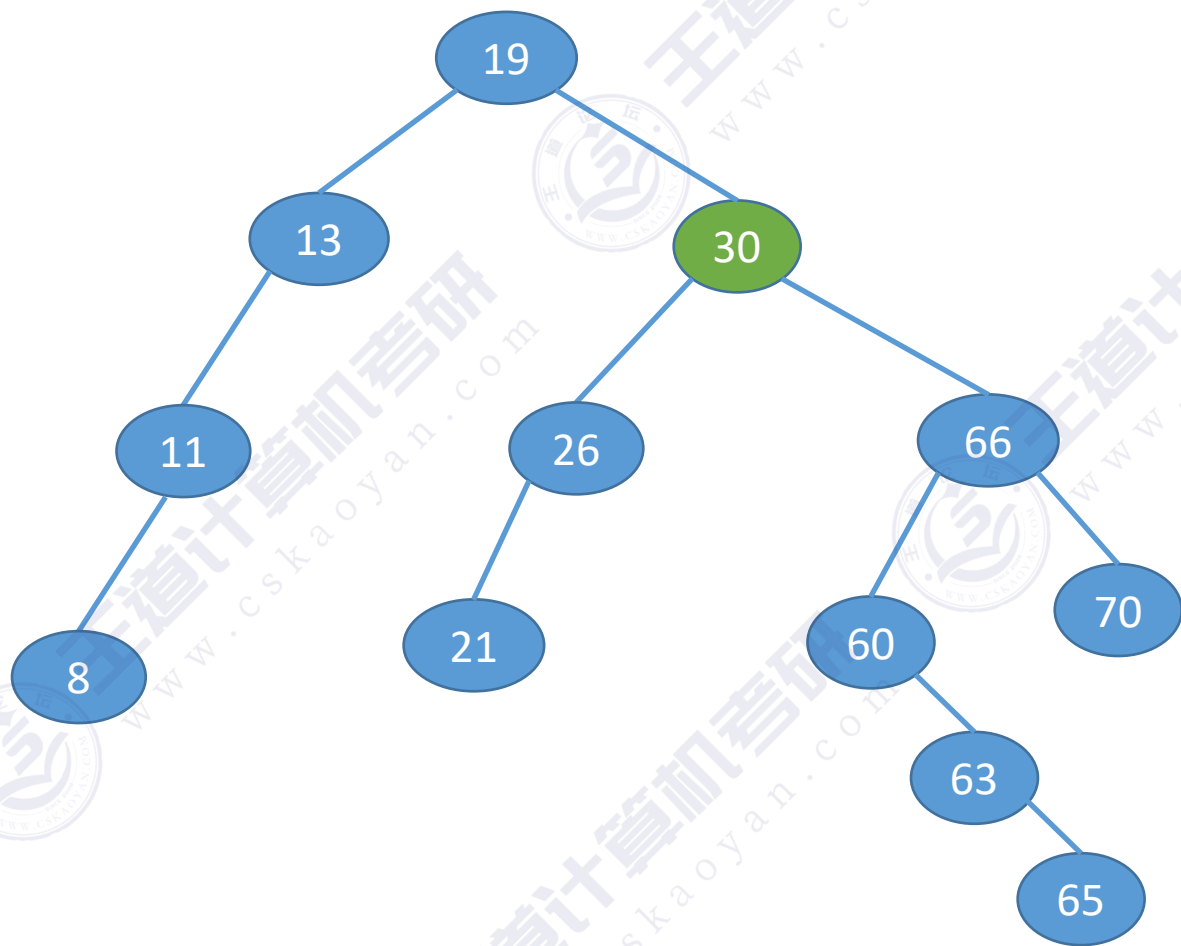
(左 根 右) 根 右

(左 根 (左 根 右)) 根 右

z 的前驱： z 的左子树中最右下结点（该节点一定没有右子树）

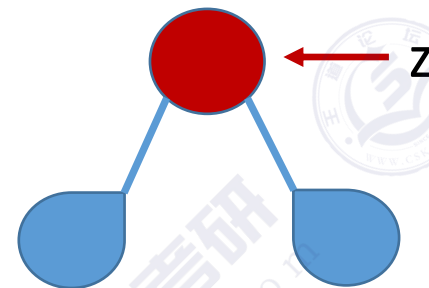
二叉排序树的删除

③ 若结点 z 有左、右两棵子树，则令 z 的直接后继（或直接前驱）替代 z ，然后从二叉排序树中删去这个直接后继（或直接前驱），这样就转换成了第一或第二种情况。



左子树结点值 < 根结点值 < 右子树结点值

进行中序遍历，可以得到一个递增的有序序列



中序遍历——左 根 右

(左 根 右) 根 右

(左 根 (左 根 右)) 根 右

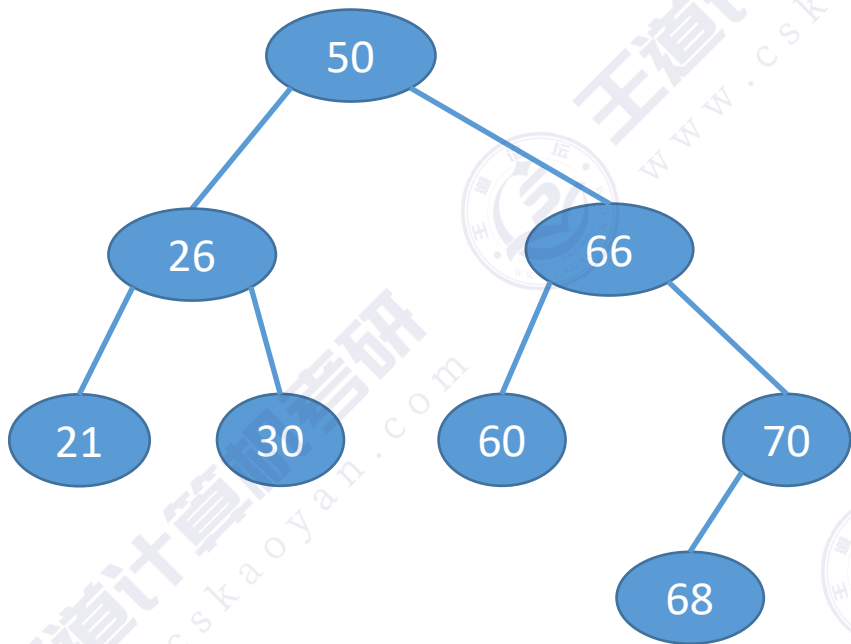
z 的前驱： z 的左子树中最右下结点（该节点一定没有右子树）

若树高 h ，找到最下层的一个结点需要对比 h 次

查找效率分析

最好情况： n 个结点的二叉树最小高度为 $\lfloor \log_2 n \rfloor + 1$ 。
平均查找长度 = $O(\log_2 n)$

查找长度——在查找运算中，需要对比关键字的次数称为查找长度，反映了查找操作时间复杂度



最坏情况：每个结点只有一个分支，树高 h =结点数 n 。平均查找长度 = $O(n)$

查找成功的平均查找长度 ASL (Average Search Length)

$$ASL = (1 \times 1 + 2 \times 2 + 3 \times 4 + 4 \times 1) / 8 = 2.625$$

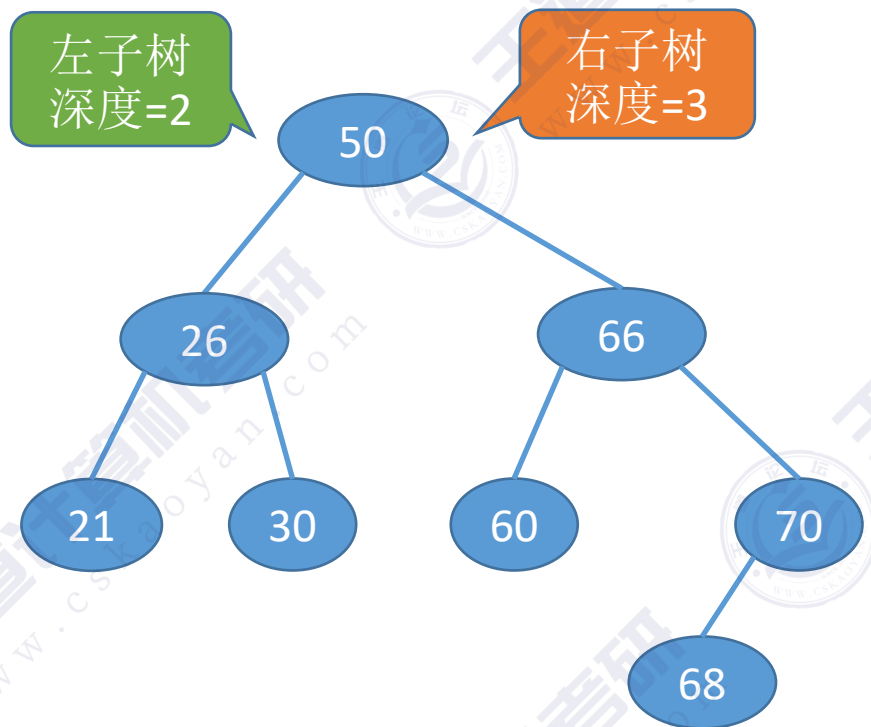
$$ASL = (1 \times 1 + 2 \times 2 + 3 \times 1 + 4 \times 1 + 5 \times 1 + 6 \times 1 + 7 \times 1) / 8 = 3.75$$



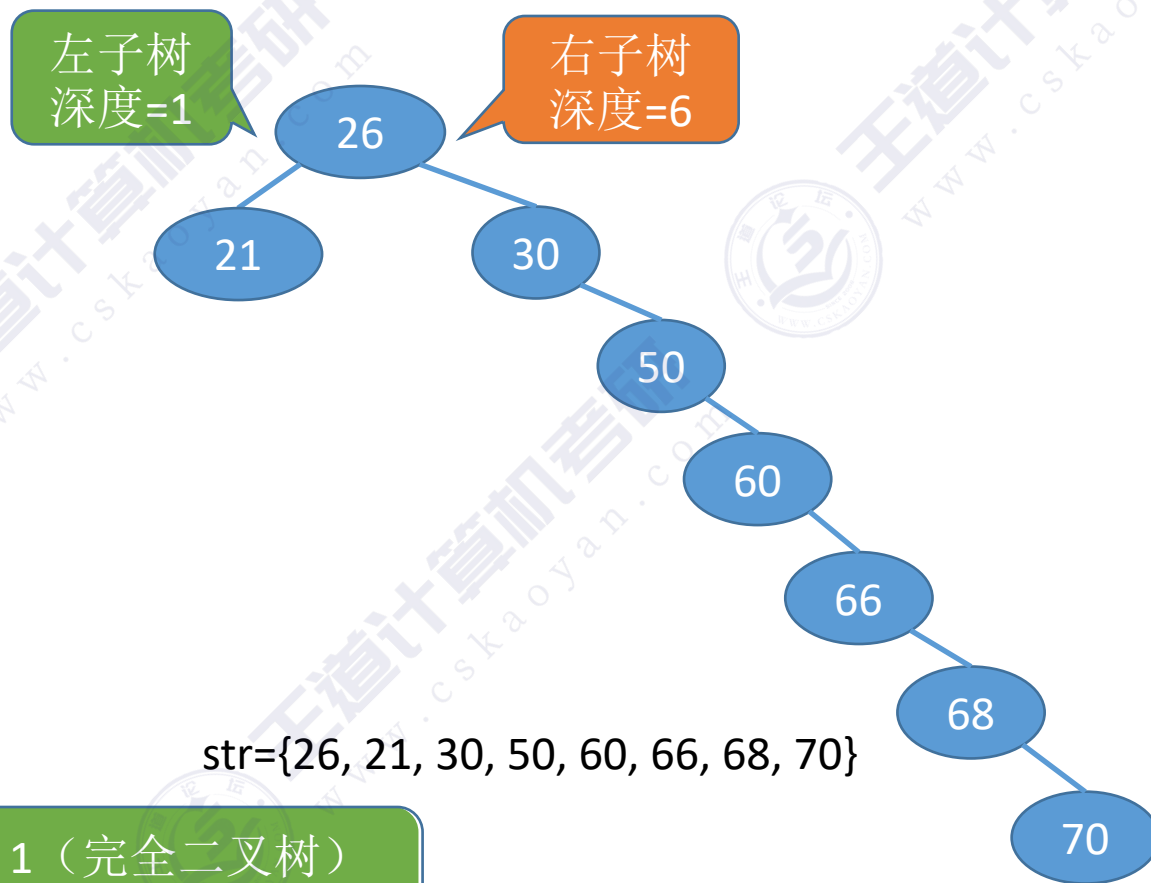
优秀

查找效率分析

平衡二叉树。树上任一结点的左子树和右子树的深度之差不超过1。



str={50, 66, 60, 26, 21, 30, 70, 68}

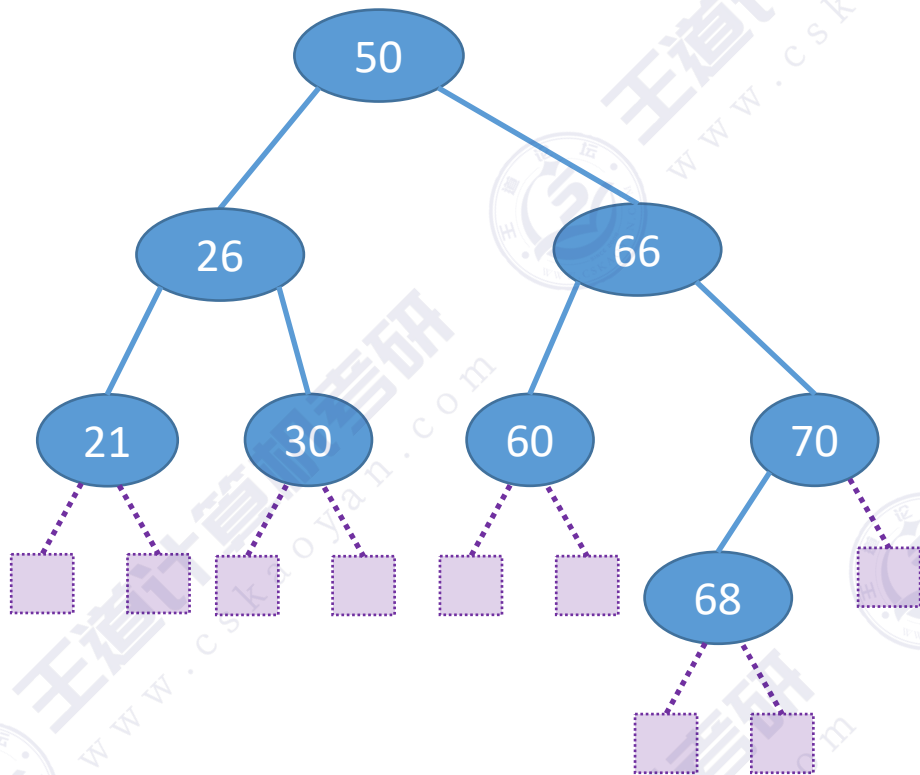


str={26, 21, 30, 50, 60, 66, 68, 70}

n个结点的二叉树最小高度为 $\lfloor \log_2 n \rfloor + 1$ (完全二叉树)
而平衡二叉树高度与完全二叉树同等数量级

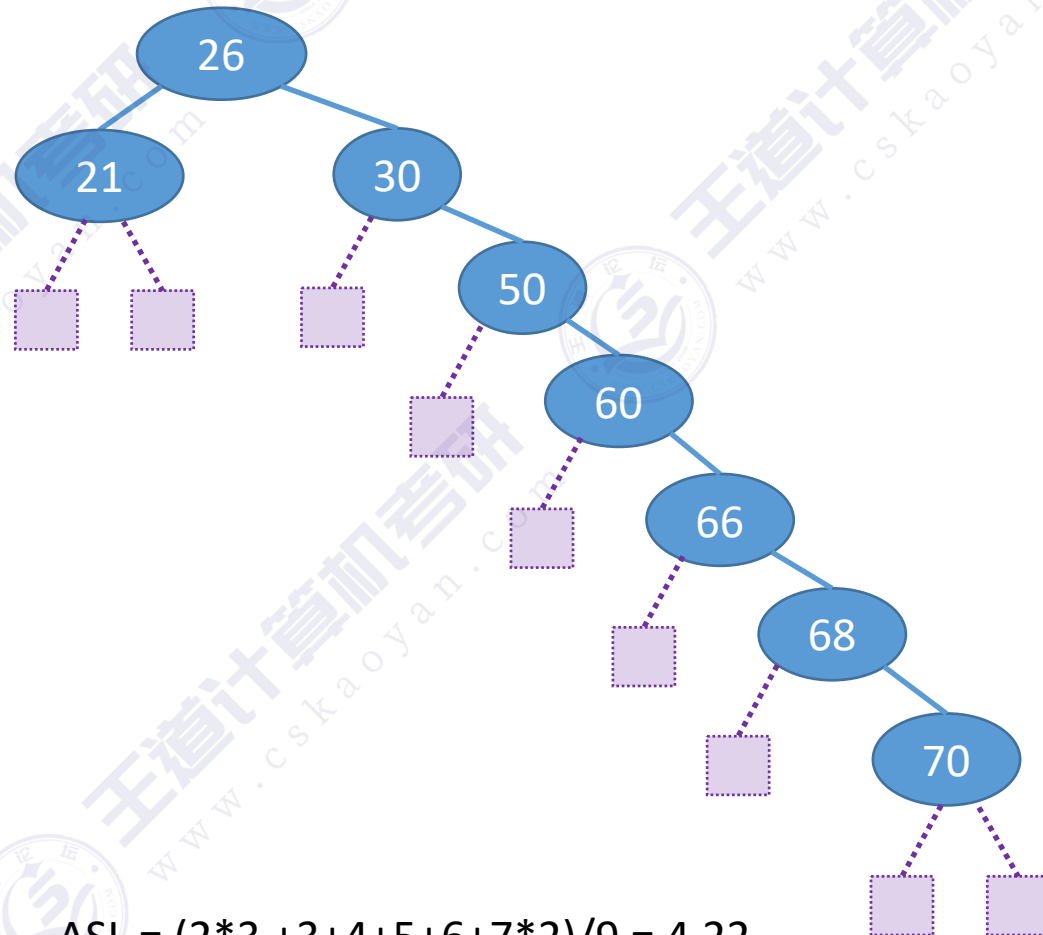
查找效率分析

查找长度——在查找运算中，需要对比关键字的次数称为查找长度。



查找失败的平均查找长度 ASL (Average Search Length)

$$ASL = (3 \times 7 + 4 \times 2) / 9 = 3.22$$



$$ASL = (2 \times 3 + 3 + 4 + 5 + 6 + 7 \times 2) / 9 = 4.22$$

知识回顾与重要考点

二叉排序树

二叉排序树的定义



左子树结点值 < 根结点值 < 右子树结点值

默认不允许两个结点的关键字相同

查找操作

从根节点开始，目标值更小往左找，目标值更大往右找

插入操作

找到应该插入的位置（一定是叶子结点），一定要注意修改其父节点指针



删除操作

①被删结点为叶子，直接删除

②被删结点只有左或只有右子树，用其子树顶替其位置

③被删结点有左、右子树

可用其后继结点顶替，再删除后继结点

或用其前驱结点顶替，再删除前驱结点

前驱：左子树中最右下的结点

后继：右子树中最左下的结点

查找效率分析

取决于树的高度，最好 $O(\log n)$ ，最坏 $O(n)$



平均查找长度的计算

查找成功的情况

查找失败的情况（需补充失败结点）

本节内容

平衡二叉树 (AVL)

知识总览

平衡二叉树

定义

插入操作

插入新结点后如何调整“不平衡”问题

查找效率分析

平衡二叉树的定义

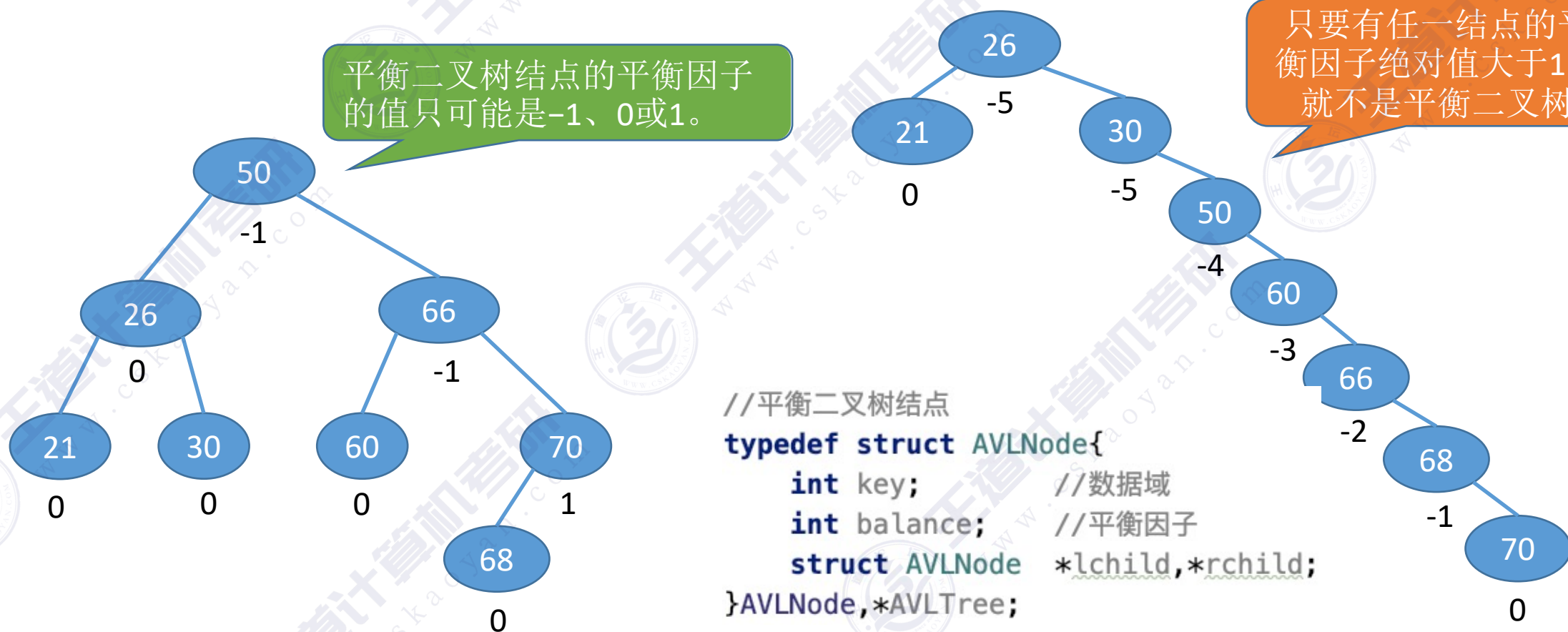
G. M. Adelson-Velsky和
E. M. Landis

平衡二叉树（Balanced Binary Tree），简称平衡树（AVL树）——树上任一结点的左子树和右子树的高度之差不超过1。

结点的平衡因子=左子树高-右子树高。

平衡二叉树结点的平衡因子的值只可能是-1、0或1。

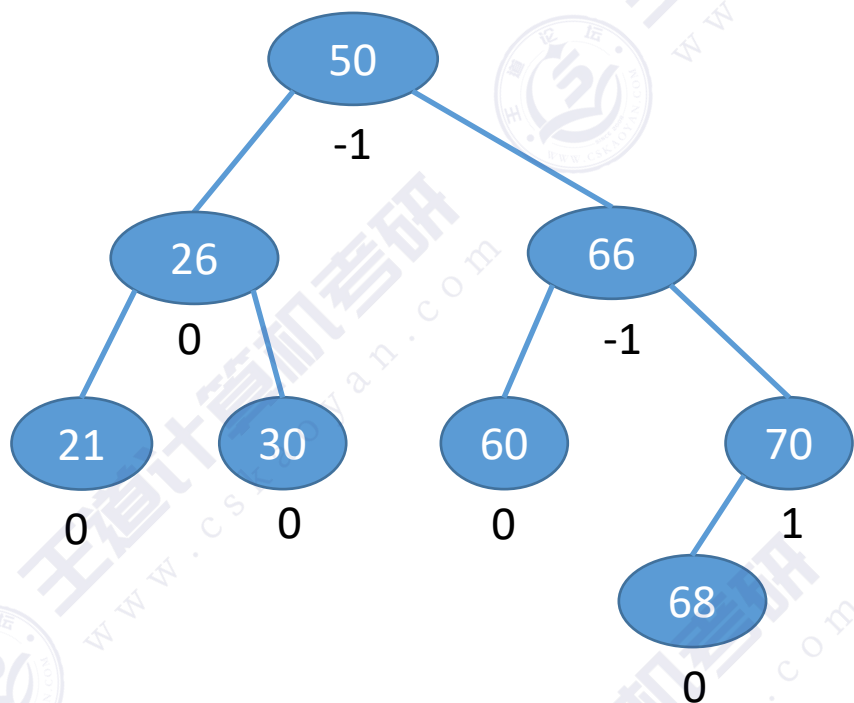
只要有任一结点的平衡因子绝对值大于1，就不是平衡二叉树



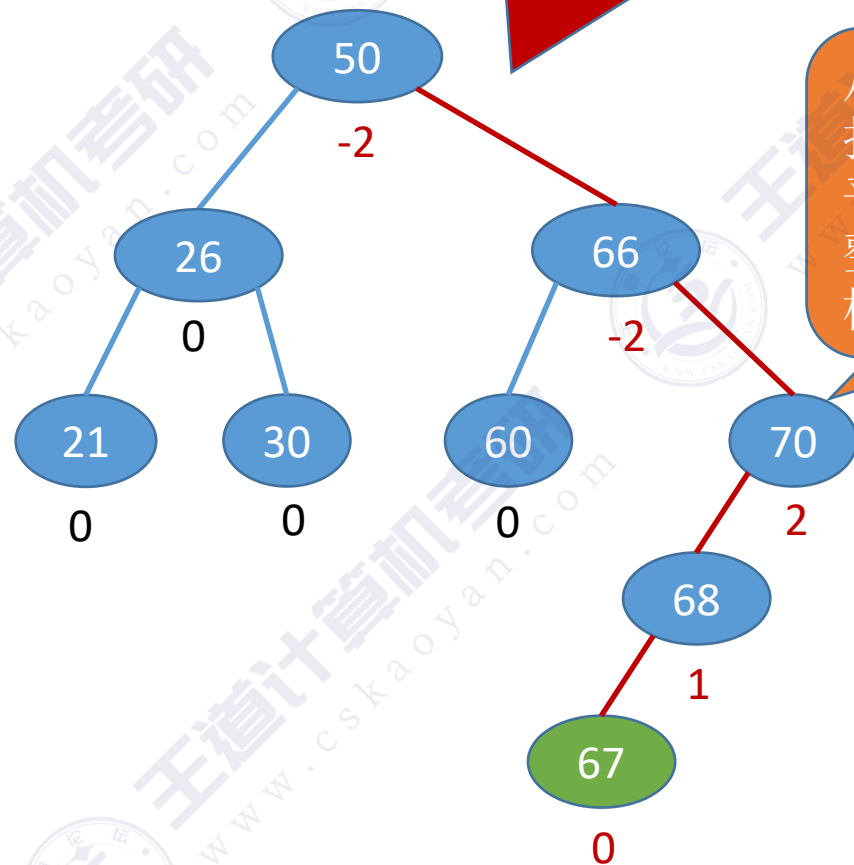
```
//平衡二叉树结点
typedef struct AVLNode{
    int key;           //数据域
    int balance;       //平衡因子
    struct AVLNode *lchild,*rchild;
}AVLNode,*AVLTree;
```

平衡二叉树的插入

在二叉排序树中插入新结点后，如何保持平衡？



插入67

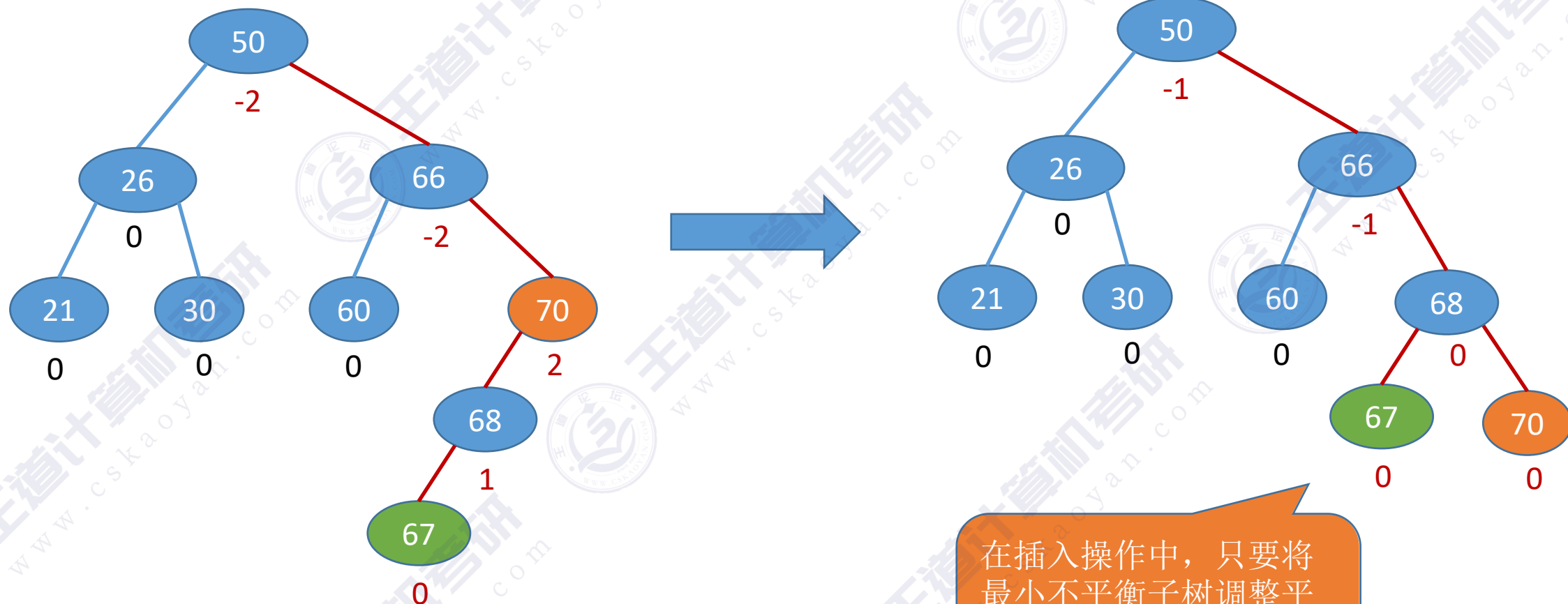


查找路径上的所有结点
都有可能受到影响

从插入点往回
找到第一个不
平衡结点，调
整以该结点为
根的子树

每次调整的对象都是“最小不平衡子树”

平衡二叉树的插入



每次调整的对象都是“最小不平衡子树”

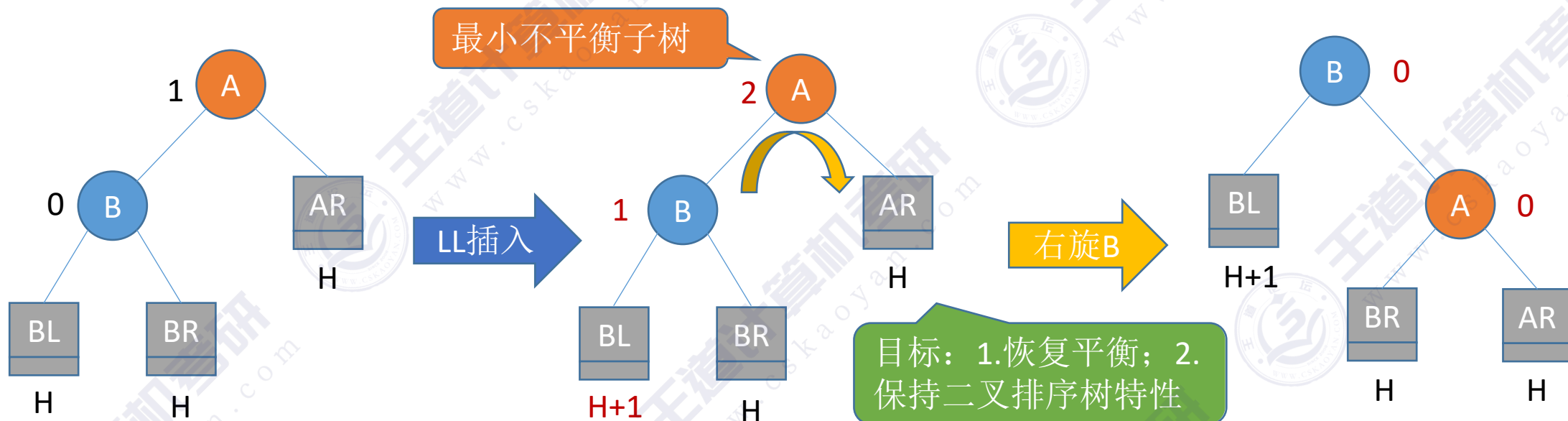
在插入操作中，只要将最小不平衡子树调整平衡，则其他祖先结点都会恢复平衡

调整最小不平衡子树

调整最小不平衡子树A

- LL $\ominus$ 在A的左孩子的左子树中插入导致不平衡
- RR $\ominus$ 在A的右孩子的右子树中插入导致不平衡
- LR $\ominus$ 在A的左孩子的右子树中插入导致不平衡
- RL $\ominus$ 在A的右孩子的左子树中插入导致不平衡

调整最小不平衡子树 (LL)



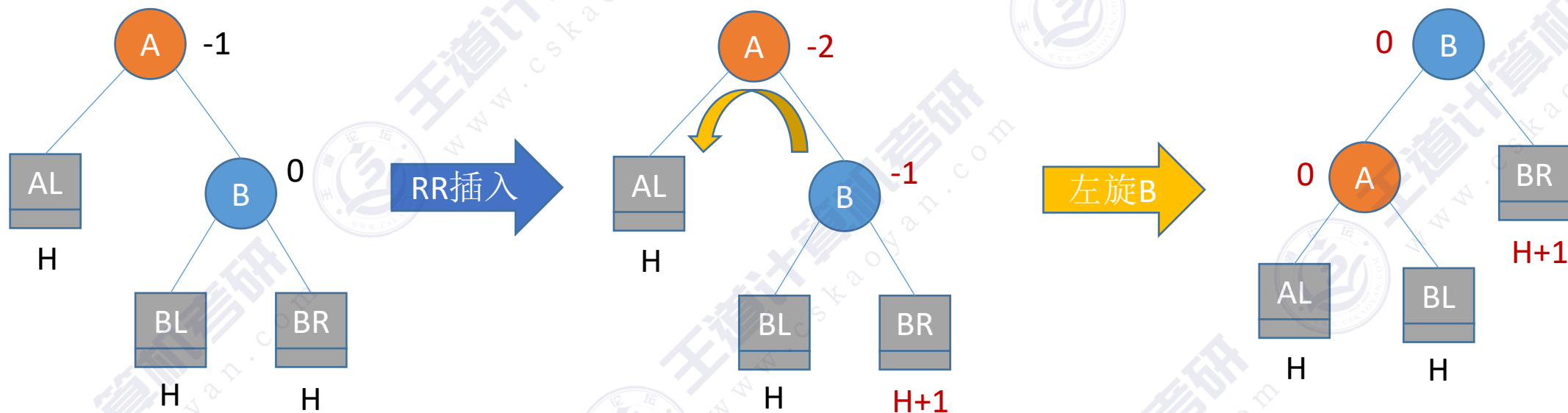
思考: 为什么要假定所有子树的高度都是H?

二叉排序树的特性: 左子树结点值 < 根结点值 < 右子树结点值

$BL < B < BR < A < AR$

1) LL平衡旋转 (右单旋转)。由于在结点A的左孩子 (L) 的左子树 (L) 上插入了新结点, A的平衡因子由1增至2, 导致以A为根的子树失去平衡, 需要一次向右的旋转操作。将A的左孩子B向右旋转代替A成为根结点, 将A结点向右下旋转成为B的右子树的根结点, 而B的原右子树则作为A结点的左子树。

调整最小不平衡子树 (RR)



二叉排序树的特性：左子树结点值 < 根结点值 < 右子树结点值

$$AL < A < BL < B < BR$$

2) RR平衡旋转 (左单旋转)。由于在结点A的右孩子 (R) 的右子树 (R) 上插入了新结点, A的平衡因子由-1减至-2, 导致以A为根的子树失去平衡, 需要一次向左的旋转操作。将A的右孩子B向左上旋转代替A成为根结点, 将A结点向左下旋转成为B的左子树的根结点, 而B的原左子树则作为A结点的右子树

代码思路

实现 **f** 向右下旋转，**p** 向右上旋转：
其中 f 是爹，p 为左孩子，gf 为 f 他爹

- ① $f \rightarrow lchild = p \rightarrow rchild;$
- ② $p \rightarrow rchild = f;$
- ③ $gf \rightarrow lchild/rchild = p;$

$BL < B < BR < A < AR$

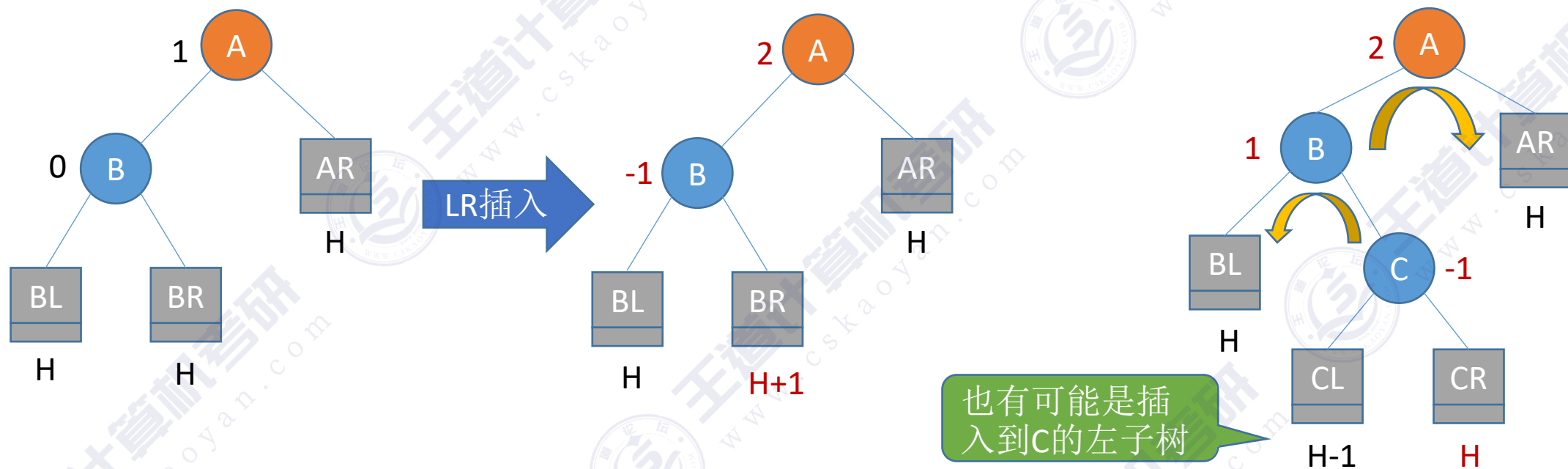
左旋、右旋操作后可以
保持二叉排序树的特性

实现 **f** 向左下旋转，**p** 向左上旋转：
其中 f 是爹，p 为右孩子，gf 为 f 他爹

- ① $f \rightarrow rchild = p \rightarrow lchild;$
- ② $p \rightarrow lchild = f;$
- ③ $gf \rightarrow lchild/rchild = p;$

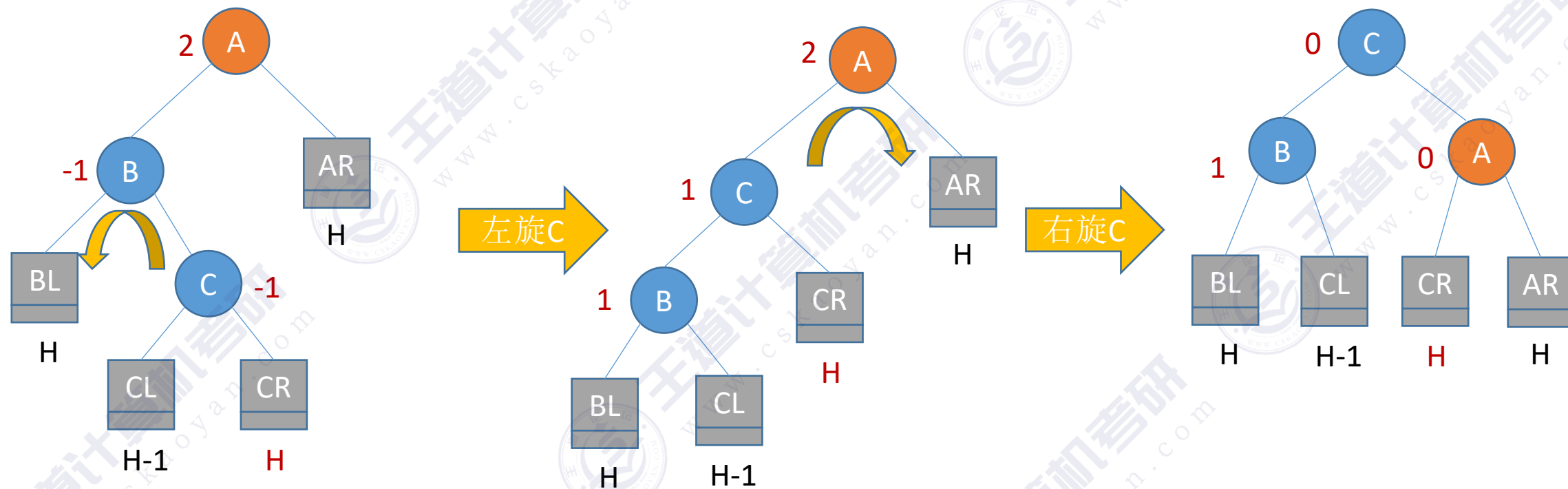
$AL < A < BL < B < BR$

调整最小不平衡子树 (LR)



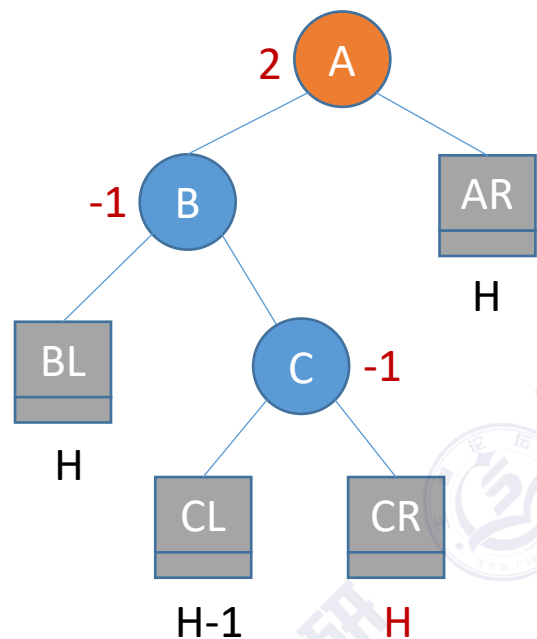
3) LR平衡旋转 (先左后右双旋转)。由于在A的左孩子 (L) 的右子树 (R) 上插入新结点, A的平衡因子由1增至2, 导致以A为根的子树失去平衡, 需要进行两次旋转操作, 先左旋转后右旋转。先将A结点的左孩子B的右子树的根结点C向左上旋转提升到B结点的位置, 然后再把该C结点向右上旋转提升到A结点的位置

调整最小不平衡子树 (LR)

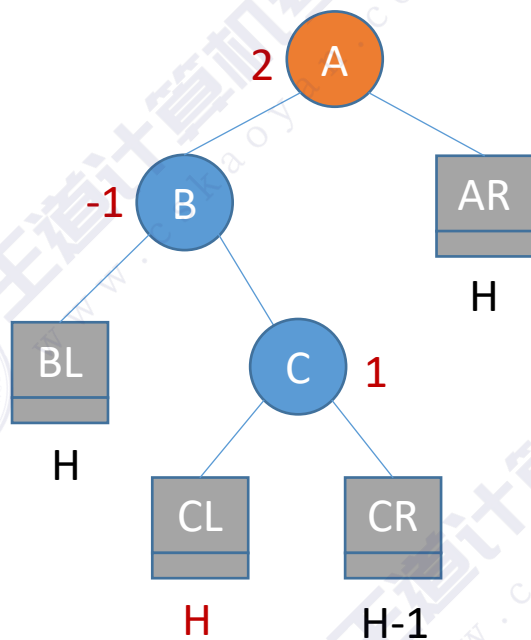
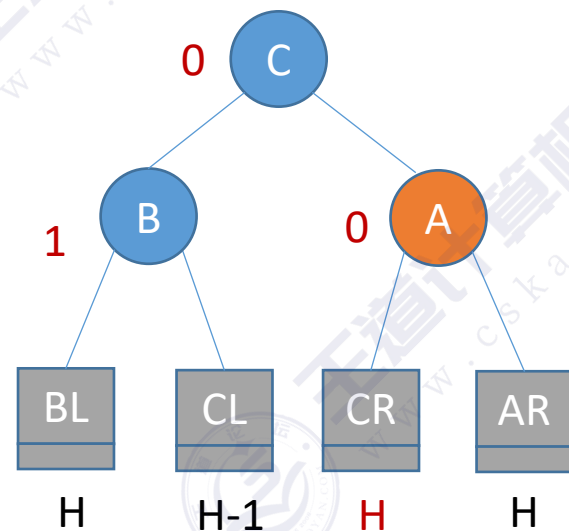


$BL < B < CL < C < CR < A < AR$

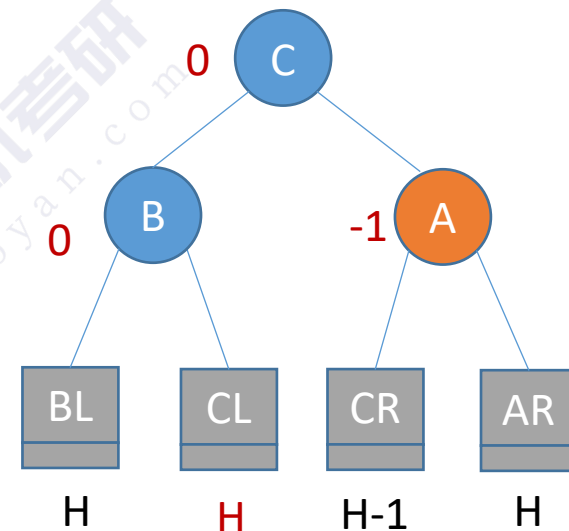
调整最小不平衡子树 (LR)



左旋C+右旋C

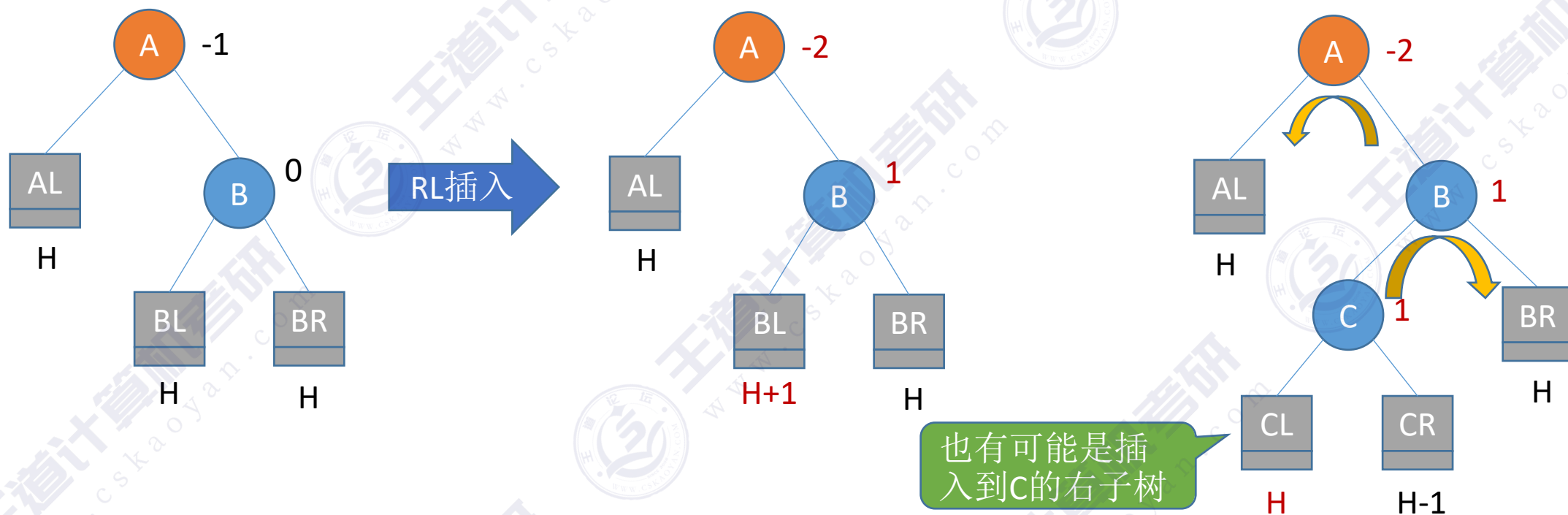


左旋C+右旋C



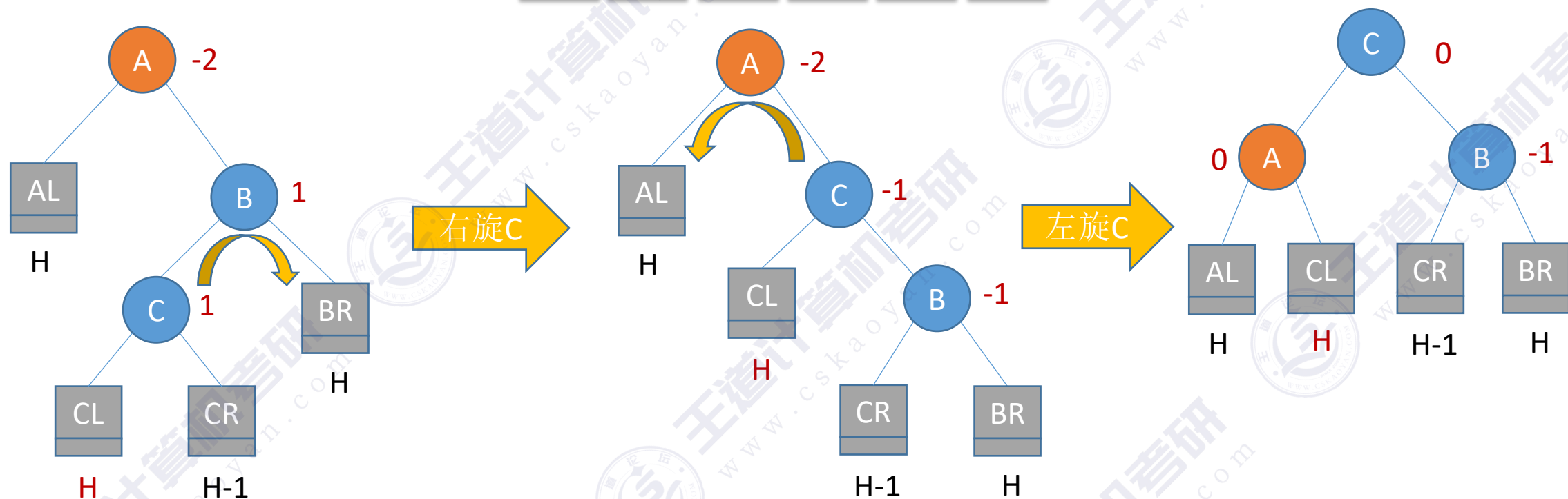
调整最小不平衡子树 (RL)

AL < A < CL < C < CR < B < BR



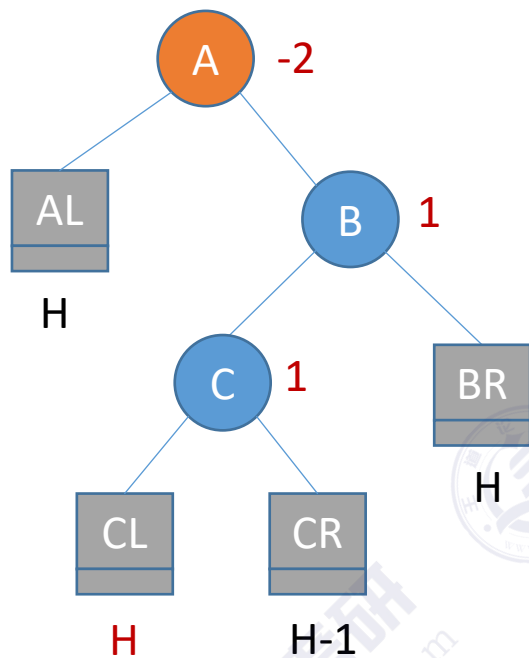
4) RL平衡旋转（先右后左双旋转）。由于在A的右孩子（R）的左子树（L）上插入新结点，A的平衡因子由-1减至-2，导致以A为根的子树失去平衡，需要进行两次旋转操作，先右旋转后左旋转。先将A结点的右孩子B的左子树的根结点C向右上旋转提升到B结点的位置，然后再把该C结点向左上旋转提升到A结点的位置

调整最小不平衡子树 (RL)

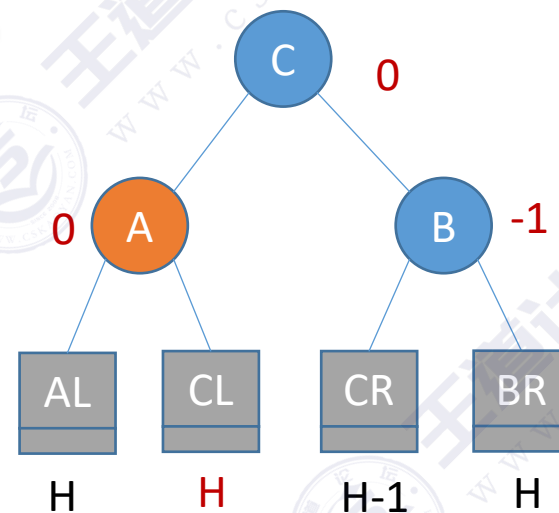


AL<A<CL<C<CR<B<BR

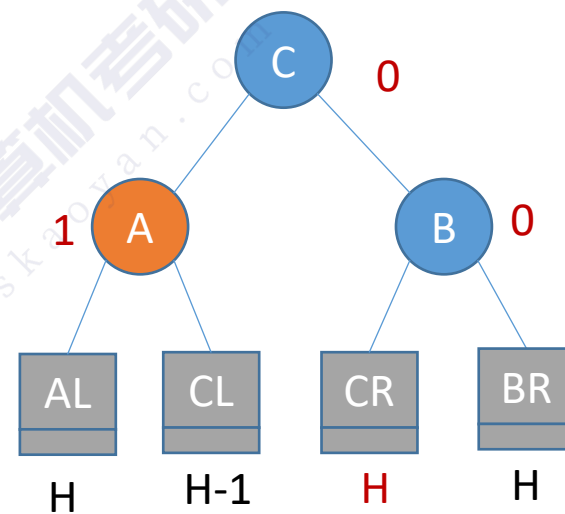
调整最小不平衡子树 (RL)



右旋C+左旋C



右旋C+左旋C



调整最小不平衡子树

只有左孩子
才能右上旋

实现 f 向右下旋转，p 向右上旋转：
其中 f 是爹，p 为左孩子，gf 为 f 他爹

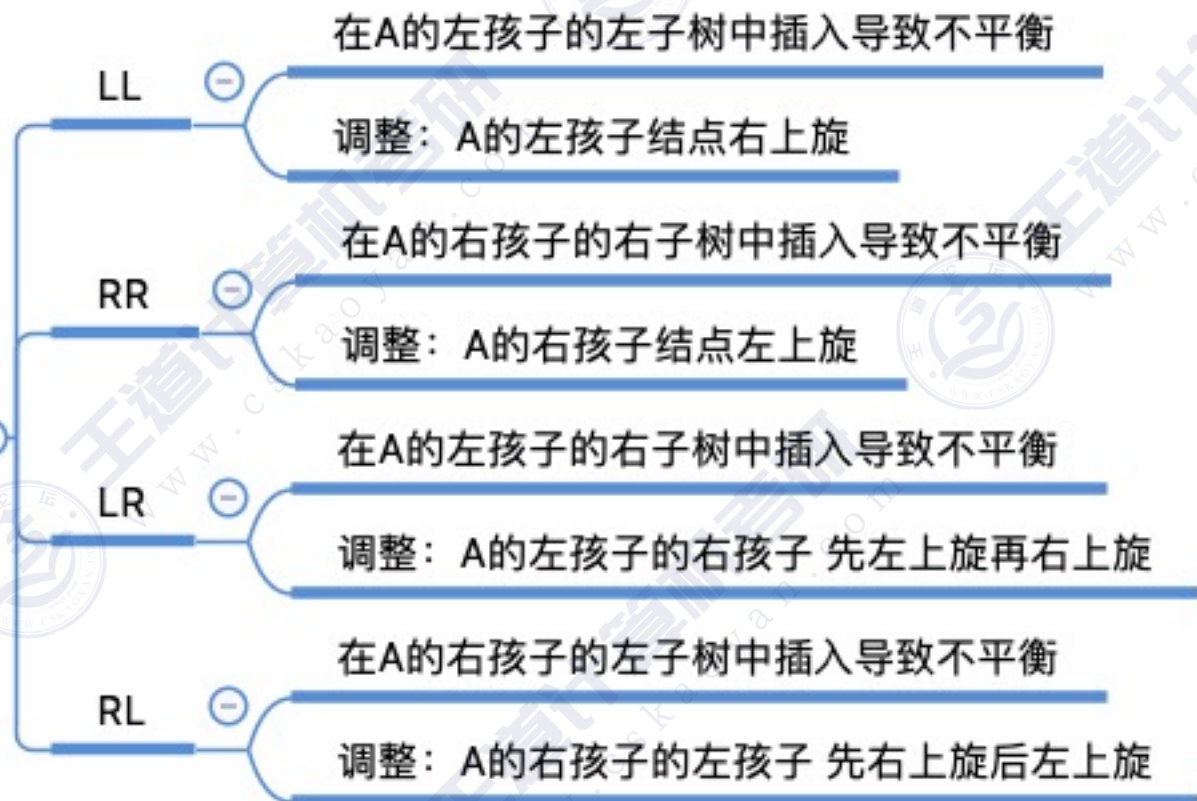
- ① $f \rightarrow lchild = p \rightarrow rchild;$
- ② $p \rightarrow rchild = f;$
- ③ $gf \rightarrow lchild/rchild = p;$

调整最小不平衡子树A

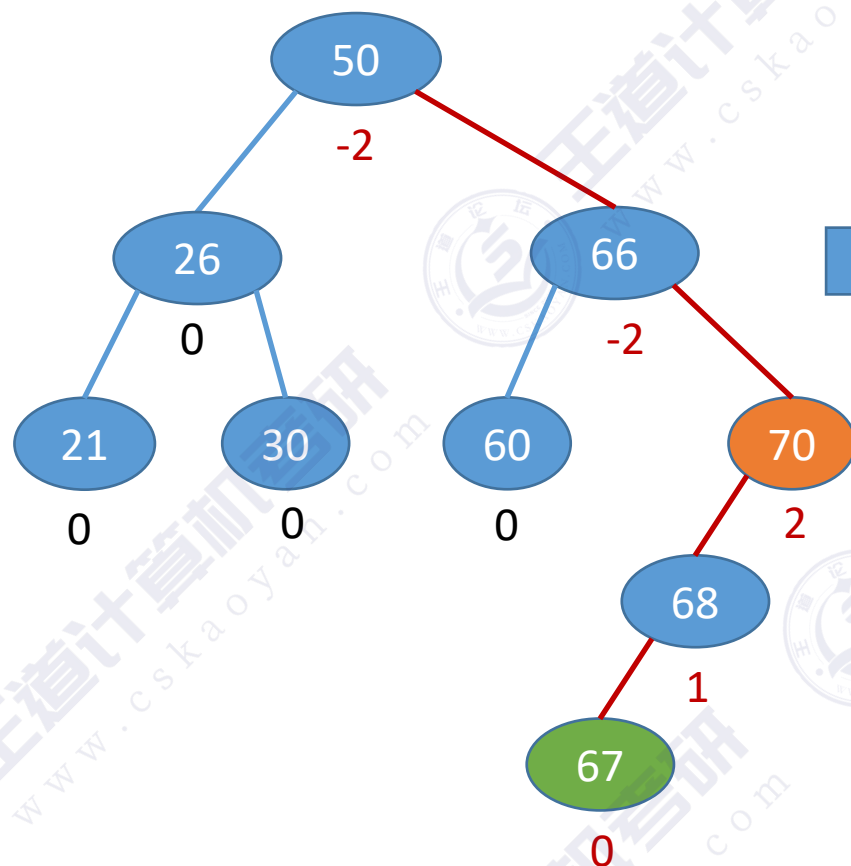
实现 f 向左下旋转，p 向左上旋转：
其中 f 是爹，p 为右孩子，gf 为 f 他爹

- ① $f \rightarrow rchild = p \rightarrow lchild;$
- ② $p \rightarrow lchild = f;$
- ③ $gf \rightarrow lchild/rchild = p;$

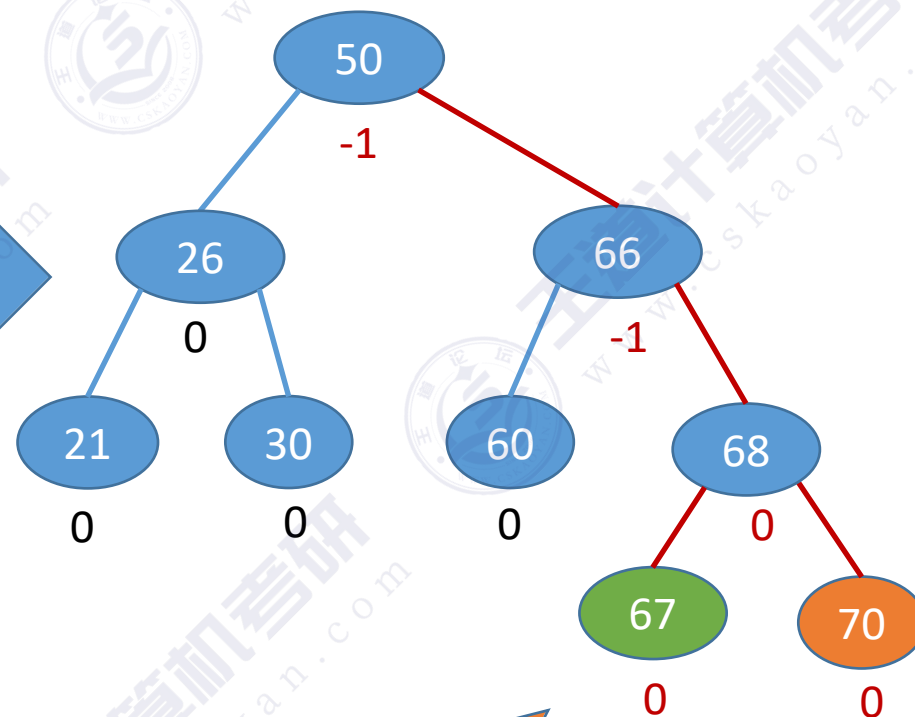
只有右孩子
才能左上旋



填个坑



LL型，左孩子右旋



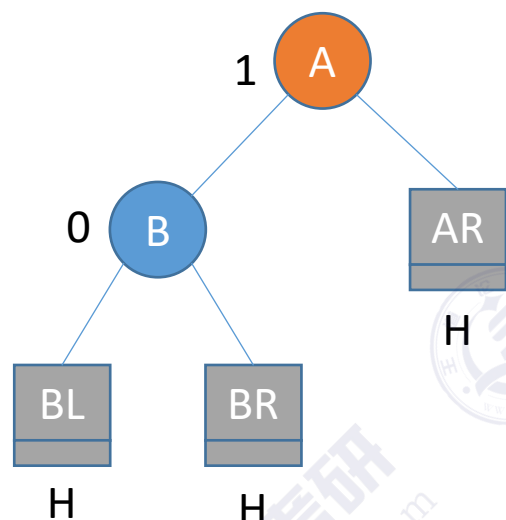
每次调整的对象都是“最小不平衡子树”



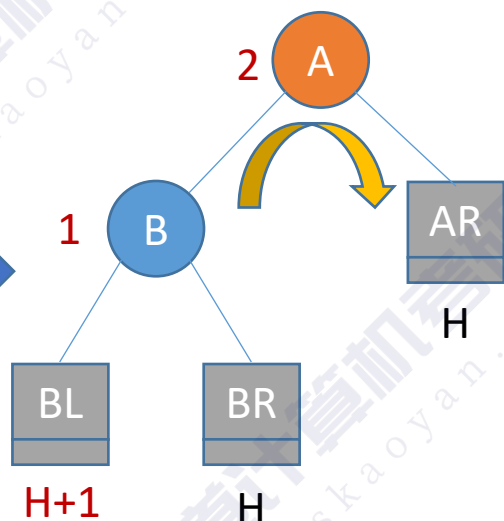
WHY?

在插入操作中，只要将最小不平衡子树调整平衡，则其他祖先结点都会恢复平衡

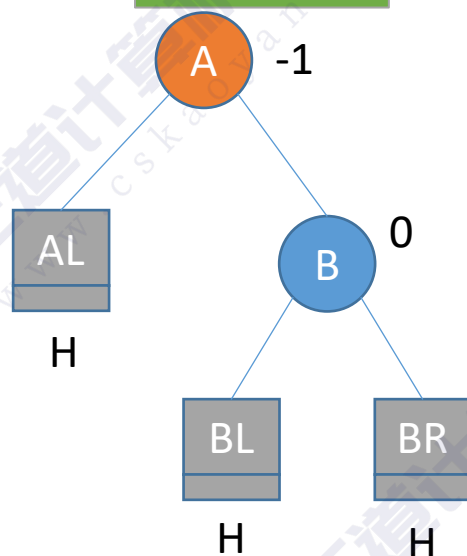
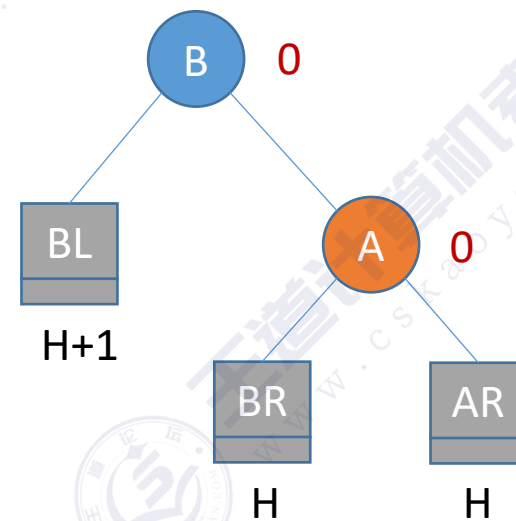
填个坑



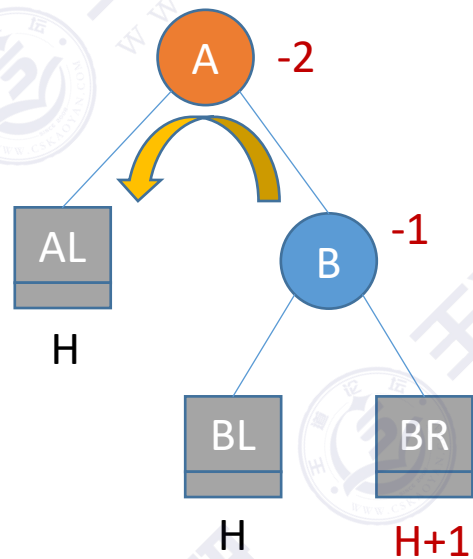
LL型



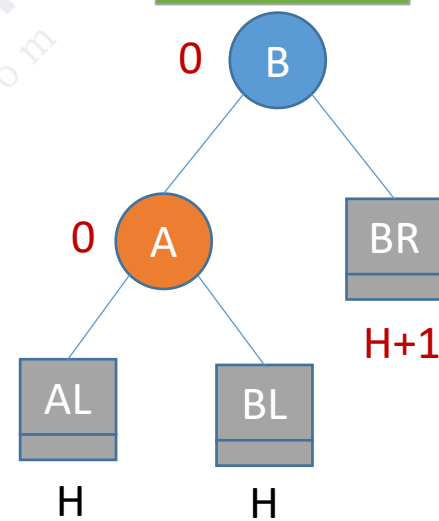
右旋B



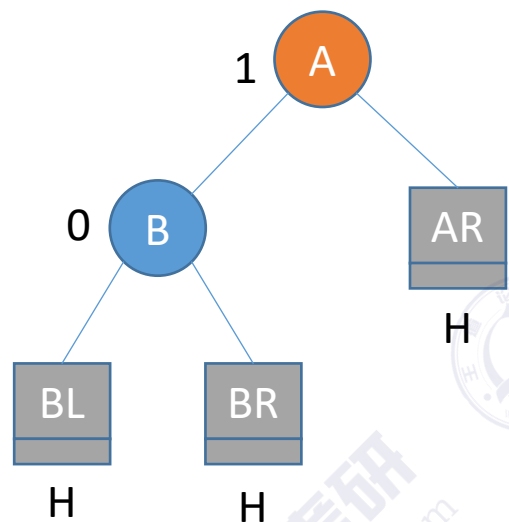
RR型



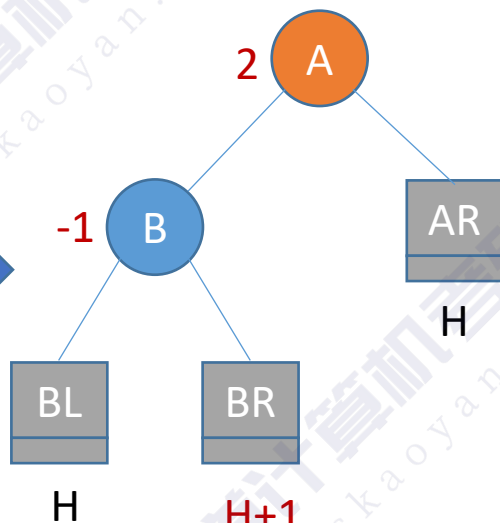
左旋B



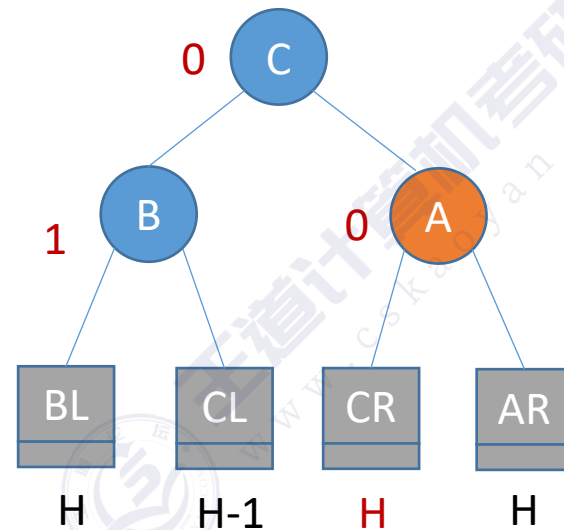
填个坑



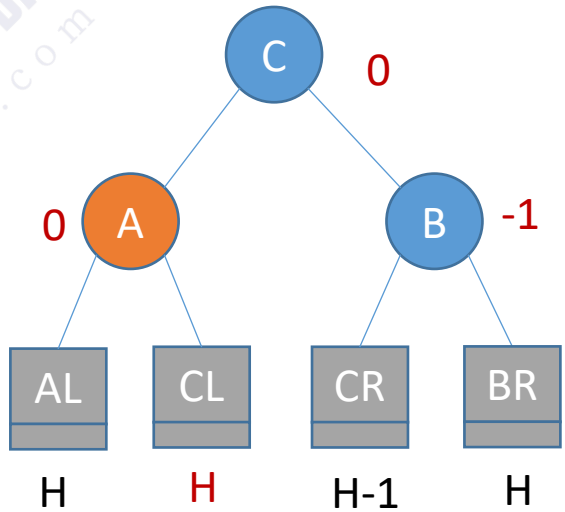
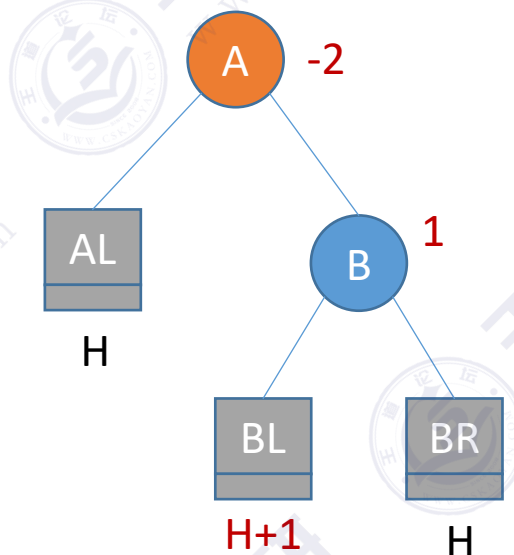
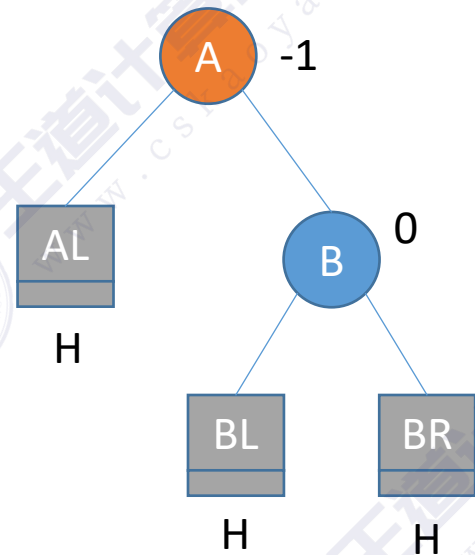
树高H+2



树高H+3



树高H+2



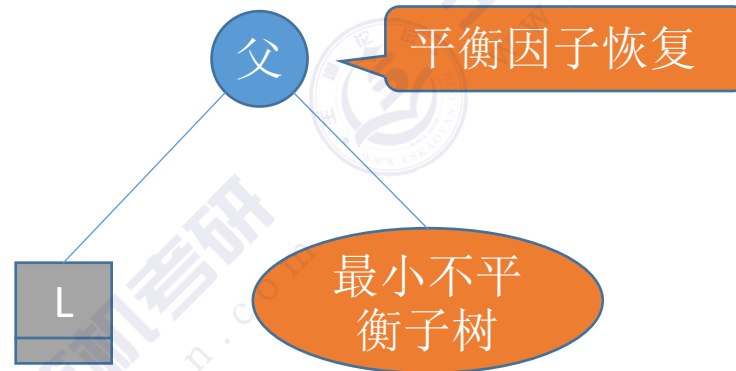
填个坑

插入操作导致“最小不平衡子树”高度+1，经过调整后高度恢复

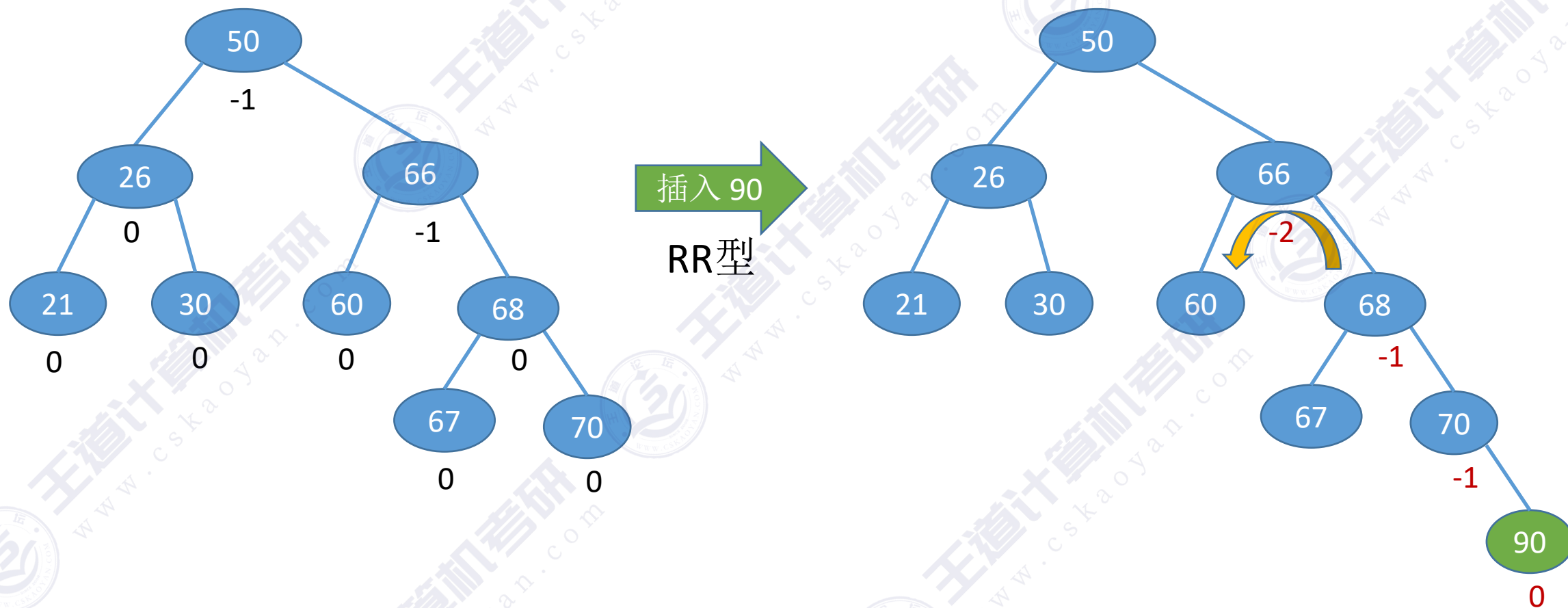


WHY?

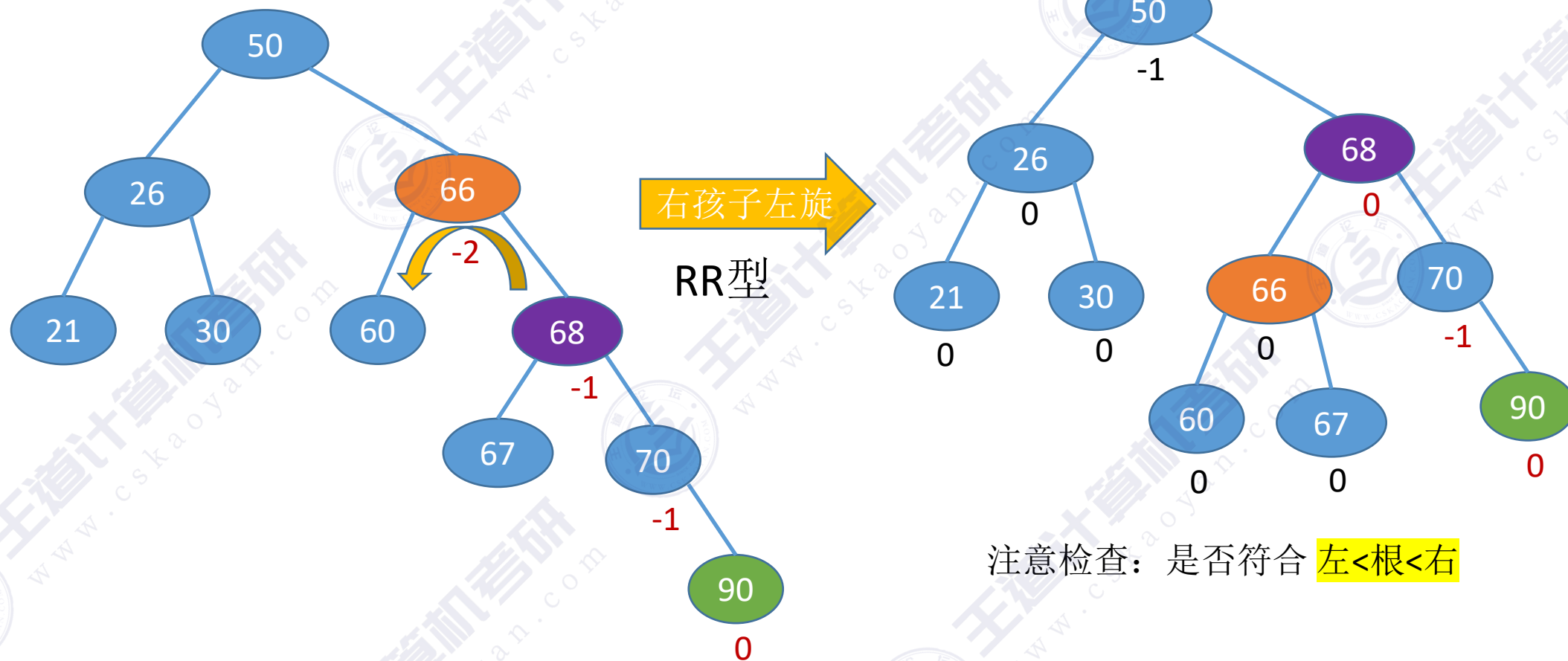
在插入操作中，只要将最小不平衡子树调整平衡，则其他祖先结点都会恢复平衡



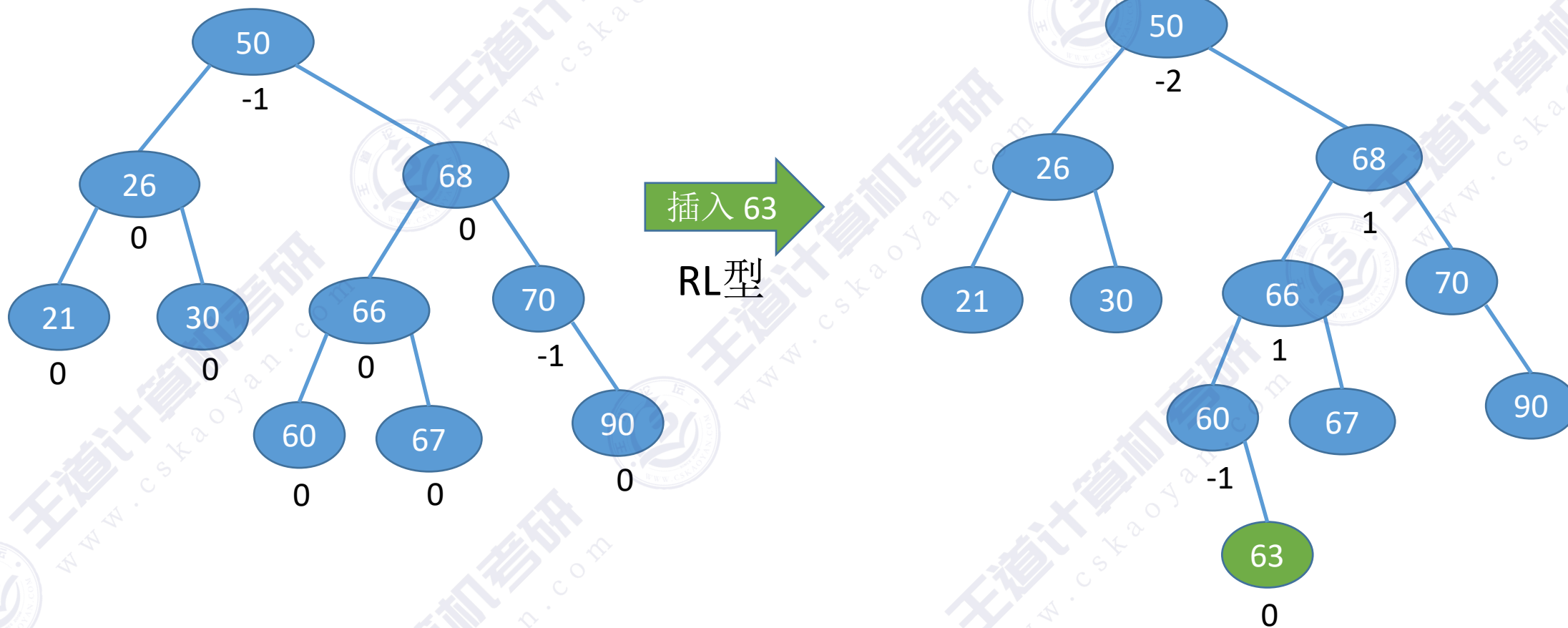
练习



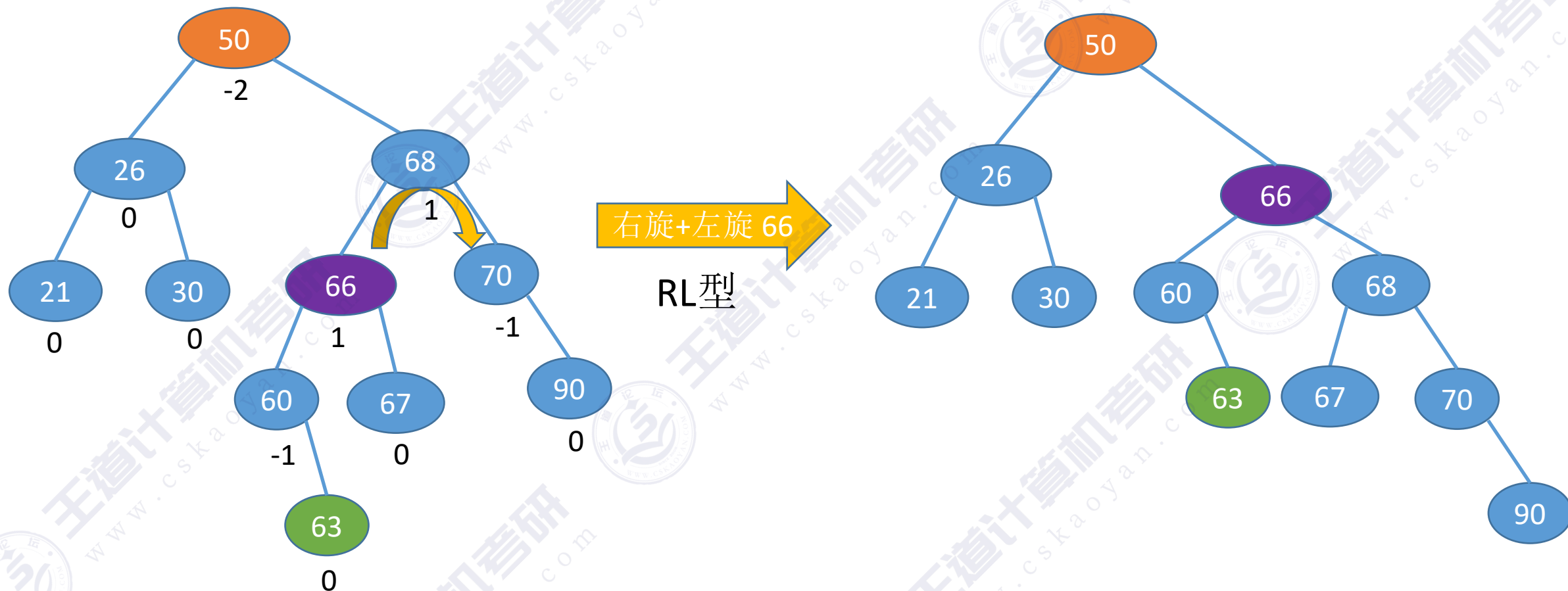
练习



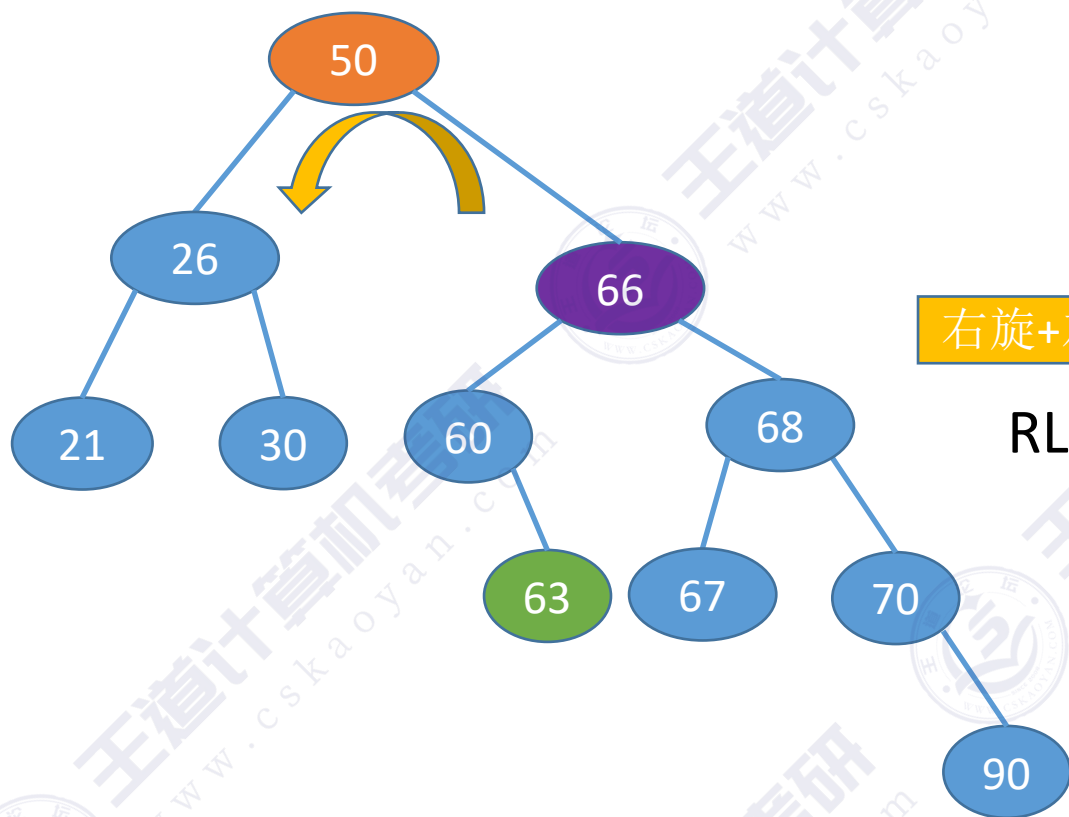
练习



练习

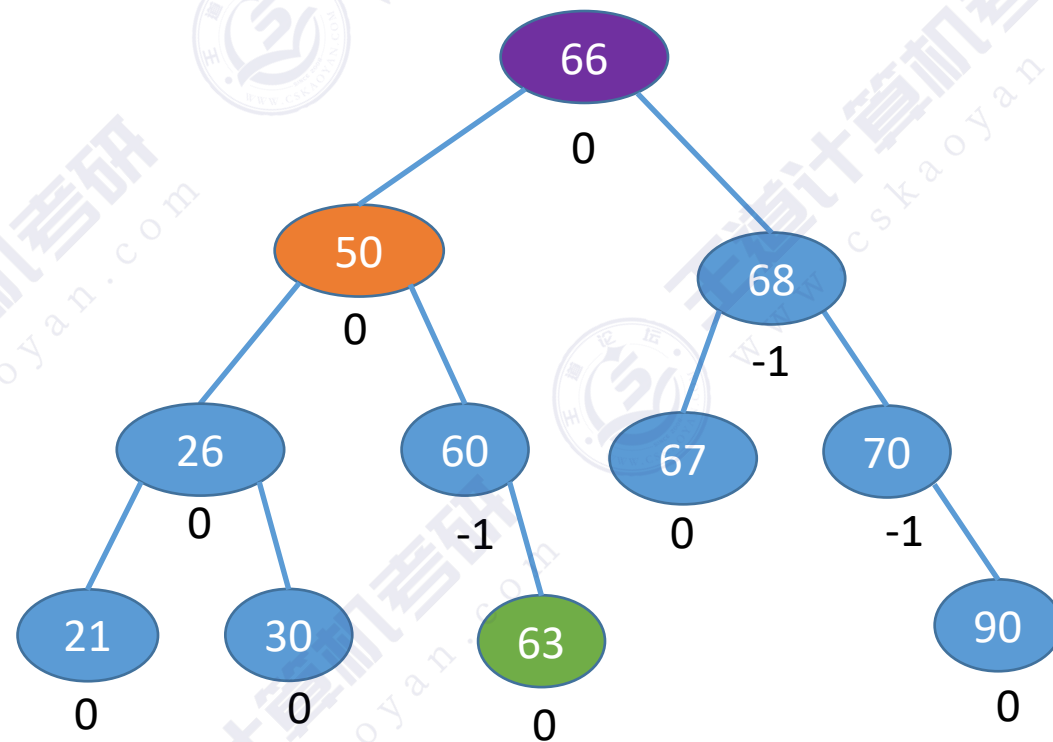


练习



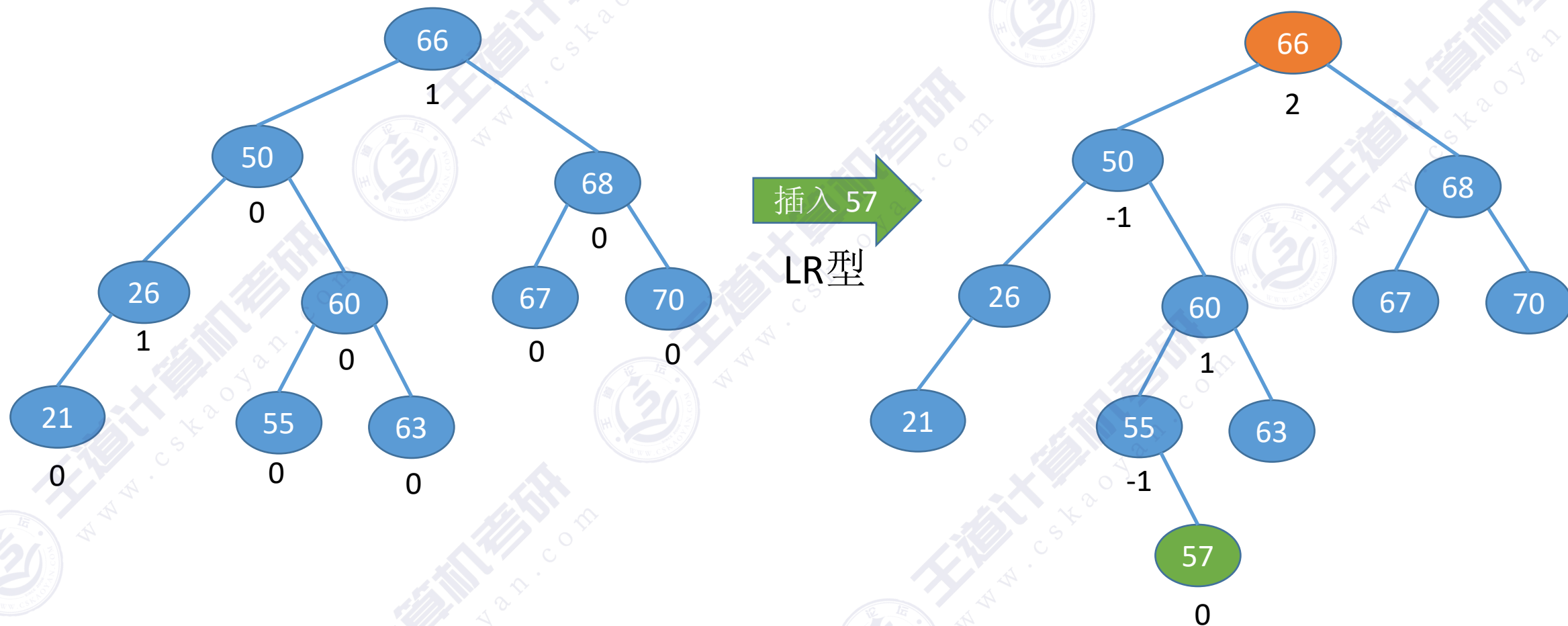
右旋+左旋 66

RL型

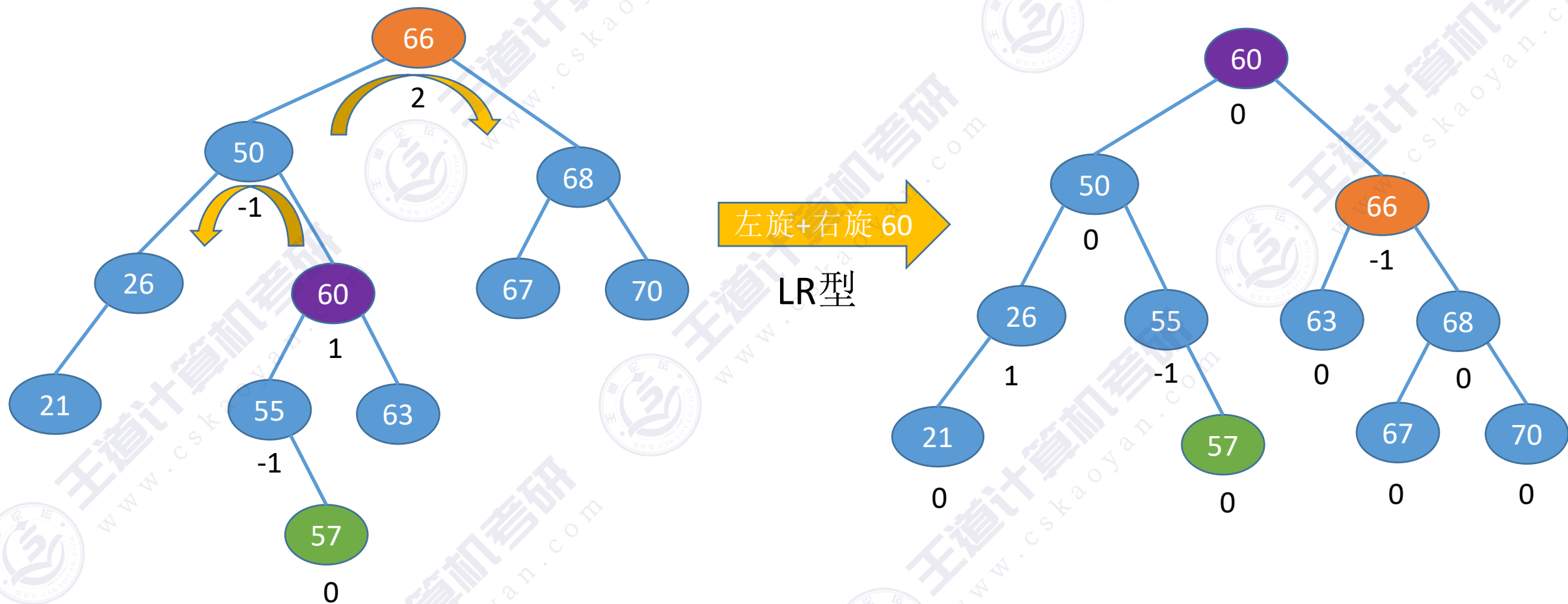


注意检查：是否符合 左<根<右

练习



练习



查找效率分析

若树高为 h ，则最坏情况下，查找一个关键字最多需要对比 h 次，即查找操作的时间复杂度不可能超过 $O(h)$

平衡二叉树——树上任一结点的左子树和右子树的高度之差不超过1。

假设以 n_h 表示深度为 h 的平衡树中含有的最少结点数。

则有 $n_0 = 0, n_1 = 1, n_2 = 2$ ，并且有 $n_h = n_{h-1} + n_{h-2} + 1$

可以证明含有 n 个结点的平衡二叉树的最大深度为 $O(\log_2 n)$ ，平衡二叉树的平均查找长度为 $O(\log_2 n)$

查找效率分析

《An algorithm for the organization of information》——G.M. Adelson-Velsky 和 E.M. Landis ,1962



Figure 1



Figure 2

The recording algorithm is such that at each moment, the reference board is an admissible tree.

Lemma 1. *Let the number of cells of the admissible tree be equal to N . Then the maximum length of the branch is not greater than $(3/2) \log_2 (N + 1)$.*

Proof. Let us denote by N_n the minimum number of cells in the admissible tree when the given maximum length of the branch is n . Then it can be easily proven (see Figure 2) that $N_n = N_{n-1} + N_{n-2} + 1$.

When we solve this equation in finite remainders, we get

$$N_n = \left(1 + \frac{2}{\sqrt{5}}\right) \left(\frac{1 + \sqrt{5}}{2}\right)^n + \left(1 - \frac{2}{\sqrt{5}}\right) \left(\frac{1 - \sqrt{5}}{2}\right)^n - 1.$$

Whence

$$n < \log_{\frac{1+\sqrt{5}}{2}} (N + 1) < \frac{3}{2} \log_2 (N + 1),$$



知识回顾与重要考点



实现 f 向右下旋转, p 向右上旋转:
其中 f 是爹, p 为左孩子, gf 为 f 他爹

- ① $f \rightarrow lchild = p \rightarrow rchild;$
- ② $p \rightarrow rchild = f;$
- ③ $gf \rightarrow lchild/rchild = p;$



实现 f 向左下旋转, p 向左上旋转:

- ① $f \rightarrow rchild = p \rightarrow lchild;$
- ② $p \rightarrow lchild = f;$
- ③ $gf \rightarrow lchild/rchild = p;$

平衡二叉树

定义

树上任一结点的左子树和右子树的高度之差不超过1

结点的平衡因子=左子树高-右子树高

插入操作

和二叉排序树一样, 找合适的位置插入

新插入的结点可能导致其祖先们平衡因子改变, 导致失衡

调整“不平衡”

找到最小不平衡子树进行调整, 记最小不平衡子树的根为A

LL 在A的左孩子的左子树插入导致A不平衡, 将A的左孩子右上旋

RR 在A的右孩子的右子树插入导致A不平衡, 将A的右孩子左上旋

LR 在A的左孩子的右子树插入导致A不平衡, 将A的左孩子的右孩子 先左上旋再右上旋

RL 在A的右孩子的左子树插入导致A不平衡, 将A的右孩子的左孩子 先右上旋再左上旋

查找效率分析

考点: 高为h的平衡二叉树最少有几个结点——递推求解

平衡二叉树最大深度为 $O(\log n)$, 平均查找长度/查找的时间复杂度为 $O(\log n)$

本节内容

平衡二叉树

删除操作

平衡二叉树的插入&删除

平衡二叉树的**插入**操作：

- **插入**新结点后，要**保持二叉排序树的特性**不变（左<中<右）
- 若插入新结点导致**不平衡**，则需要**调整平衡**

平衡二叉树的**删除**操作：

- **删除**结点后，要**保持二叉排序树的特性**不变（左<中<右）
- 若删除结点导致**不平衡**，则需要**调整平衡**



平衡二叉树的删除

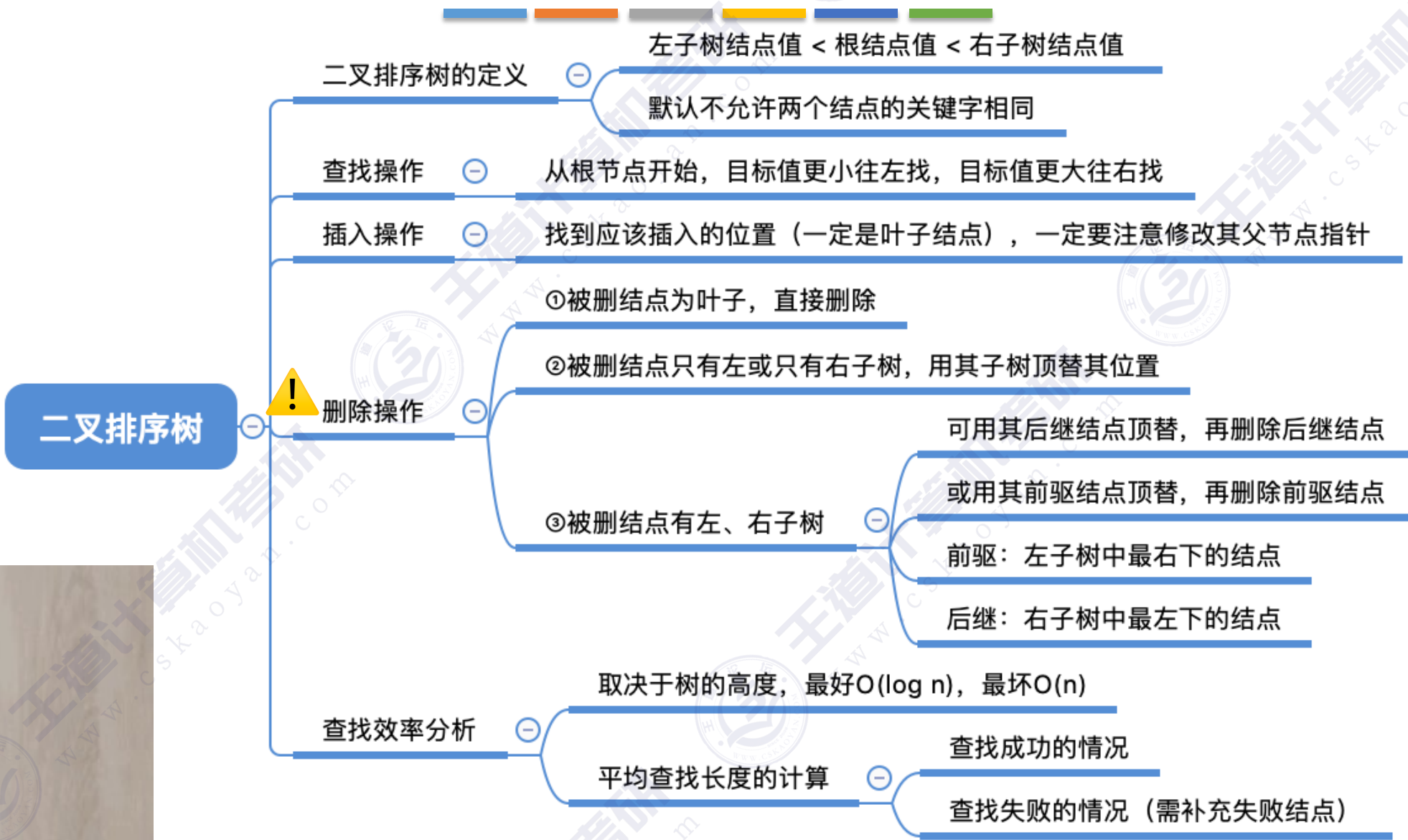
平衡二叉树的删除操作：

- 删除结点后，要保持二叉排序树的特性不变（左<中<右）
- 若删除结点导致不平衡，则需要调整平衡

平衡二叉树的删除操作具体步骤：

- ①删除结点（方法同“二叉排序树”）
- ②一路向北找到最小不平衡子树，找不到就完结撒花
- ③找最小不平衡子树下，“个头”最高的儿子、孙子
- ④根据孙子的位置，调整平衡（LL/RR/LR/RL）
- ⑤如果不平衡向上传导，继续②

暂停偷看：二叉排序树的删除操作



AVL树删除操作——例1

平衡二叉树的删除操作具体步骤：

①删除结点（方法同“二叉排序树”）

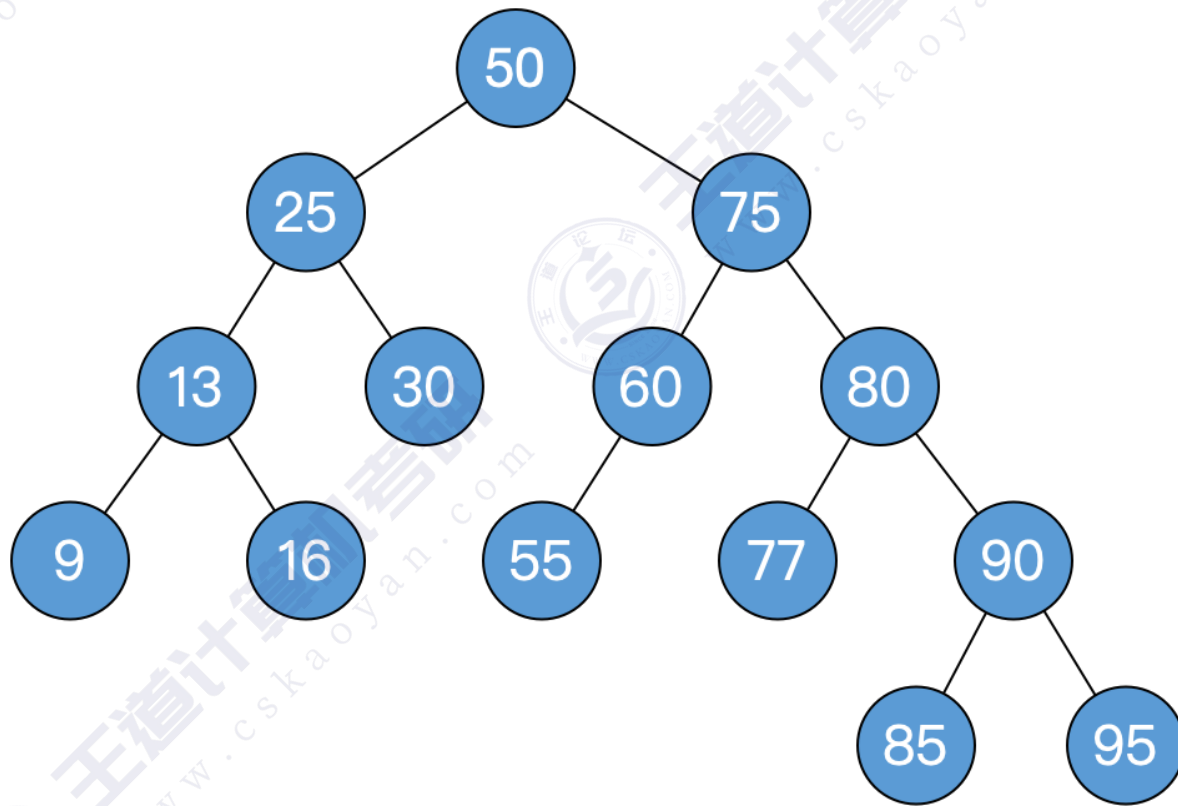
- 若删除的结点是叶子，直接删。
- 若删除的结点只有一个子树，用子树顶替删除位置
- 若删除的结点有两棵子树，用前驱（或后继）结点顶替，并转换为对前驱（或后继）结点的删除。

②一路向北找到最小不平衡子树，找不到就完结撒花

③找最小不平衡子树下，“个头”最高的儿子、孙子

④根据孙子的位置，调整平衡（LL/RR/LR/RL）

⑤如果不平衡向上传导，继续②



AVL树删除操作——例1

平衡二叉树的**删除**操作具体步骤：

①删除结点（方法同“二叉排序树”）

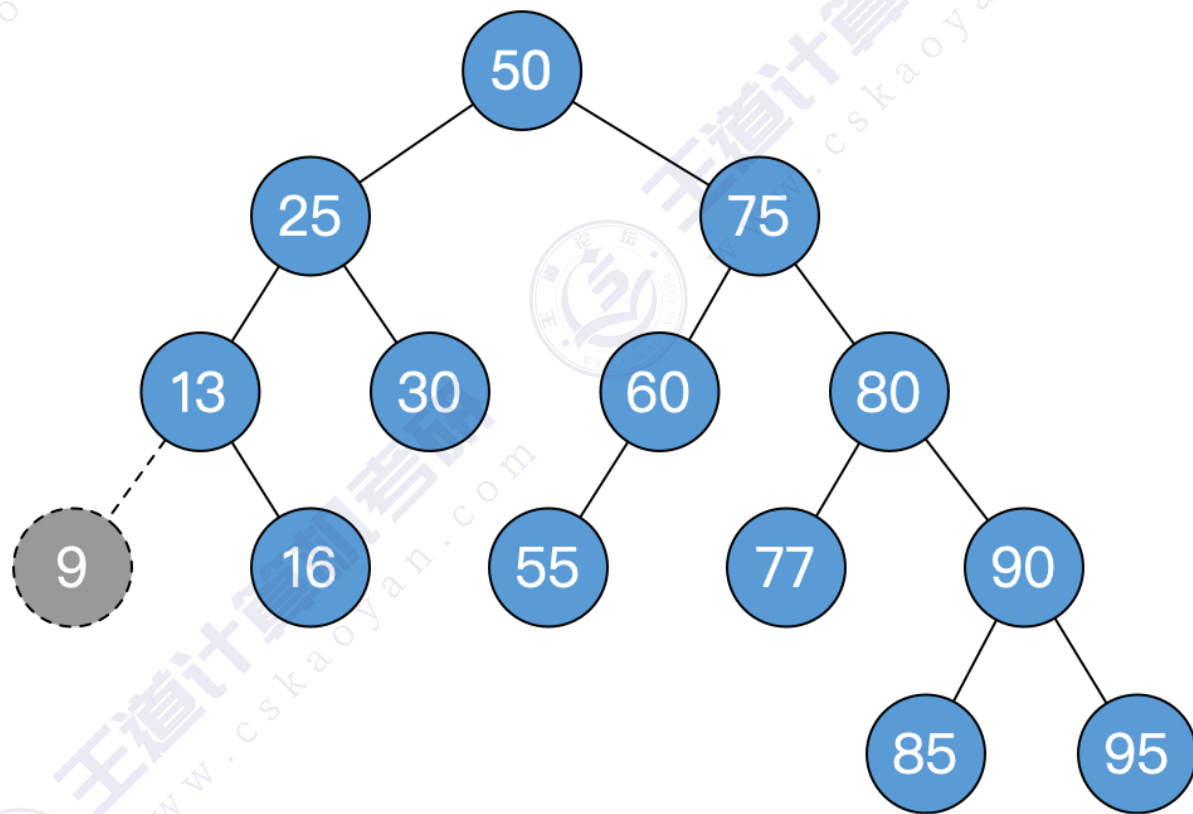
- 若删除的结点是叶子，直接删。
- 若删除的结点只有一个子树，用子树顶替删除位置
- 若删除的结点有两棵子树，用前驱（或后继）结点顶替，并转换为对前驱（或后继）结点的删除。

②一路向北找到最小不平衡子树，找不到就完结撒花

③找最小不平衡子树下，“个头”最高的儿子、孙子

④根据孙子的位置，调整平衡（LL/RR/LR/RL）

⑤如果不平衡向上传导，继续②



AVL树删除操作——例1

平衡二叉树的**删除**操作具体步骤：

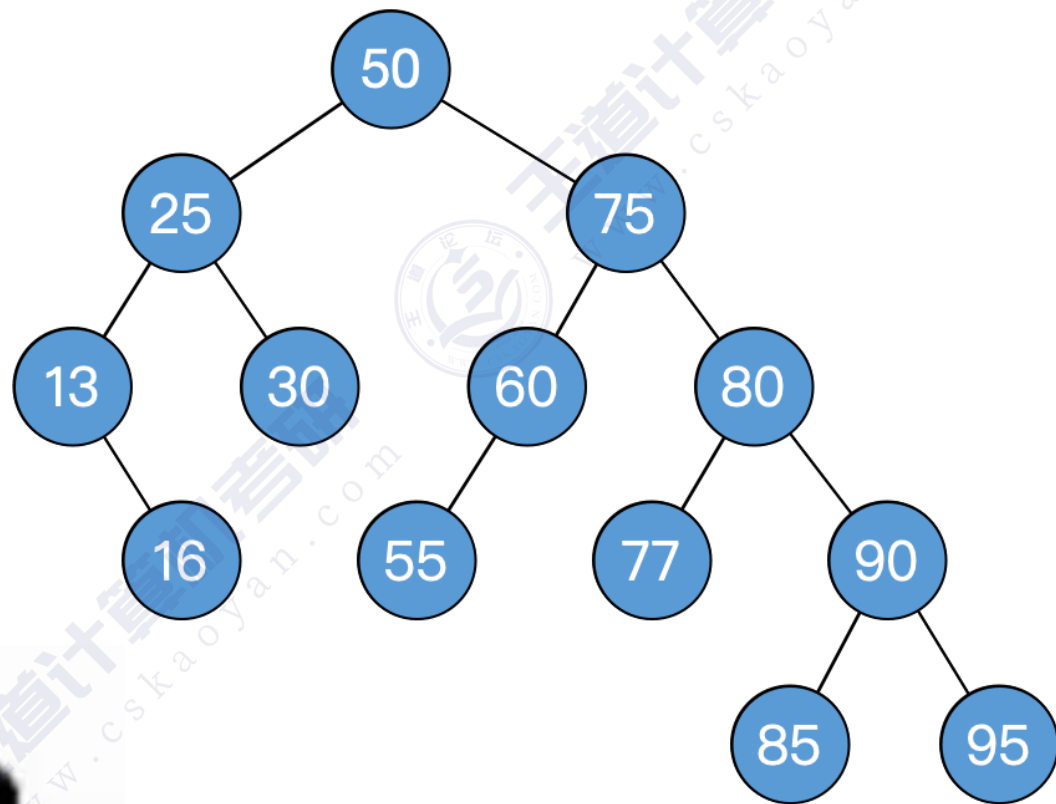
①删除结点（方法同“二叉排序树”）

②一路向北找到最小不平衡子树，找不到就完结撒花

③找最小不平衡子树下，“个头”最高的儿子、孙子

④根据孙子的位置，调整平衡（LL/RR/LR/RL）

⑤如果不平衡向上传导，继续②



Over !



AVL树删除操作——例2

平衡二叉树的删除操作具体步骤：

①删除结点（方法同“二叉排序树”）

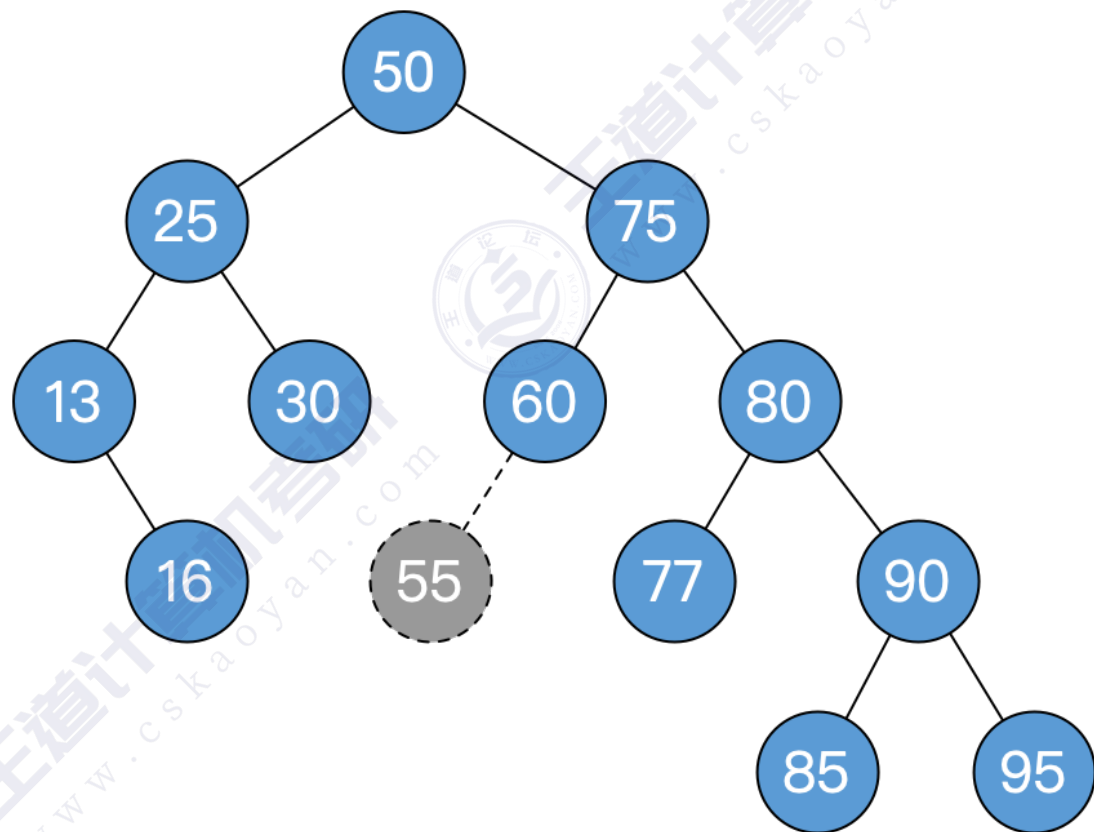
- 若删除的结点是叶子，直接删。
- 若删除的结点只有一个子树，用子树顶替删除位置
- 若删除的结点有两棵子树，用前驱（或后继）结点顶替，并转换为对前驱（或后继）结点的删除。

②一路向北找到最小不平衡子树，找不到就完结撒花

③找最小不平衡子树下，“个头”最高的儿子、孙子

④根据孙子的位置，调整平衡（LL/RR/LR/RL）

⑤如果不平衡向上传导，继续②



AVL树删除操作——例2

平衡二叉树的**删除**操作具体步骤：

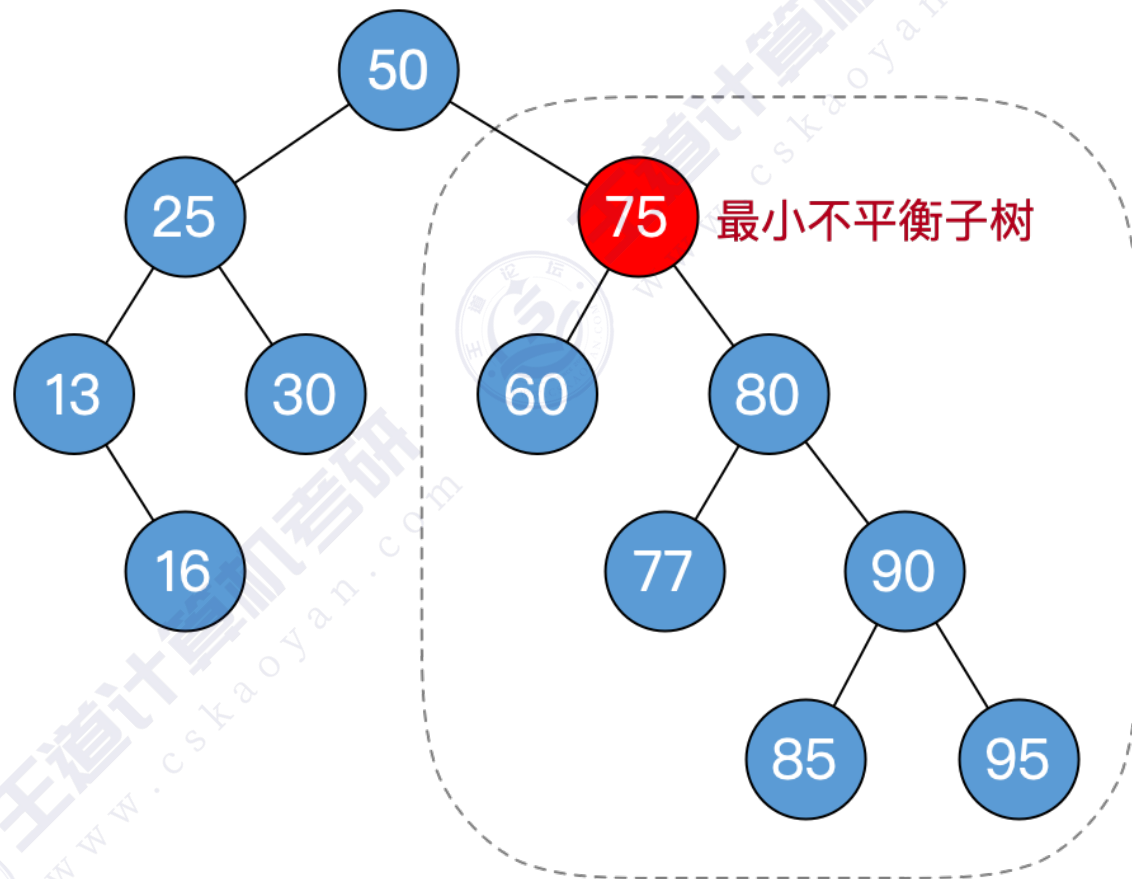
①删除结点（方法同“二叉排序树”）

②一路向北找到最小不平衡子树，找不到就完结撒花

③找最小不平衡子树下，“个头”最高的儿子、孙子

④根据孙子的位置，调整平衡（LL/RR/LR/RL）

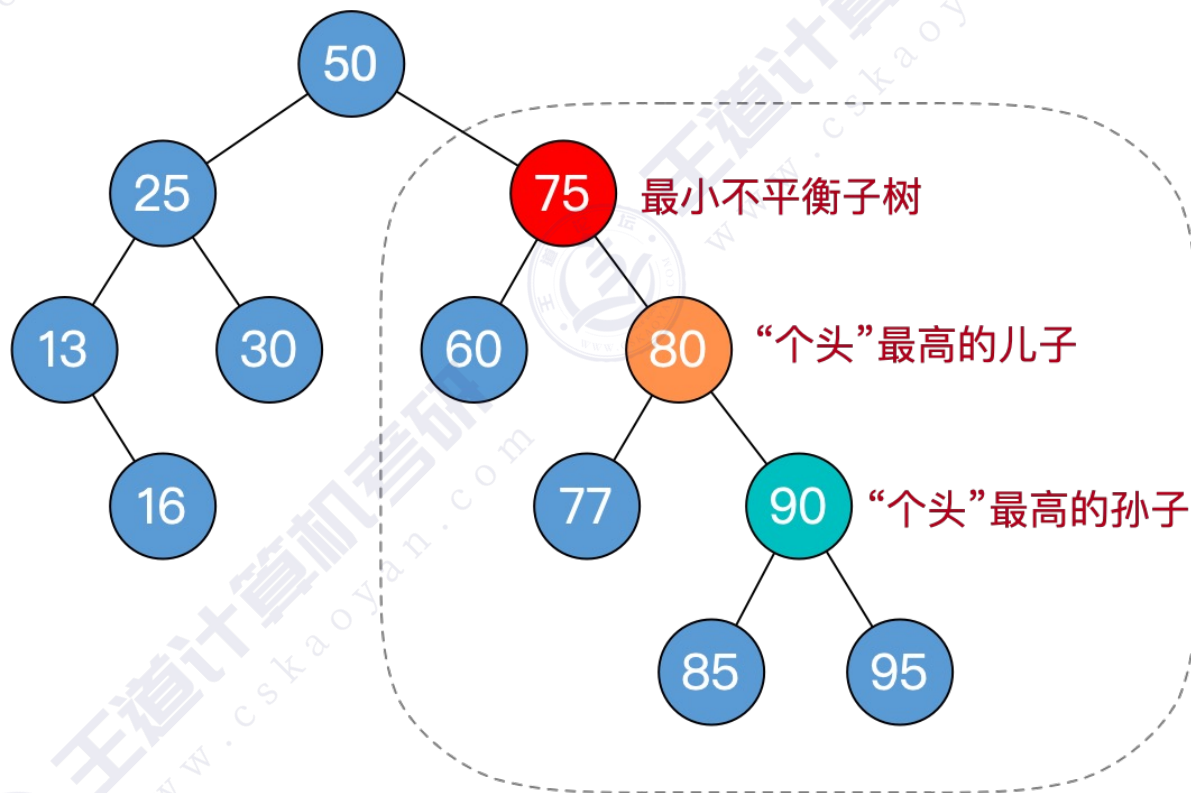
⑤如果不平衡向上传导，继续②



AVL树删除操作——例2

平衡二叉树的**删除**操作具体步骤：

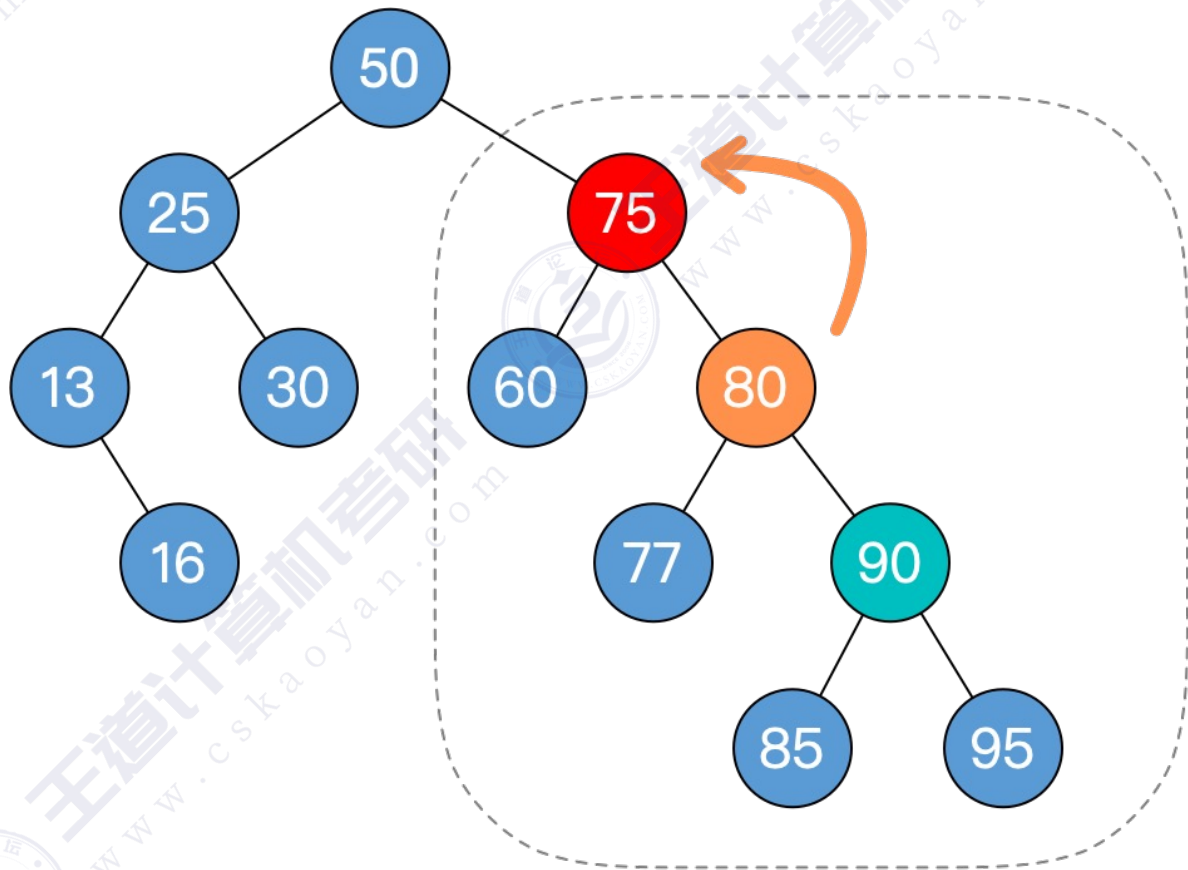
- ①删除结点（方法同“二叉排序树”）
- ②一路向北找到最小不平衡子树，找不到就完结撒花
- ③找最小不平衡子树下，“个头”最高的儿子、孙子
- ④根据孙子的位置，调整平衡（LL/RR/LR/RL）
- ⑤如果不平衡向上传导，继续②



AVL树删除操作——例2

平衡二叉树的删除操作具体步骤：

- ①删除结点（方法同“二叉排序树”）
- ②一路向北找到最小不平衡子树，找不到就完结撒花
- ③找最小不平衡子树下，“个头”最高的儿子、孙子
- ④根据孙子的位置，调整平衡（LL/RR/LR/RL）
 - 孙子在LL：儿子右单旋
 - 孙子在RR：儿子左单旋
 - 孙子在LR：孙子先左旋，再右旋
 - 孙子在RL：孙子先右旋，再左旋
- ⑤如果不平衡向上传导，继续②

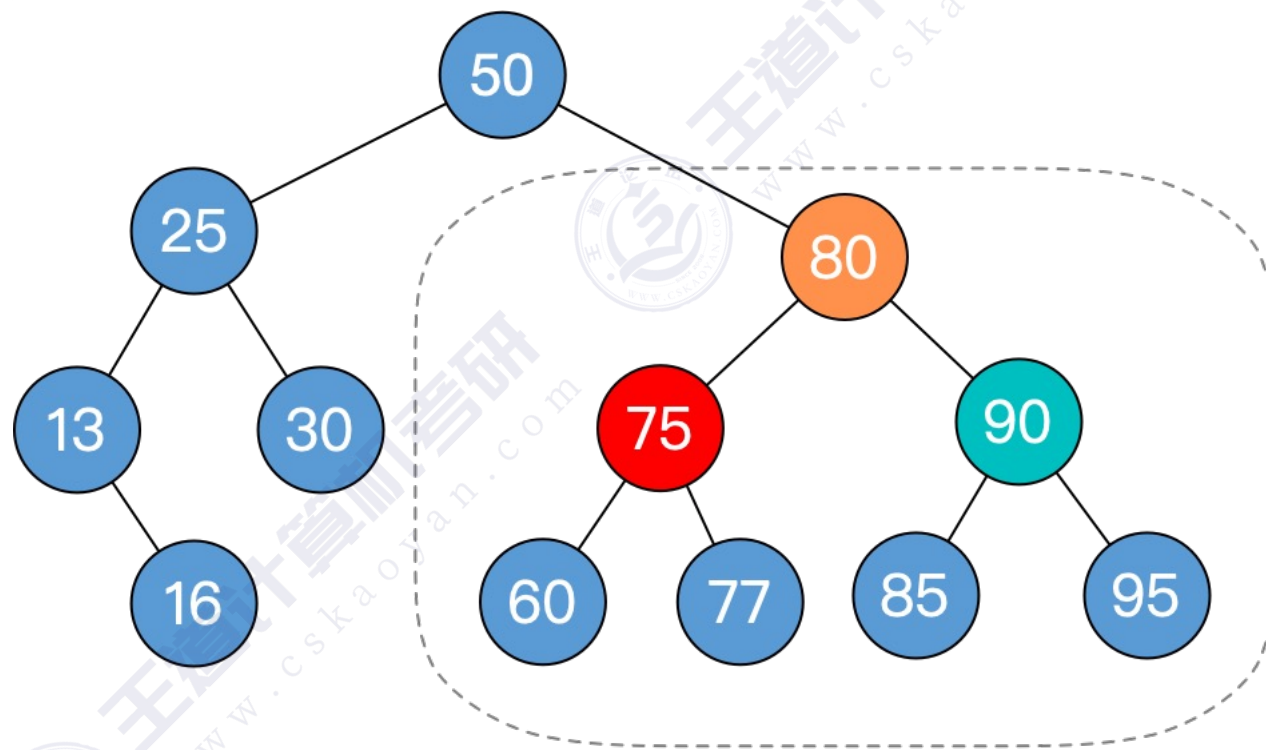


孙子在RR，儿子左单旋

AVL树删除操作——例2

平衡二叉树的删除操作具体步骤：

- ①删除结点（方法同“二叉排序树”）
- ②一路向北找到最小不平衡子树，找不到就完结撒花
- ③找最小不平衡子树下，“个头”最高的儿子、孙子
- ④根据孙子的位置，调整平衡（LL/RR/LR/RL）
 - 孙子在LL：儿子右单旋
 - 孙子在RR：儿子左单旋
 - 孙子在LR：孙子先左旋，再右旋
 - 孙子在RL：孙子先右旋，再左旋
- ⑤如果不平衡向上传导，继续②

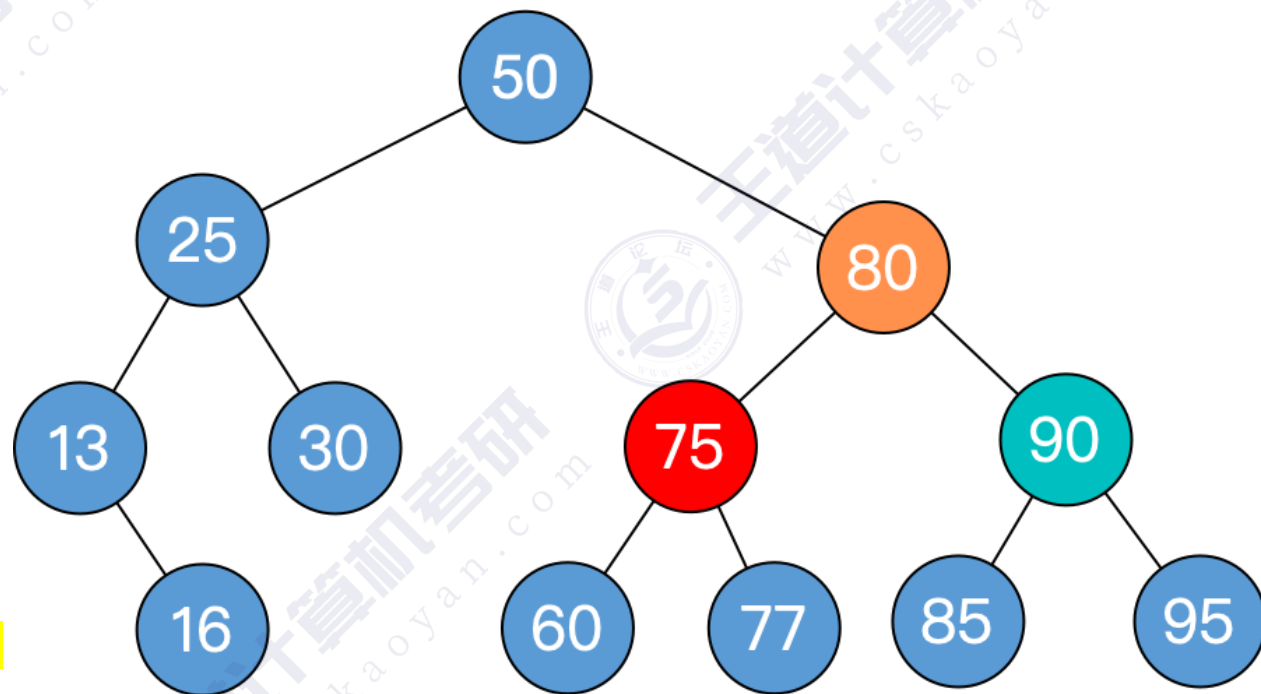


最小不平衡子树恢复平衡

AVL树删除操作——例2

平衡二叉树的**删除**操作具体步骤：

- ①删除结点（方法同“二叉排序树”）
- ②一路向北找到最小不平衡子树，找不到就完结撒花
- ③找最小不平衡子树下，“个头”最高的儿子、孙子
- ④根据孙子的位置，调整平衡（LL/RR/LR/RL）
- ⑤如果不平衡向上传导，继续②
 - 对最小不平衡子树的旋转可能导致树变矮，从而导致上层祖先不平衡（不平衡向上传递）



Over !



AVL树删除操作——例3

平衡二叉树的删除操作具体步骤：

①删除结点（方法同“二叉排序树”）

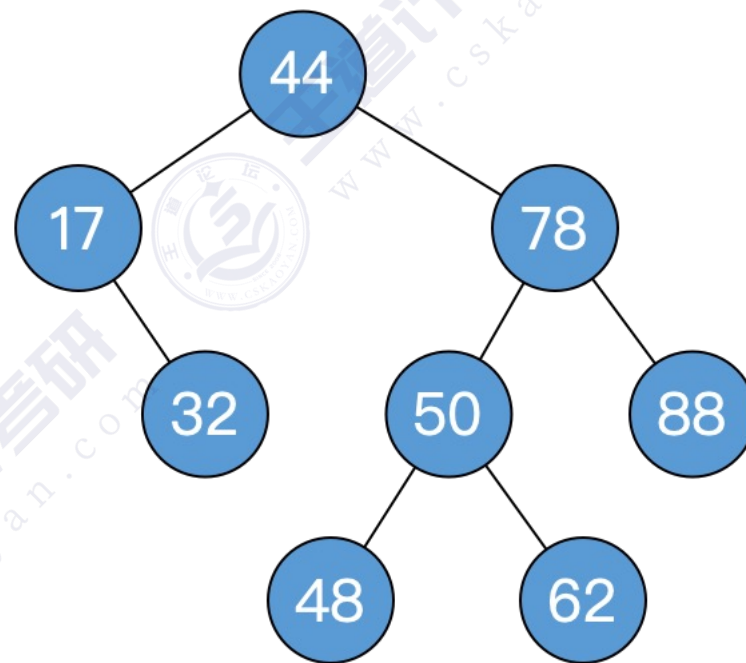
- 若删除的结点是叶子，直接删。
- 若删除的结点只有一个子树，用子树顶替删除位置
- 若删除的结点有两棵子树，用前驱（或后继）结点顶替，并转换为对前驱（或后继）结点的删除。

②一路向北找到最小不平衡子树，找不到就完结撒花

③找最小不平衡子树下，“个头”最高的儿子、孙子

④根据孙子的位置，调整平衡（LL/RR/LR/RL）

⑤如果不平衡向上传导，继续②



AVL树删除操作——例3

平衡二叉树的**删除**操作具体步骤：

①删除结点（方法同“二叉排序树”）

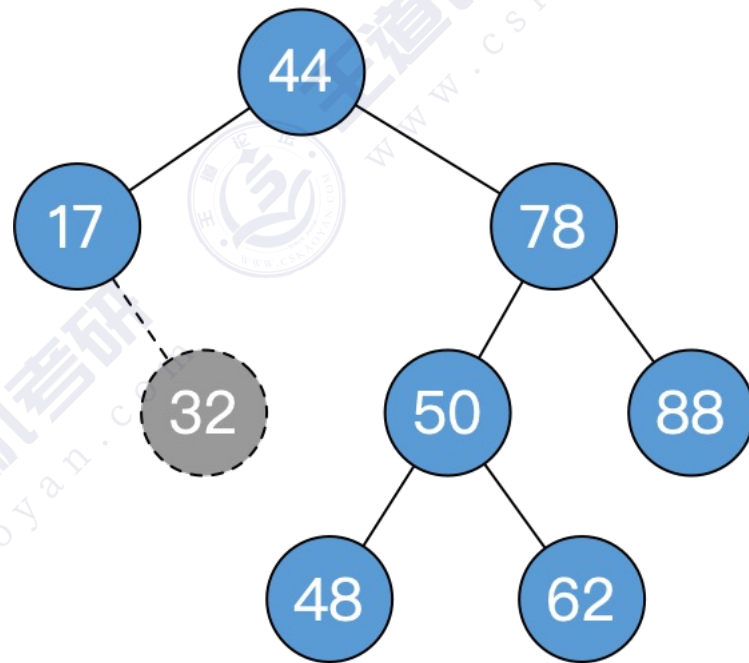
- 若删除的结点是叶子，直接删。
- 若删除的结点只有一个子树，用子树顶替删除位置
- 若删除的结点有两棵子树，用前驱（或后继）结点顶替，并转换为对前驱（或后继）结点的删除。

②一路向北找到最小不平衡子树，找不到就完结撒花

③找最小不平衡子树下，“个头”最高的儿子、孙子

④根据孙子的位置，调整平衡（LL/RR/LR/RL）

⑤如果不平衡向上传导，继续②



AVL树删除操作——例3

平衡二叉树的删除操作具体步骤：

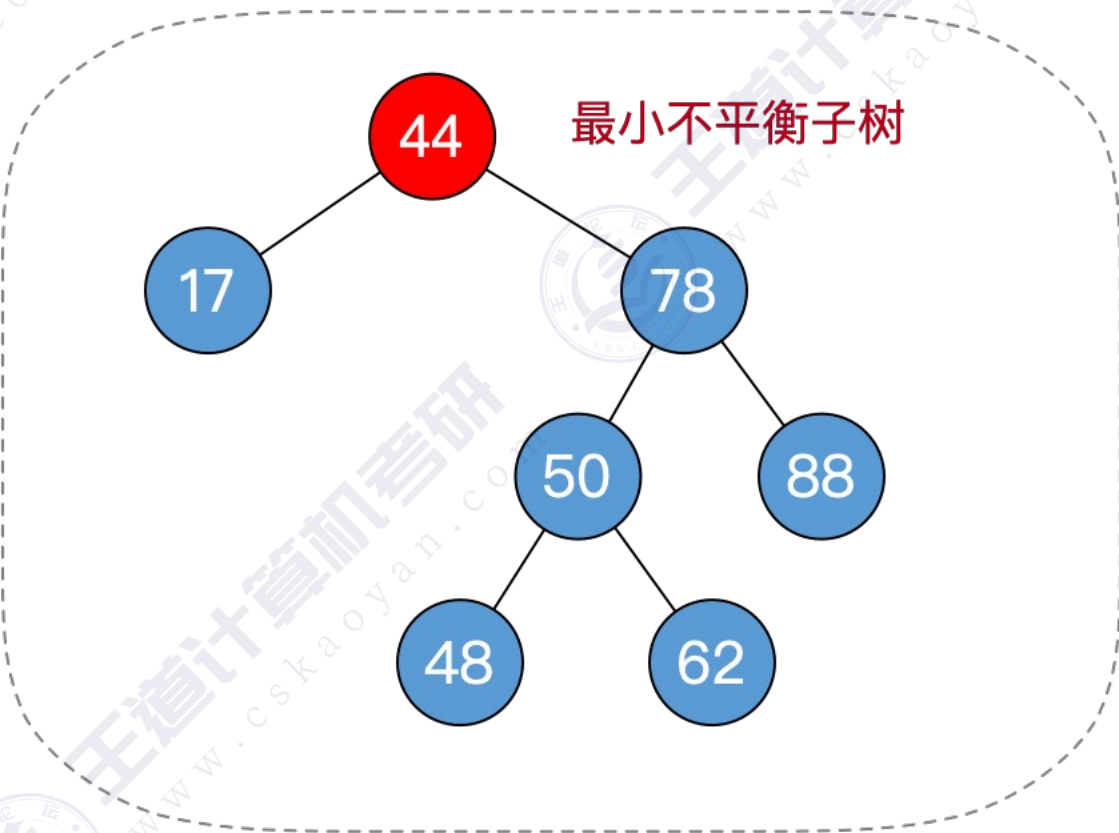
①删除结点（方法同“二叉排序树”）

②一路向北找到最小不平衡子树，找不到就完结撒花

③找最小不平衡子树下，“个头”最高的儿子、孙子

④根据孙子的位置，调整平衡（LL/RR/LR/RL）

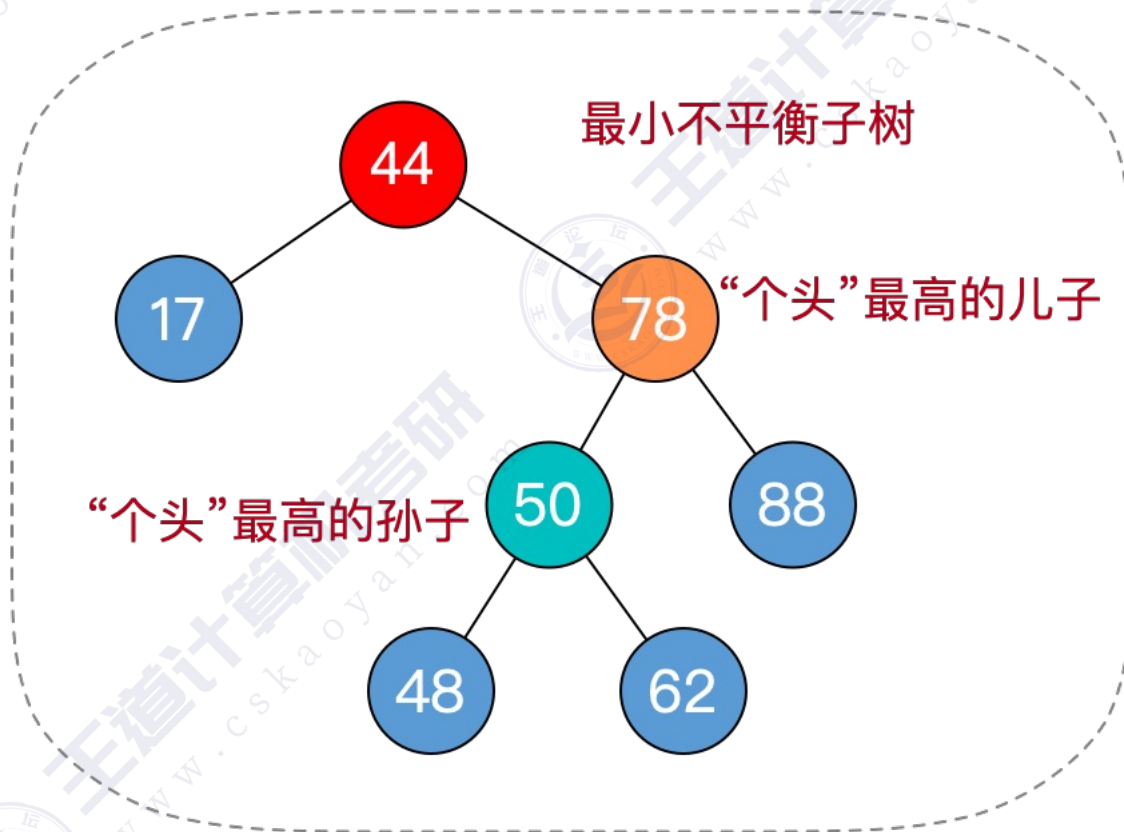
⑤如果不平衡向上传导，继续②



AVL树删除操作——例3

平衡二叉树的**删除**操作具体步骤：

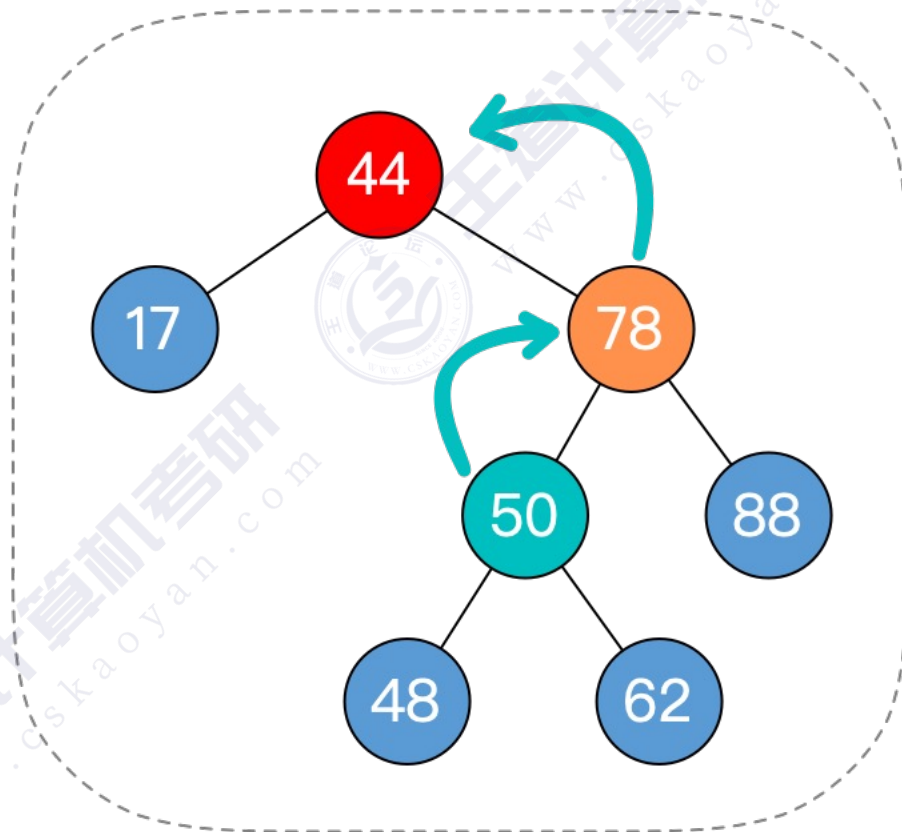
- ①删除结点（方法同“二叉排序树”）
- ②一路向北找到最小不平衡子树，找不到就完结撒花
- ③找最小不平衡子树下，“个头”最高的儿子、孙子
- ④根据孙子的位置，调整平衡（LL/RR/LR/RL）
- ⑤如果不平衡向上传导，继续②



AVL树删除操作——例3

平衡二叉树的删除操作具体步骤：

- ①删除结点（方法同“二叉排序树”）
- ②一路向北找到最小不平衡子树，找不到就完结撒花
- ③找最小不平衡子树下，“个头”最高的儿子、孙子
- ④根据孙子的位置，调整平衡（LL/RR/LR/RL）
 - 孙子在LL：儿子右单旋
 - 孙子在RR：儿子左单旋
 - 孙子在LR：孙子先左旋，再右旋
 - 孙子在RL：孙子先右旋，再左旋
- ⑤如果不平衡向上传导，继续②

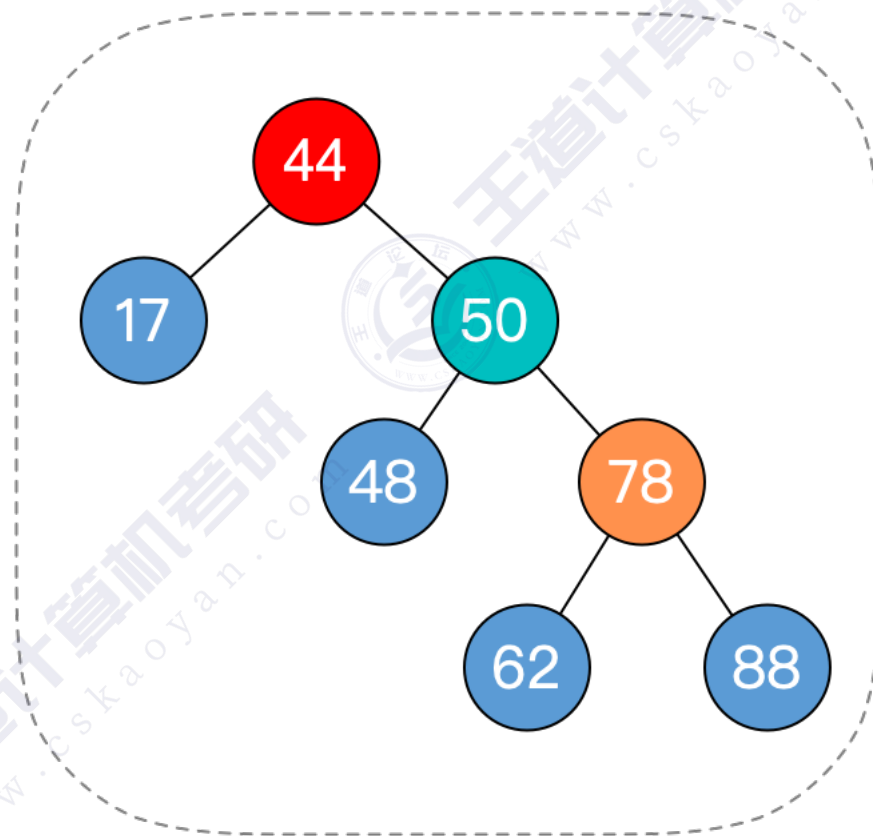


孙子在RL，孙子先右旋，再左旋

AVL树删除操作——例3

平衡二叉树的删除操作具体步骤：

- ①删除结点（方法同“二叉排序树”）
- ②一路向北找到最小不平衡子树，找不到就完结撒花
- ③找最小不平衡子树下，“个头”最高的儿子、孙子
- ④根据孙子的位置，调整平衡（LL/RR/LR/RL）
 - 孙子在LL：儿子右单旋
 - 孙子在RR：儿子左单旋
 - 孙子在LR：孙子先左旋，再右旋
 - 孙子在RL：孙子先右旋，再左旋
- ⑤如果不平衡向上传导，继续②

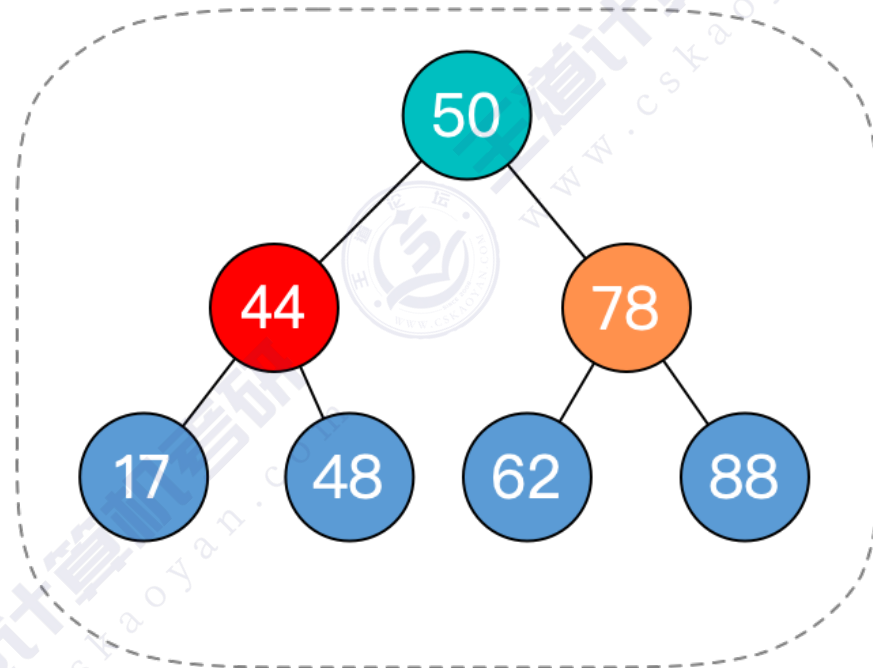


先右旋

AVL树删除操作——例3

平衡二叉树的删除操作具体步骤：

- ①删除结点（方法同“二叉排序树”）
- ②一路向北找到最小不平衡子树，找不到就完结撒花
- ③找最小不平衡子树下，“个头”最高的儿子、孙子
- ④根据孙子的位置，调整平衡（LL/RR/LR/RL）
 - 孙子在LL：儿子右单旋
 - 孙子在RR：儿子左单旋
 - 孙子在LR：孙子先左旋，再右旋
 - 孙子在RL：孙子先右旋，再左旋
- ⑤如果不平衡向上传导，继续②



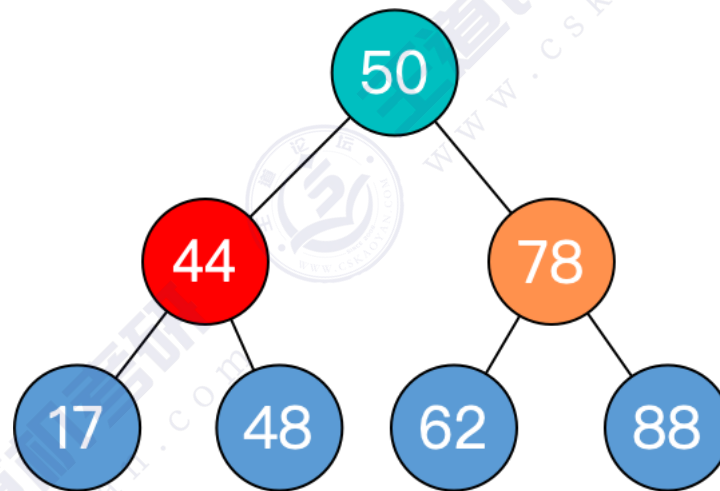
再左旋

最小不平衡子树恢复平衡

AVL树删除操作——例3

平衡二叉树的**删除**操作具体步骤：

- ①删除结点（方法同“二叉排序树”）
- ②一路向北找到最小不平衡子树，找不到就完结撒花
- ③找最小不平衡子树下，“个头”最高的儿子、孙子
- ④根据孙子的位置，调整平衡（LL/RR/LR/RL）
- ⑤如果不平衡向上传导，继续②
 - 对最小不平衡子树的旋转可能导致树变矮，从而导致上层祖先不平衡（不平衡向上传递）



Over !



AVL树删除操作——例4

平衡二叉树的**删除**操作具体步骤:

①删除结点（方法同“二叉排序树”）

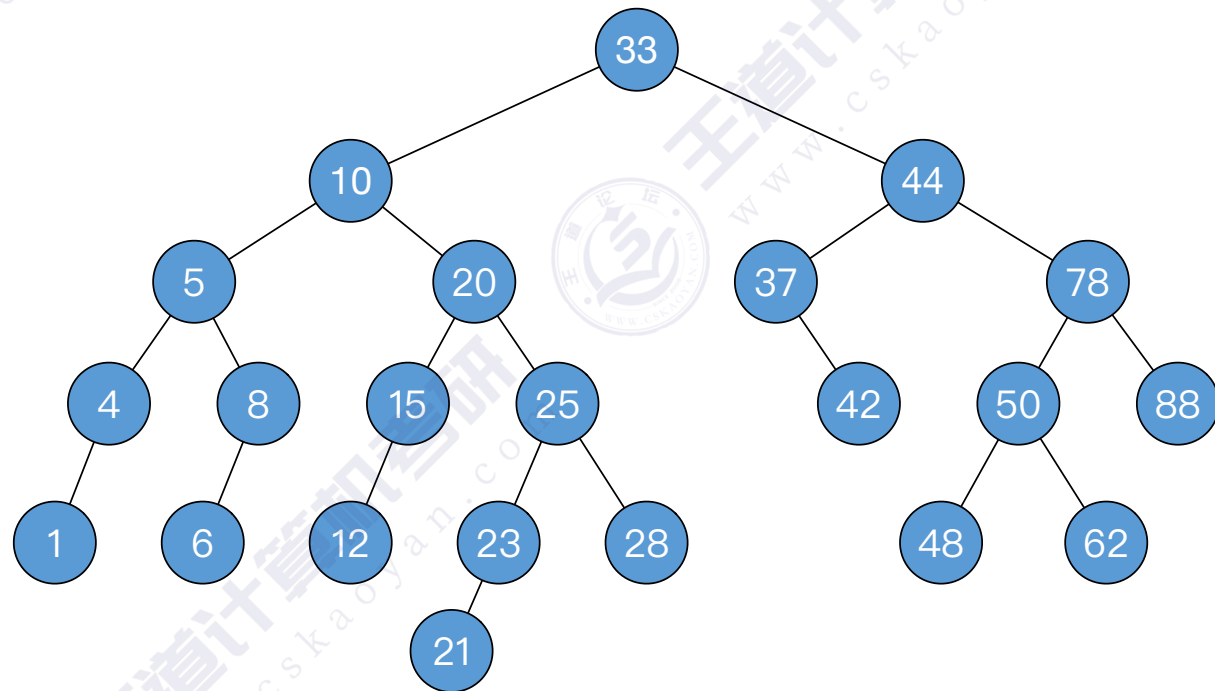
- 若删除的结点是叶子，直接删。
- 若删除的结点只有一个子树，用子树顶替删除位置
- 若删除的结点有两棵子树，用前驱（或后继）结点顶替，并转换为对前驱（或后继）结点的删除。

②一路向北找到最小不平衡子树，找不到就完结撒花

③找最小不平衡子树下，“个头”最高的儿子、孙子

④根据孙子的位置，调整平衡（LL/RR/LR/RL）

⑤如果不平衡向上传导，继续②



AVL树删除操作——例4

平衡二叉树的删除操作具体步骤：

①删除结点（方法同“二叉排序树”）

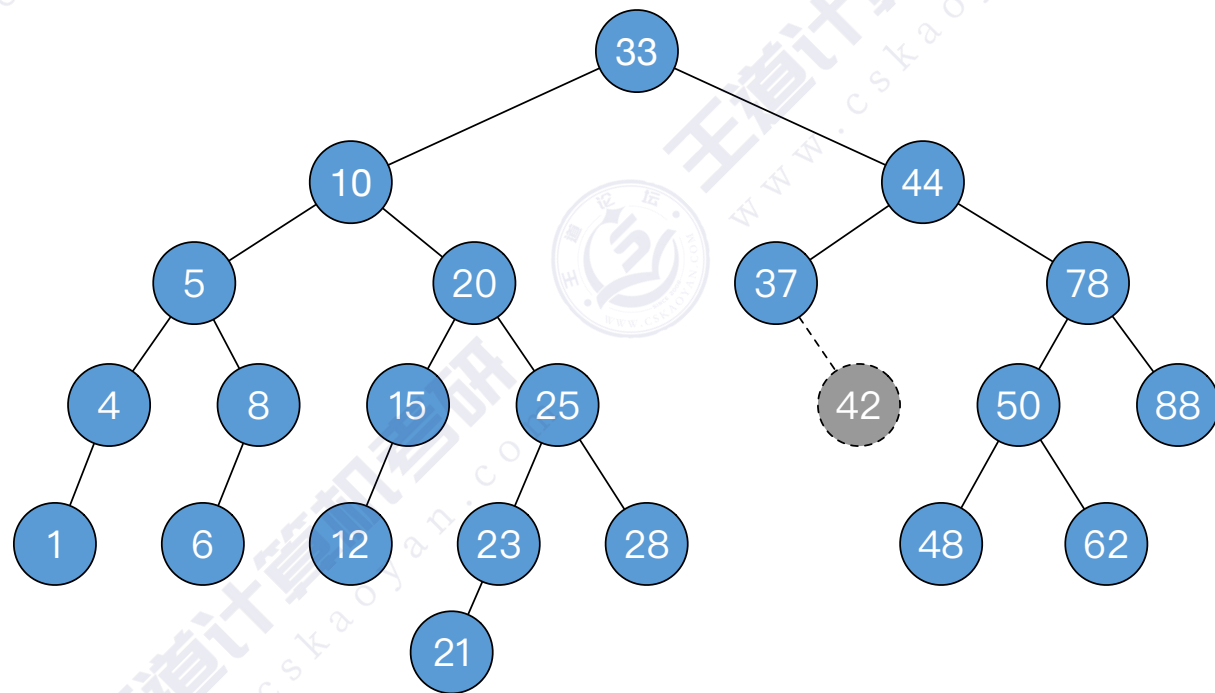
- 若删除的结点是叶子，直接删。
- 若删除的结点只有一个子树，用子树顶替删除位置
- 若删除的结点有两棵子树，用前驱（或后继）结点顶替，并转换为对前驱（或后继）结点的删除。

②一路向北找到最小不平衡子树，找不到就完结撒花

③找最小不平衡子树下，“个头”最高的儿子、孙子

④根据孙子的位置，调整平衡（LL/RR/LR/RL）

⑤如果不平衡向上传导，继续②



AVL树删除操作——例4

平衡二叉树的删除操作具体步骤：

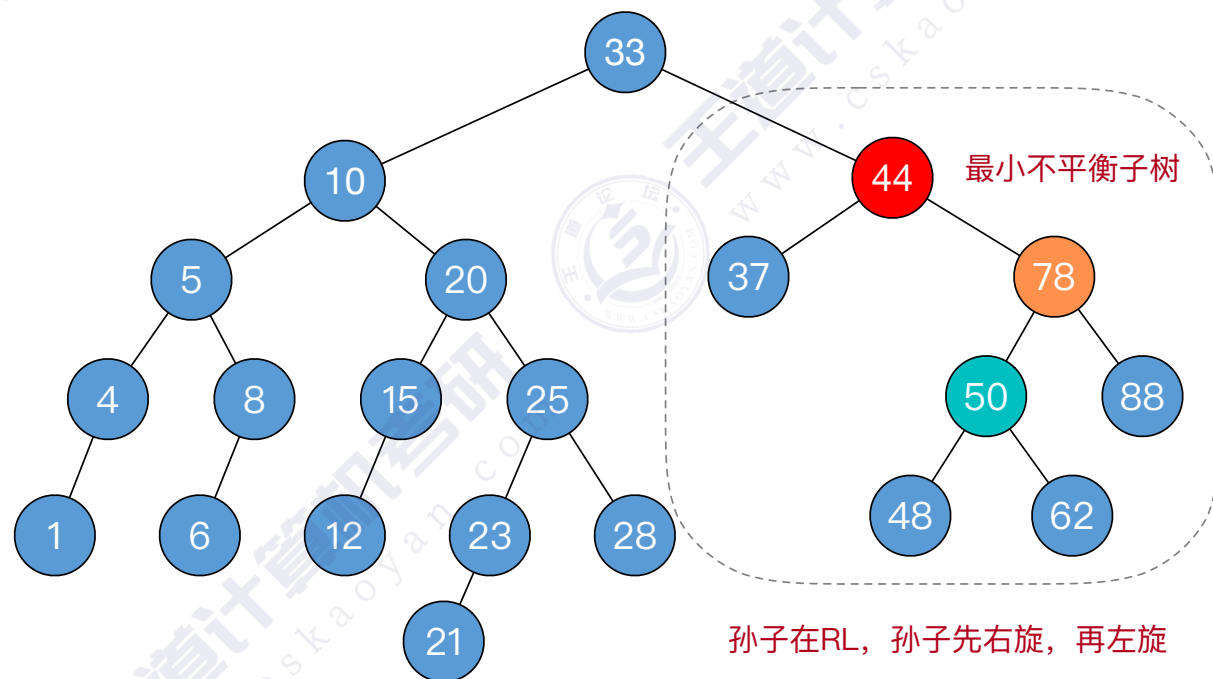
①删除结点（方法同“二叉排序树”）

②一路向北找到最小不平衡子树，找不到就完结撒花

③找最小不平衡子树下，“个头”最高的儿子、孙子

④根据孙子的位置，调整平衡（LL/RR/LR/RL）

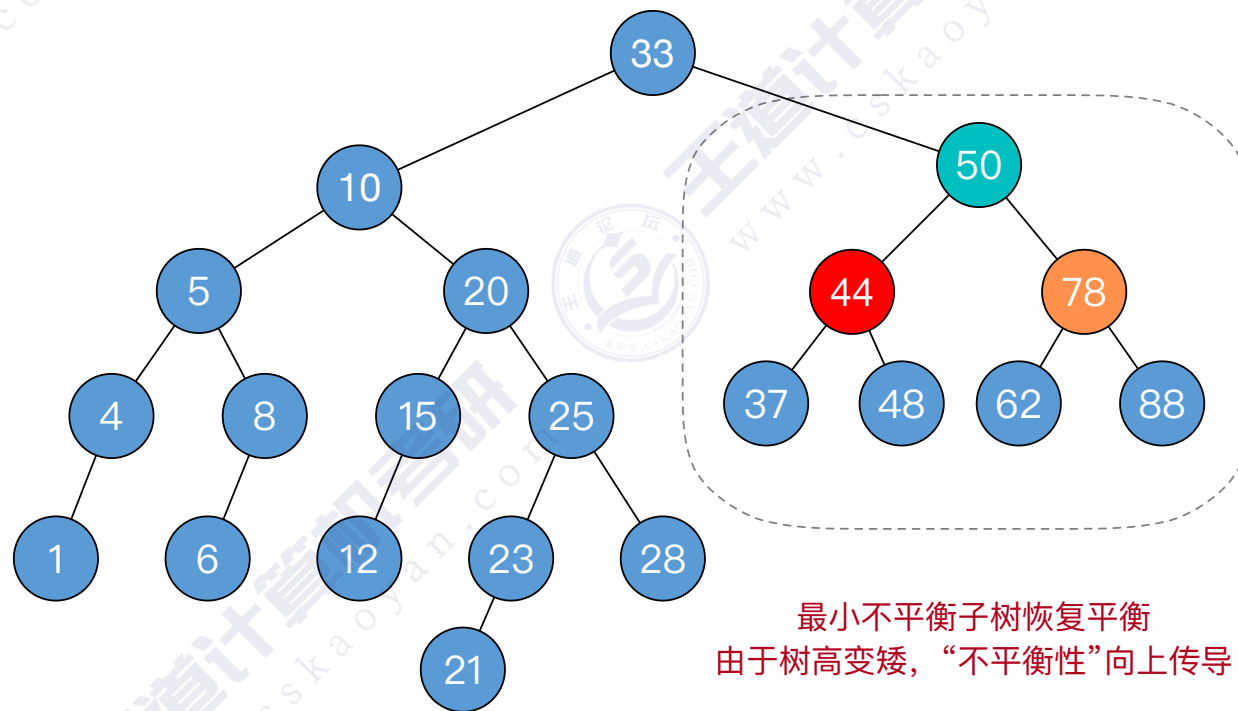
⑤如果不平衡向上传导，继续②



AVL树删除操作——例4

平衡二叉树的**删除**操作具体步骤：

- ①删除结点（方法同“二叉排序树”）
- ②一路向北找到最小不平衡子树，找不到就完结撒花
- ③找最小不平衡子树下，“个头”最高的儿子、孙子
- ④根据孙子的位置，调整平衡（LL/RR/LR/RL）
- ⑤如果不平衡向上传导，继续②
 - 对最小不平衡子树的旋转可能导致树变矮，从而导致上层祖先不平衡（不平衡向上传递）



AVL树删除操作——例4

平衡二叉树的删除操作具体步骤：

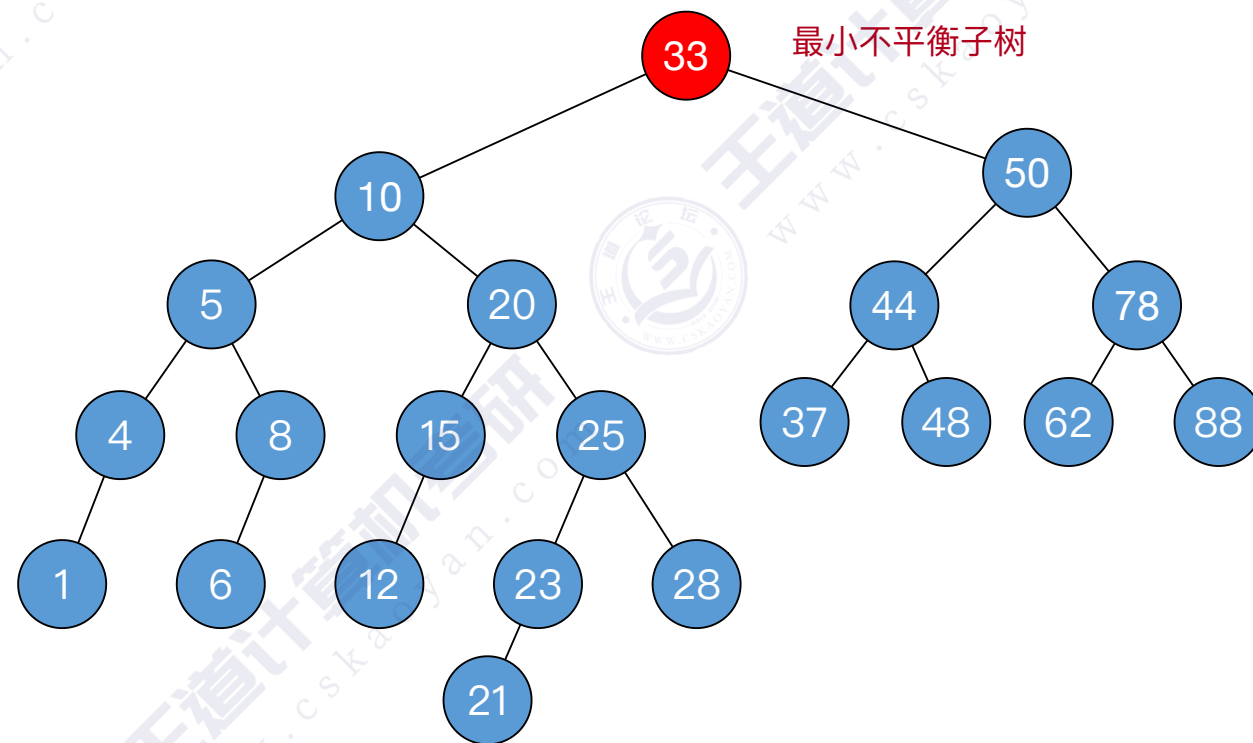
①删除结点（方法同“二叉排序树”）

②一路向北找到最小不平衡子树，找不到就完结撒花

③找最小不平衡子树下，“个头”最高的儿子、孙子

④根据孙子的位置，调整平衡（LL/RR/LR/RL）

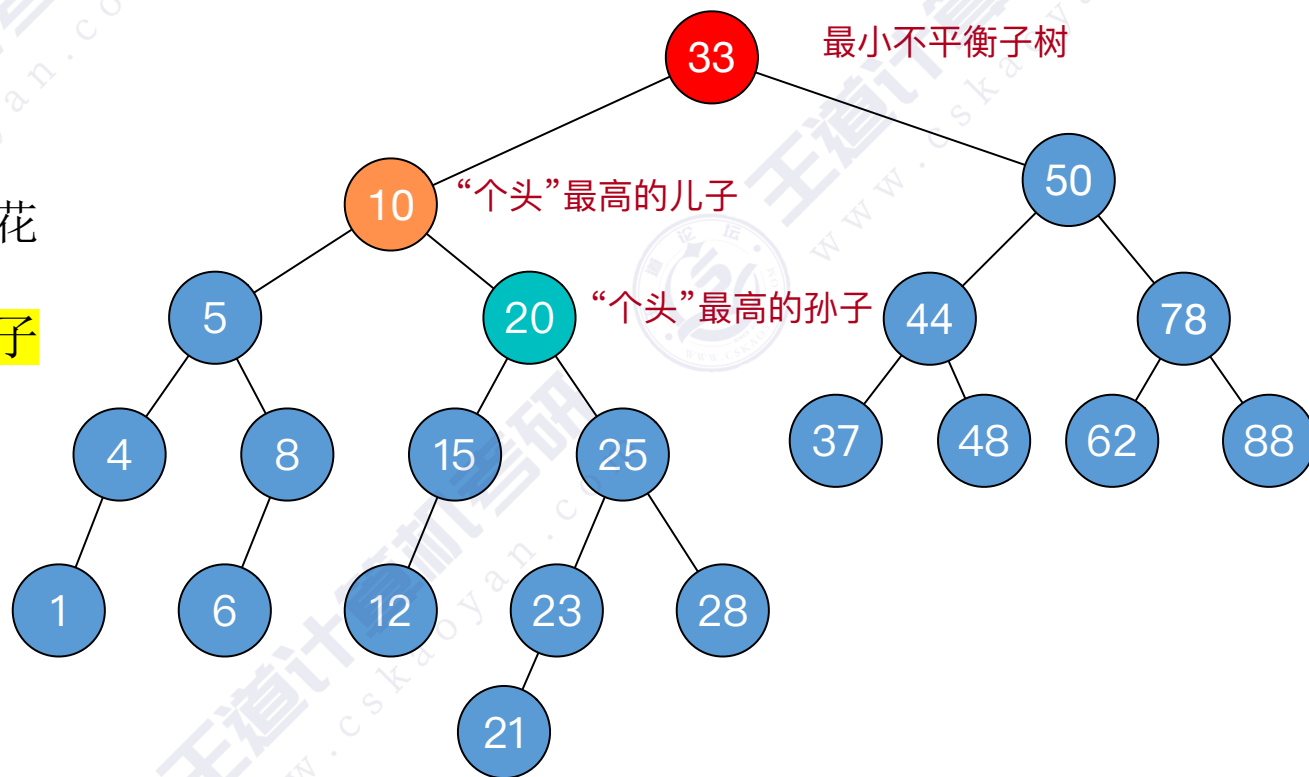
⑤如果不平衡向上传导，继续②



AVL树删除操作——例4

平衡二叉树的删除操作具体步骤：

- ①删除结点（方法同“二叉排序树”）
- ②一路向北找到最小不平衡子树，找不到就完结撒花
- ③找最小不平衡子树下，“个头”最高的儿子、孙子
- ④根据孙子的位置，调整平衡（LL/RR/LR/RL）
- ⑤如果不平衡向上传导，继续②

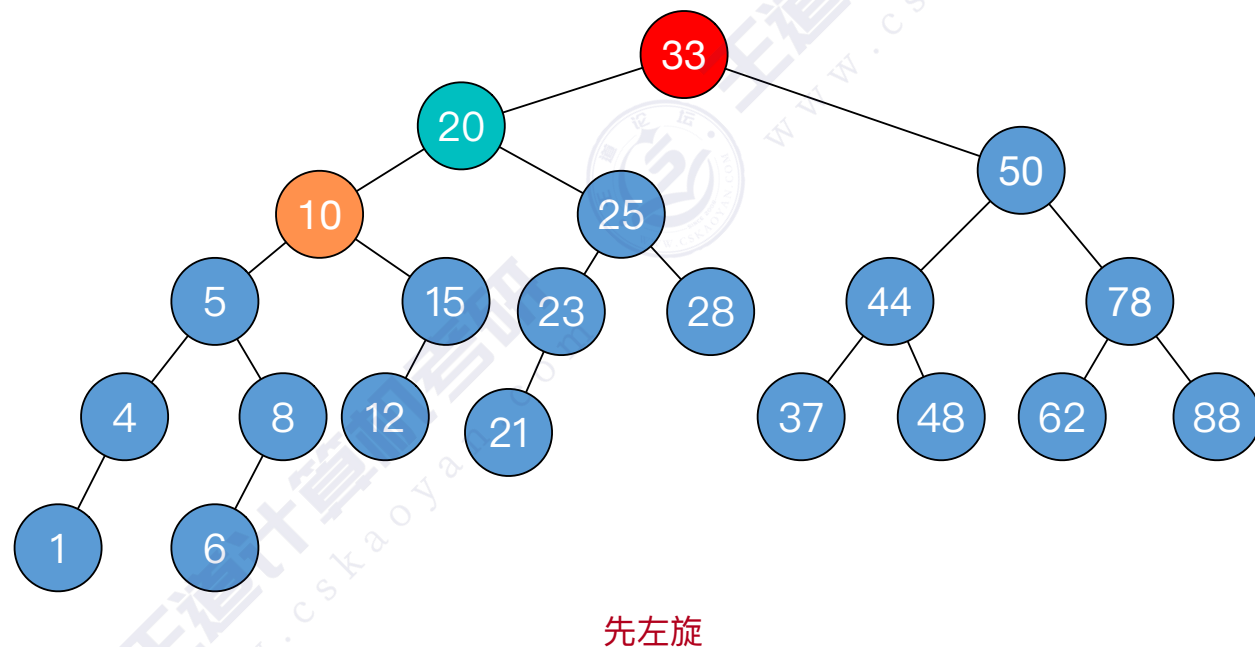


孙子在LR，孙子先左，再右旋

AVL树删除操作——例4

平衡二叉树的删除操作具体步骤：

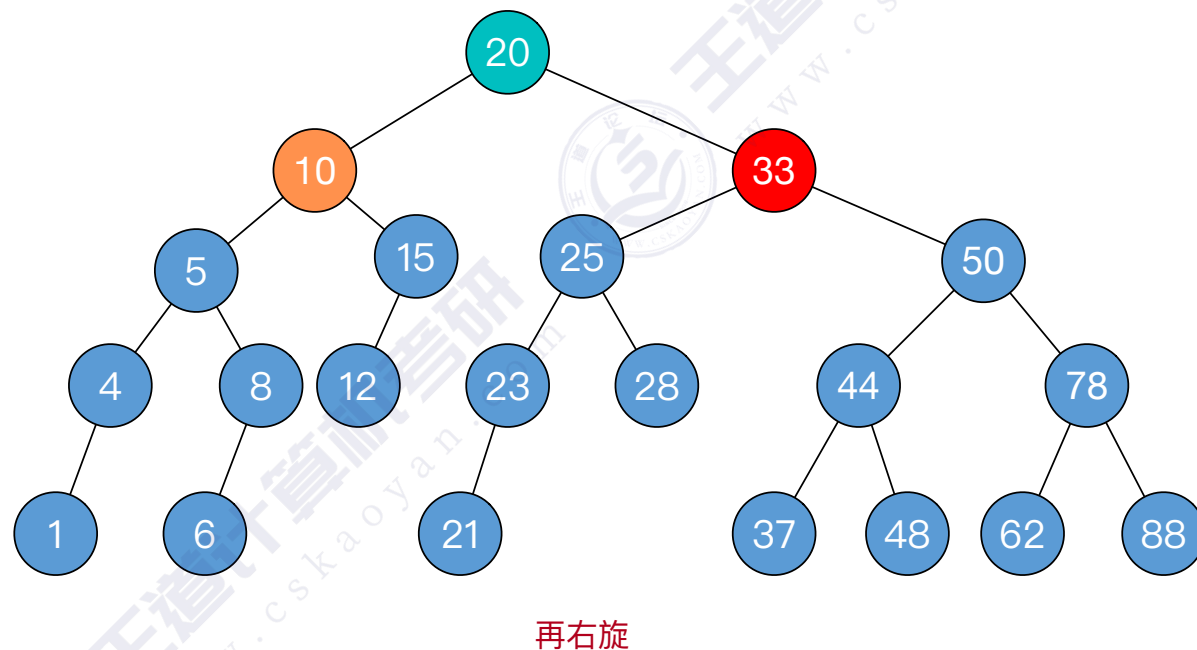
- ①删除结点（方法同“二叉排序树”）
- ②一路向北找到最小不平衡子树，找不到就完结撒花
- ③找最小不平衡子树下，“个头”最高的儿子、孙子
- ④根据孙子的位置，调整平衡（LL/RR/LR/RL）
 - 孙子在LL：儿子右单旋
 - 孙子在RR：儿子左单旋
 - 孙子在LR：孙子先左旋，再右旋
 - 孙子在RL：孙子先右旋，再左旋
- ⑤如果不平衡向上传导，继续②



AVL树删除操作——例4

平衡二叉树的删除操作具体步骤：

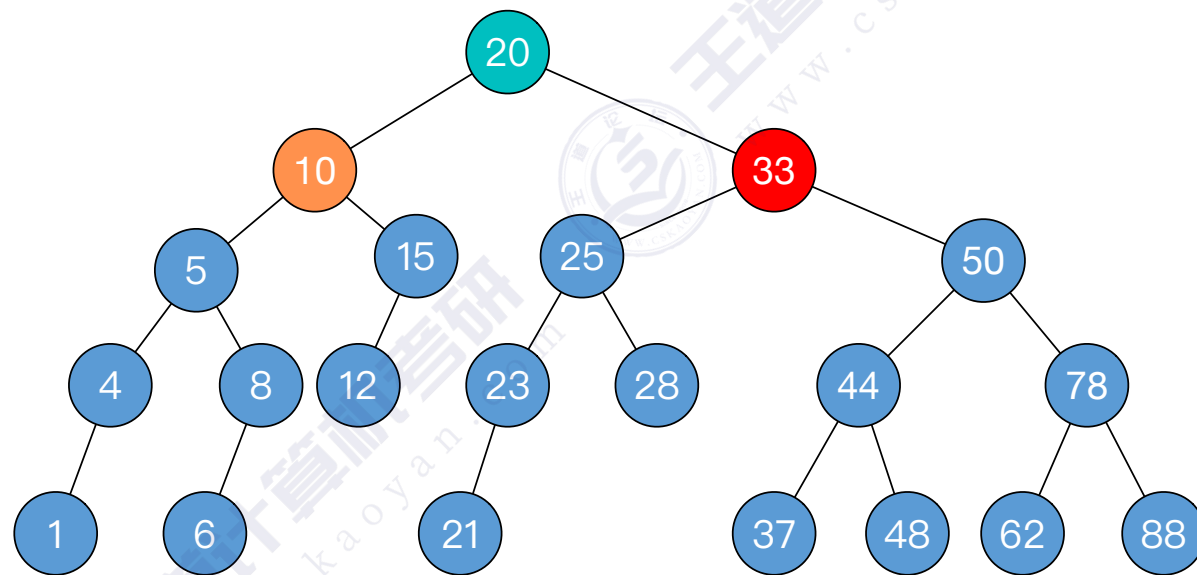
- ①删除结点（方法同“二叉排序树”）
- ②一路向北找到最小不平衡子树，找不到就完结撒花
- ③找最小不平衡子树下，“个头”最高的儿子、孙子
- ④根据孙子的位置，调整平衡（LL/RR/LR/RL）
 - 孙子在LL：儿子右单旋
 - 孙子在RR：儿子左单旋
 - 孙子在LR：孙子先左旋，再右旋
 - 孙子在RL：孙子先右旋，再左旋
- ⑤如果不平衡向上传导，继续②



AVL树删除操作——例4

平衡二叉树的**删除**操作具体步骤：

- ①删除结点（方法同“二叉排序树”）
- ②一路向北找到最小不平衡子树，找不到就完结撒花
- ③找最小不平衡子树下，“个头”最高的儿子、孙子
- ④根据孙子的位置，调整平衡（LL/RR/LR/RL）
- ⑤如果不平衡向上传导，继续②
 - 对最小不平衡子树的旋转可能导致树变矮，从而导致上层祖先不平衡（不平衡向上传递）



Over!



AVL树删除操作——例5

平衡二叉树的删除操作具体步骤：

①删除结点（方法同“二叉排序树”）

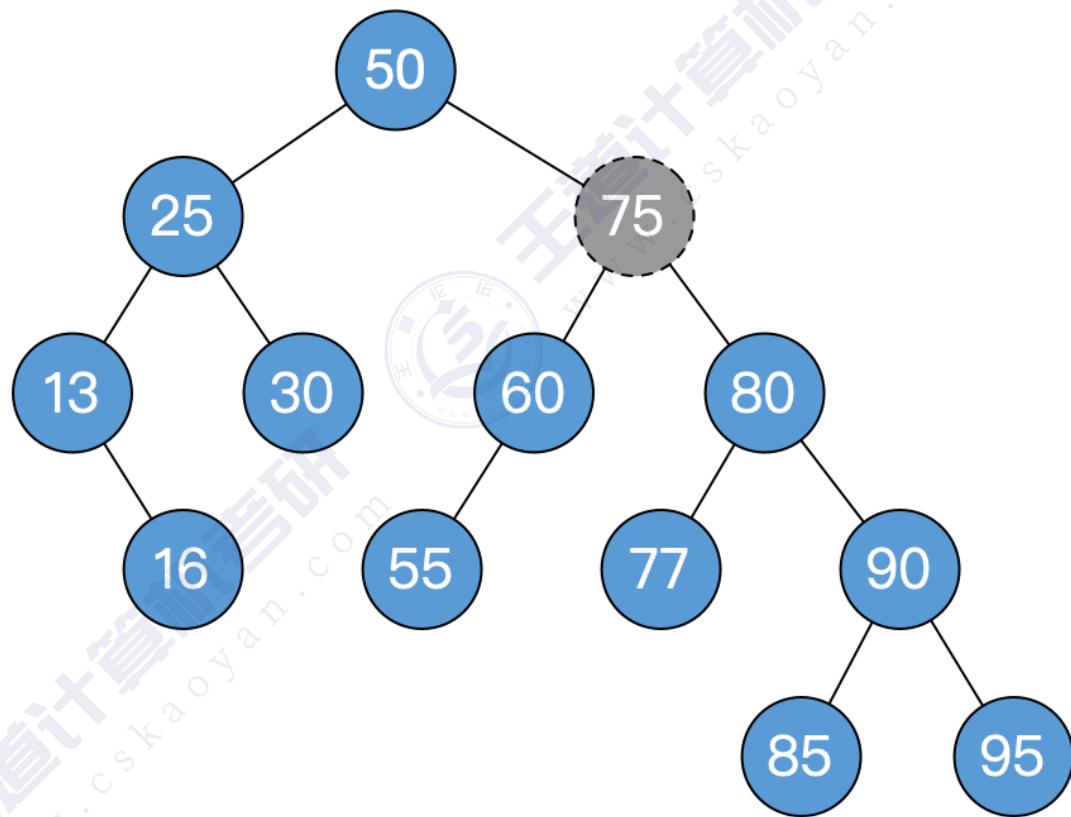
- 若删除的结点是叶子，直接删。
- 若删除的结点只有一个子树，用子树顶替删除位置
- 若删除的结点有两棵子树，用前驱（或后继）结点顶替，并转换为对前驱（或后继）结点的删除。

②一路向北找到最小不平衡子树，找不到就完结撒花

③找最小不平衡子树下，“个头”最高的儿子、孙子

④根据孙子的位置，调整平衡（LL/RR/LR/RL）

⑤如果不平衡向上传导，继续②



被删除结点有左右子树，用前驱结点顶替（复制数据即可）
并转化为对前驱结点的删除

AVL树删除操作——例5

平衡二叉树的删除操作具体步骤：

①删除结点（方法同“二叉排序树”）

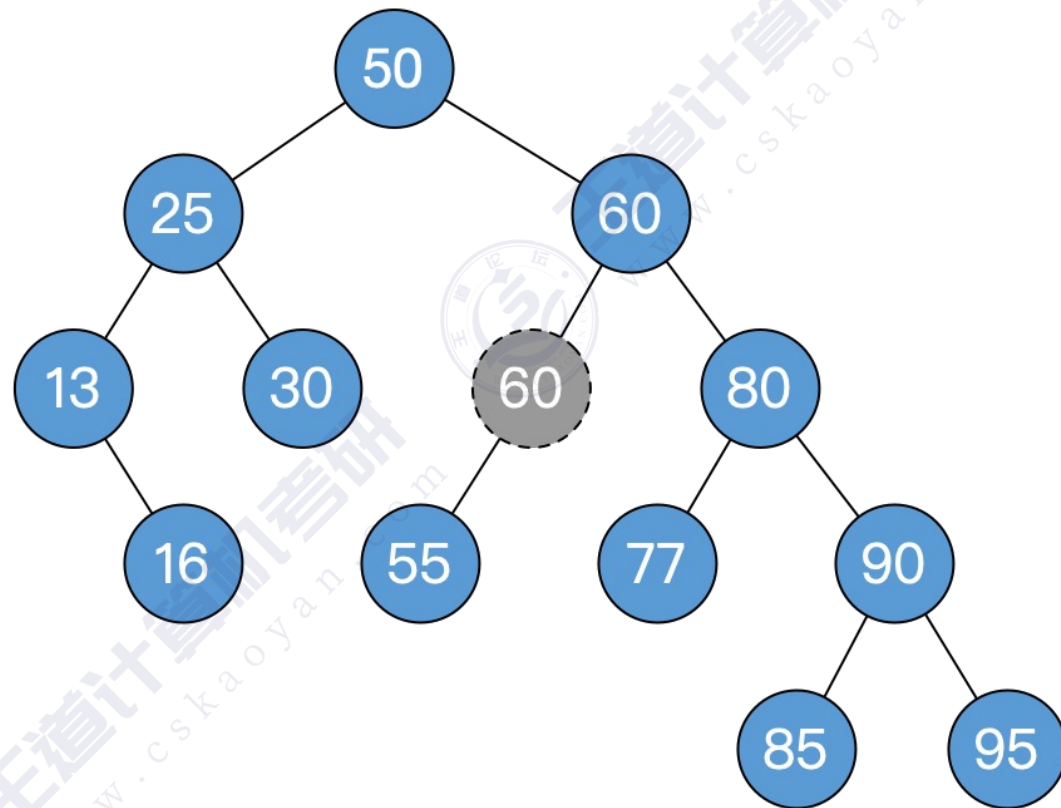
- 若删除的结点是叶子，直接删。
- 若删除的结点只有一个子树，用子树顶替删除位置
- 若删除的结点有两棵子树，用前驱（或后继）结点顶替，并转换为对前驱（或后继）结点的删除。

②一路向北找到最小不平衡子树，找不到就完结撒花

③找最小不平衡子树下，“个头”最高的儿子、孙子

④根据孙子的位置，调整平衡（LL/RR/LR/RL）

⑤如果不平衡向上传导，继续②



被删除结点只有左子树，用子树顶替删除位置（用结点实体顶替）

AVL树删除操作——例5

平衡二叉树的删除操作具体步骤：

①删除结点（方法同“二叉排序树”）

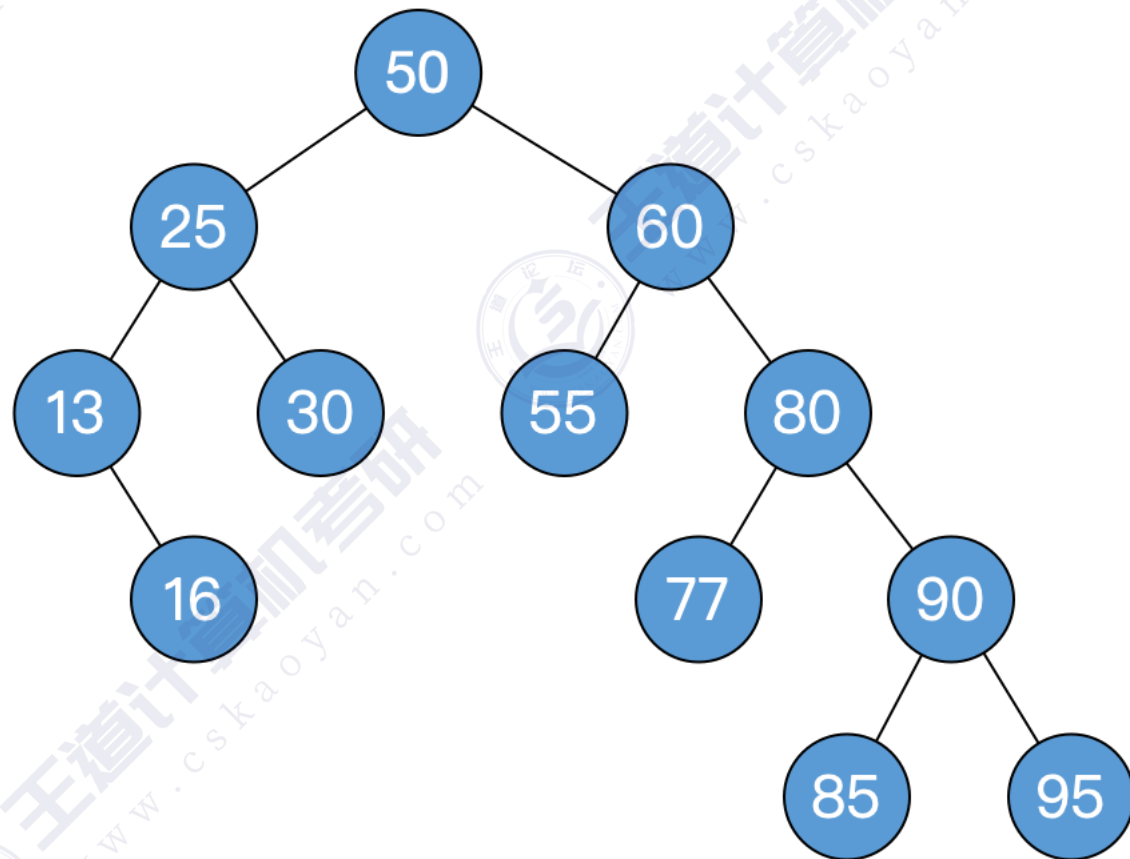
- 若删除的结点是叶子，直接删。
- 若删除的结点只有一个子树，用子树顶替删除位置
- 若删除的结点有两棵子树，用前驱（或后继）结点顶替，并转换为对前驱（或后继）结点的删除。

②一路向北找到最小不平衡子树，找不到就完结撒花

③找最小不平衡子树下，“个头”最高的儿子、孙子

④根据孙子的位置，调整平衡（LL/RR/LR/RL）

⑤如果不平衡向上传导，继续②



AVL树删除操作——例5

平衡二叉树的删除操作具体步骤：

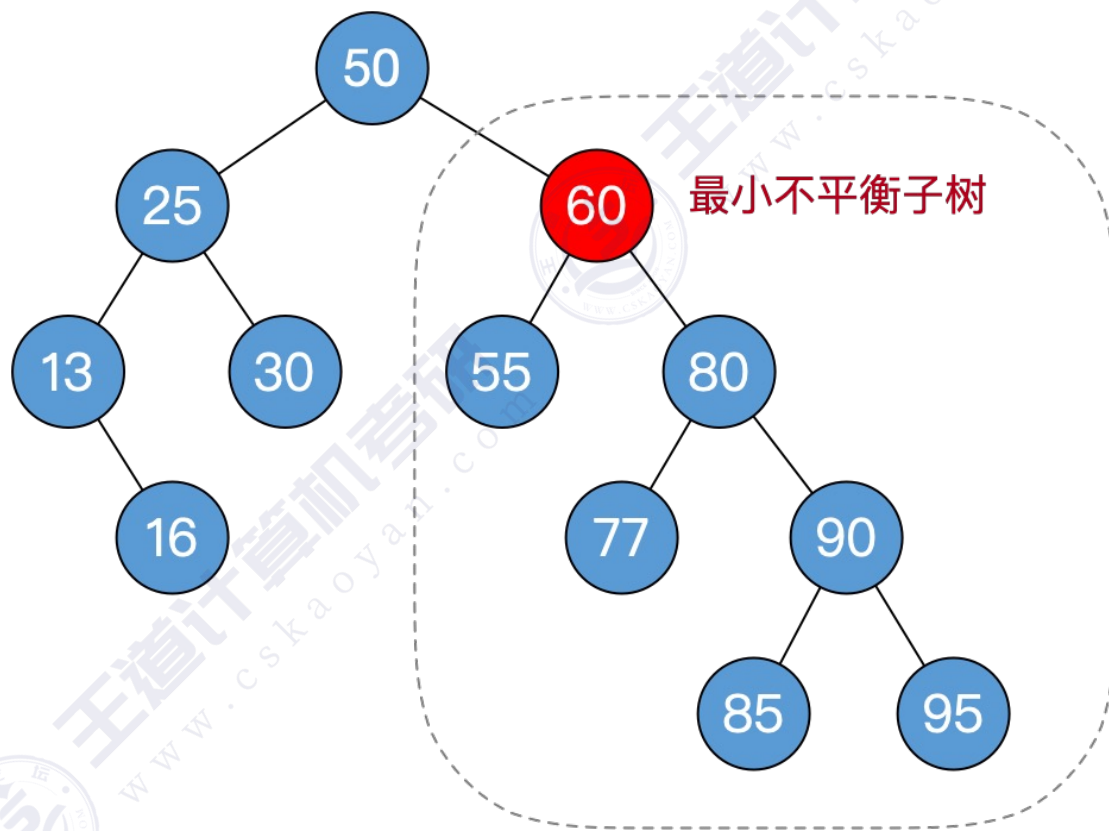
①删除结点（方法同“二叉排序树”）

②一路向北找到最小不平衡子树，找不到就完结撒花

③找最小不平衡子树下，“个头”最高的儿子、孙子

④根据孙子的位置，调整平衡（LL/RR/LR/RL）

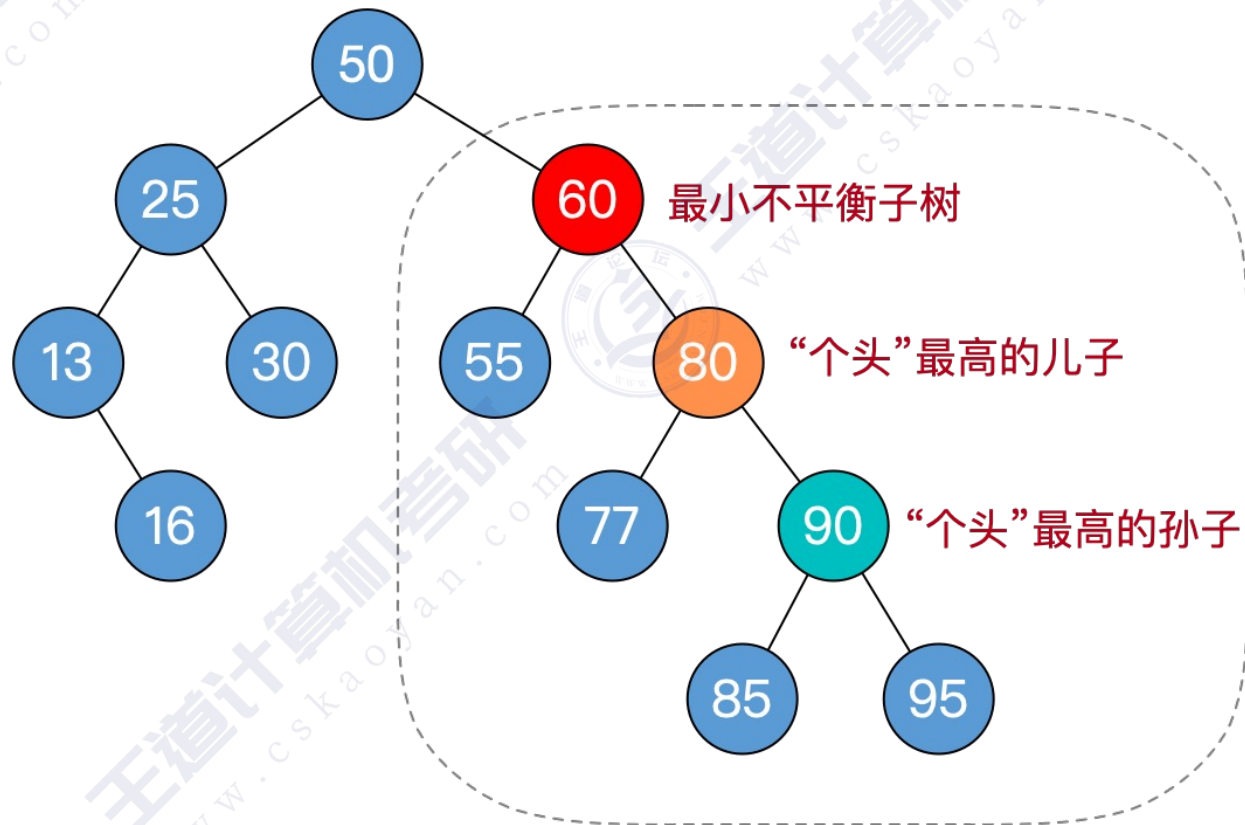
⑤如果不平衡向上传导，继续②



AVL树删除操作——例5

平衡二叉树的**删除**操作具体步骤：

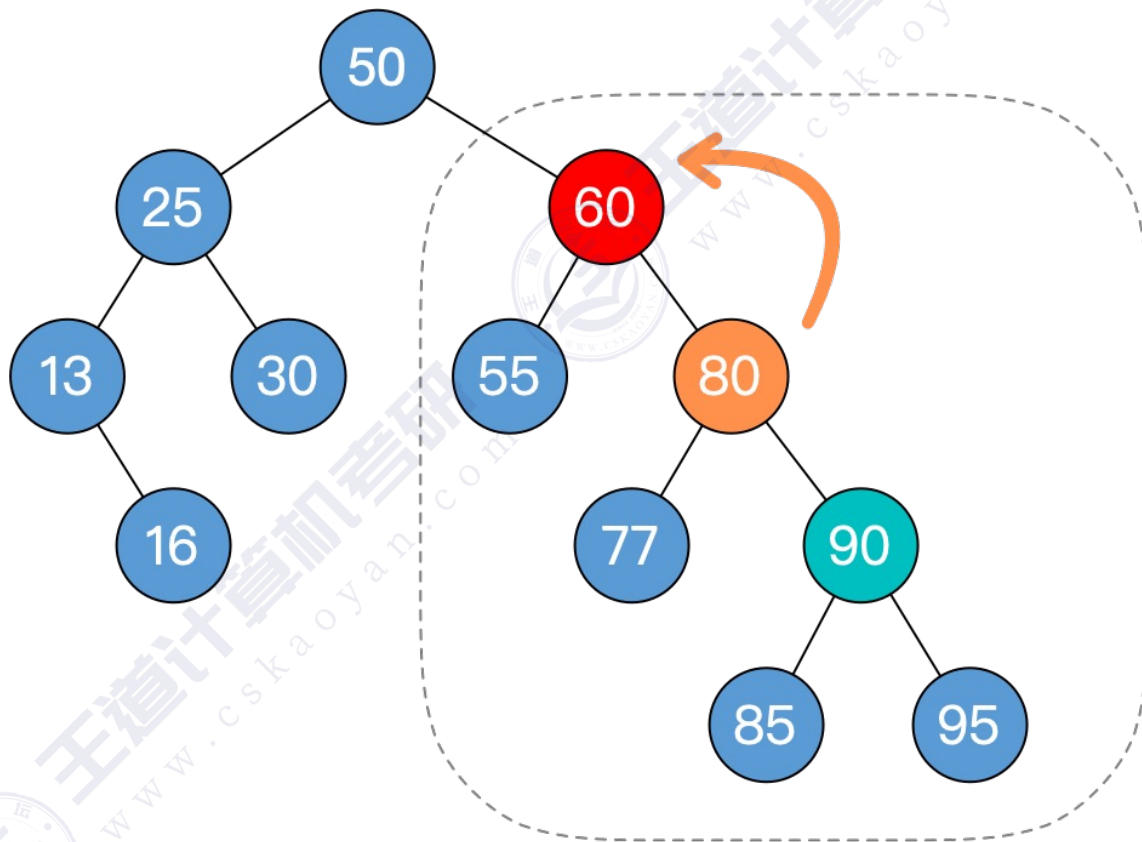
- ①删除结点（方法同“二叉排序树”）
- ②一路向北找到最小不平衡子树，找不到就完结撒花
- ③找最小不平衡子树下，“个头”最高的儿子、孙子
- ④根据孙子的位置，调整平衡（LL/RR/LR/RL）
- ⑤如果不平衡向上传导，继续②



AVL树删除操作——例5

平衡二叉树的删除操作具体步骤：

- ①删除结点（方法同“二叉排序树”）
- ②一路向北找到最小不平衡子树，找不到就完结撒花
- ③找最小不平衡子树下，“个头”最高的儿子、孙子
- ④根据孙子的位置，调整平衡（LL/RR/LR/RL）
 - 孙子在LL：儿子右单旋
 - 孙子在RR：儿子左单旋
 - 孙子在LR：孙子先左旋，再右旋
 - 孙子在RL：孙子先右旋，再左旋
- ⑤如果不平衡向上传导，继续②

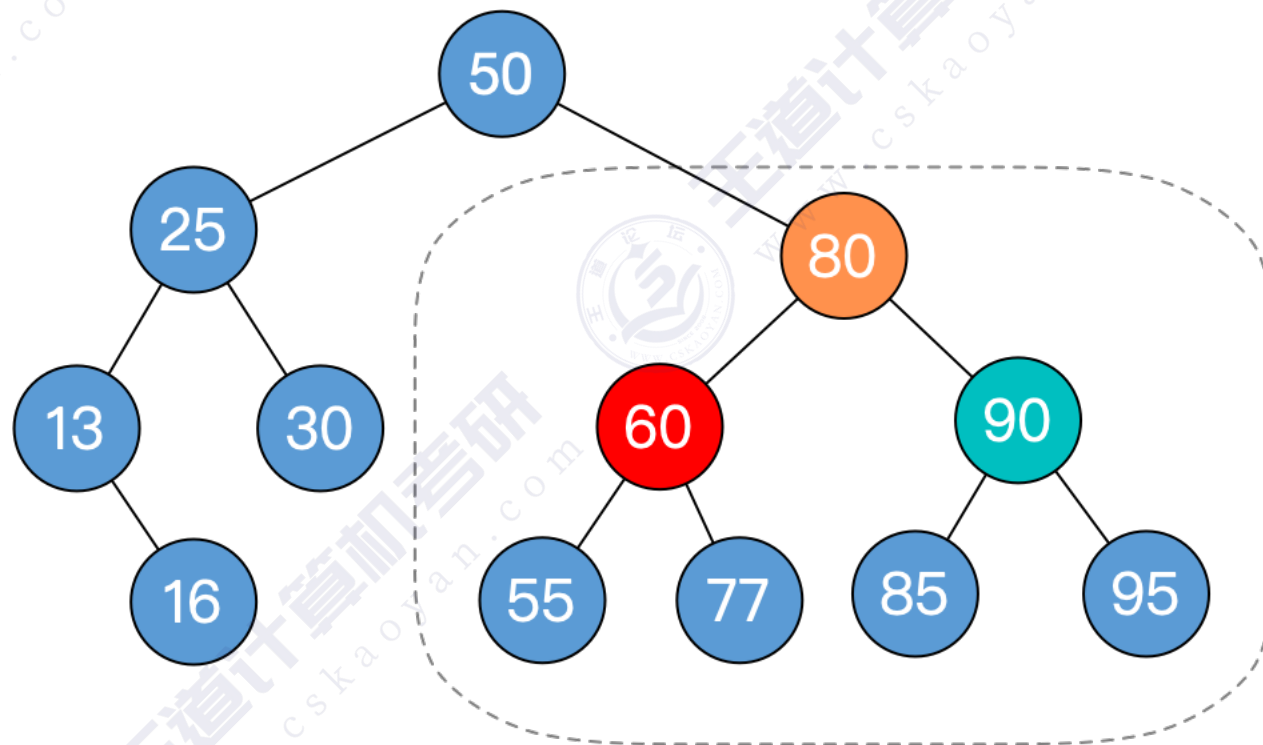


孙子在RR，儿子左单旋

AVL树删除操作——例5

平衡二叉树的**删除**操作具体步骤：

- ①删除结点（方法同“二叉排序树”）
- ②一路向北找到最小不平衡子树，找不到就完结撒花
- ③找最小不平衡子树下，“个头”最高的儿子、孙子
- ④根据孙子的位置，调整平衡（LL/RR/LR/RL）
- ⑤如果不平衡向上传导，继续②
 - 对最小不平衡子树的旋转可能导致树变矮，从而导致上层祖先不平衡（不平衡向上传递）



Over!



AVL树删除操作——例6

平衡二叉树的**删除**操作具体步骤:

①删除结点（方法同“二叉排序树”）

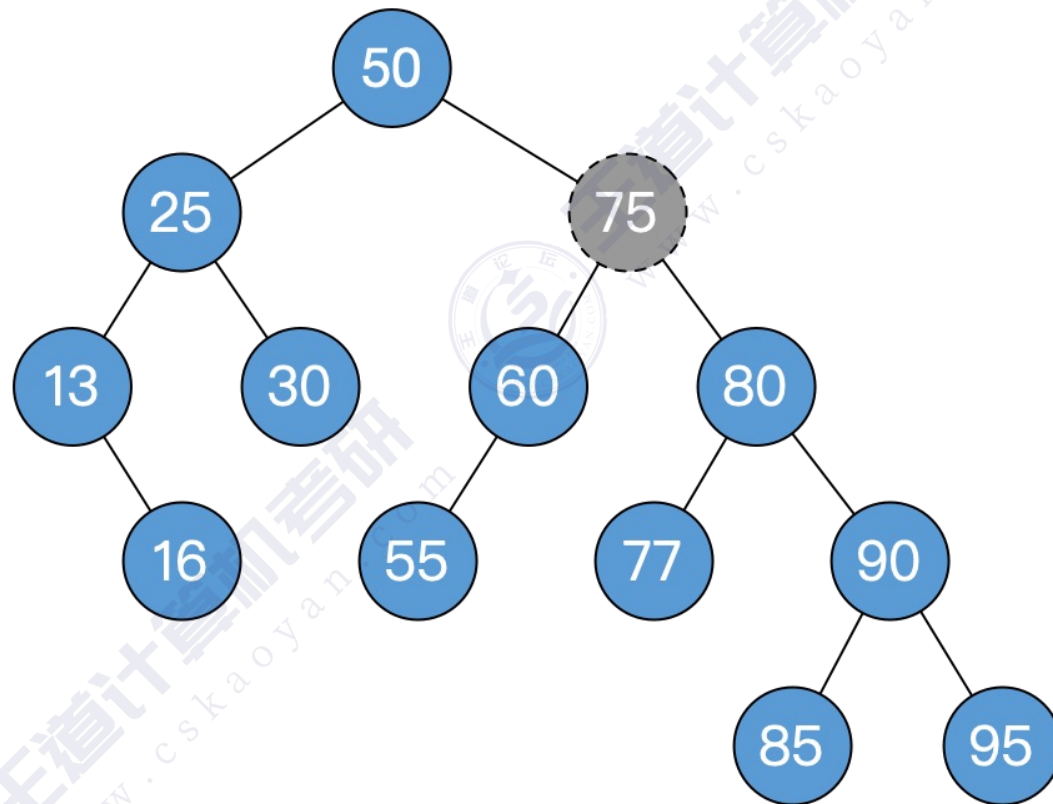
- 若删除的结点是叶子，直接删。
- 若删除的结点只有一个子树，用子树顶替删除位置
- 若删除的结点有两棵子树，用前驱（或后继）结点顶替，并转换为对前驱（或后继）结点的删除。

②一路向北找到最小不平衡子树，找不到就完结撒花

③找最小不平衡子树下，“个头”最高的儿子、孙子

④根据孙子的位置，调整平衡（LL/RR/LR/RL）

⑤如果不平衡向上传导，继续②



被删除结点有左右子树，用后继结点顶替（复制数据即可）
并转化为对后继结点的删除

AVL树删除操作——例6

平衡二叉树的删除操作具体步骤：

①删除结点（方法同“二叉排序树”）

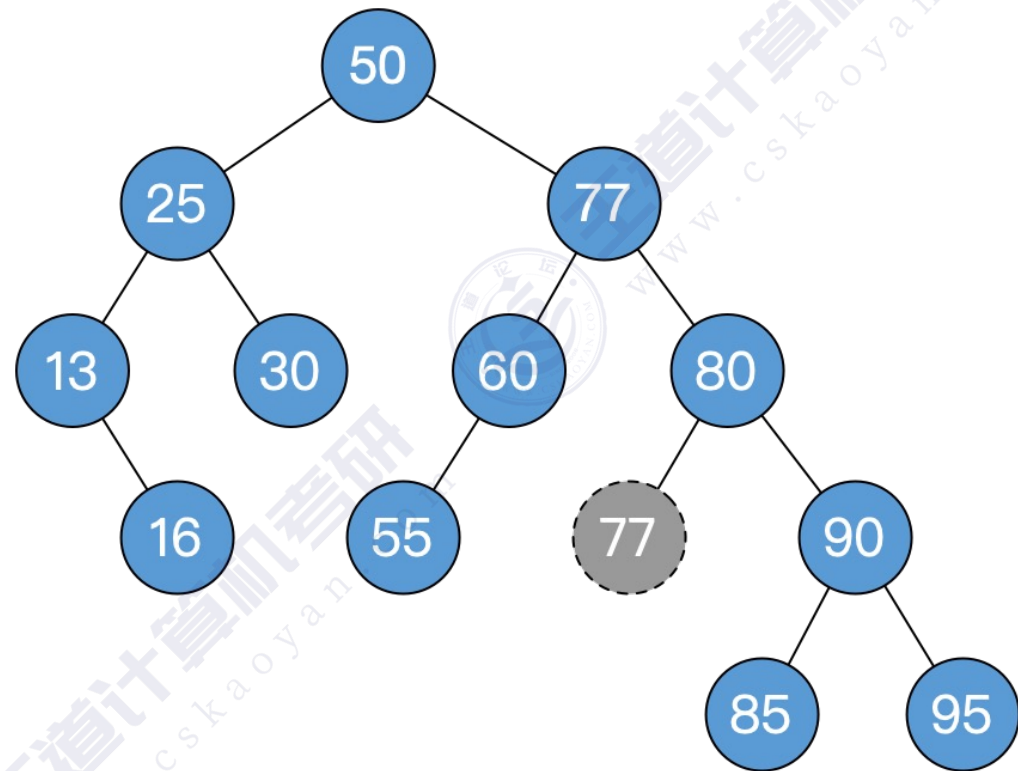
- 若删除的结点是叶子，直接删。
- 若删除的结点只有一个子树，用子树顶替删除位置
- 若删除的结点有两棵子树，用前驱（或后继）结点顶替，并转换为对前驱（或后继）结点的删除。

②一路向北找到最小不平衡子树，找不到就完结撒花

③找最小不平衡子树下，“个头”最高的儿子、孙子

④根据孙子的位置，调整平衡（LL/RR/LR/RL）

⑤如果不平衡向上传导，继续②



被删除结点为叶子，直接删即可

AVL树删除操作——例6

平衡二叉树的删除操作具体步骤：

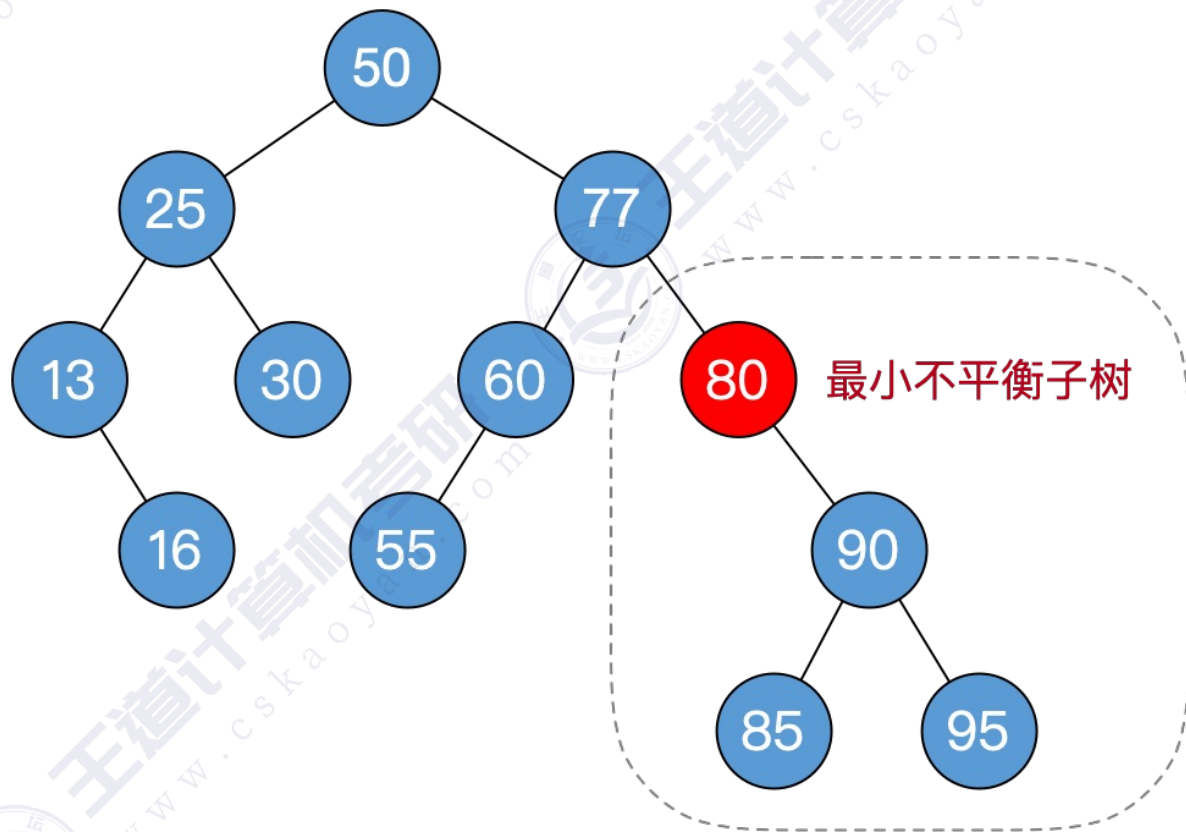
①删除结点（方法同“二叉排序树”）

②一路向北找到最小不平衡子树，找不到就完结撒花

③找最小不平衡子树下，“个头”最高的儿子、孙子

④根据孙子的位置，调整平衡（LL/RR/LR/RL）

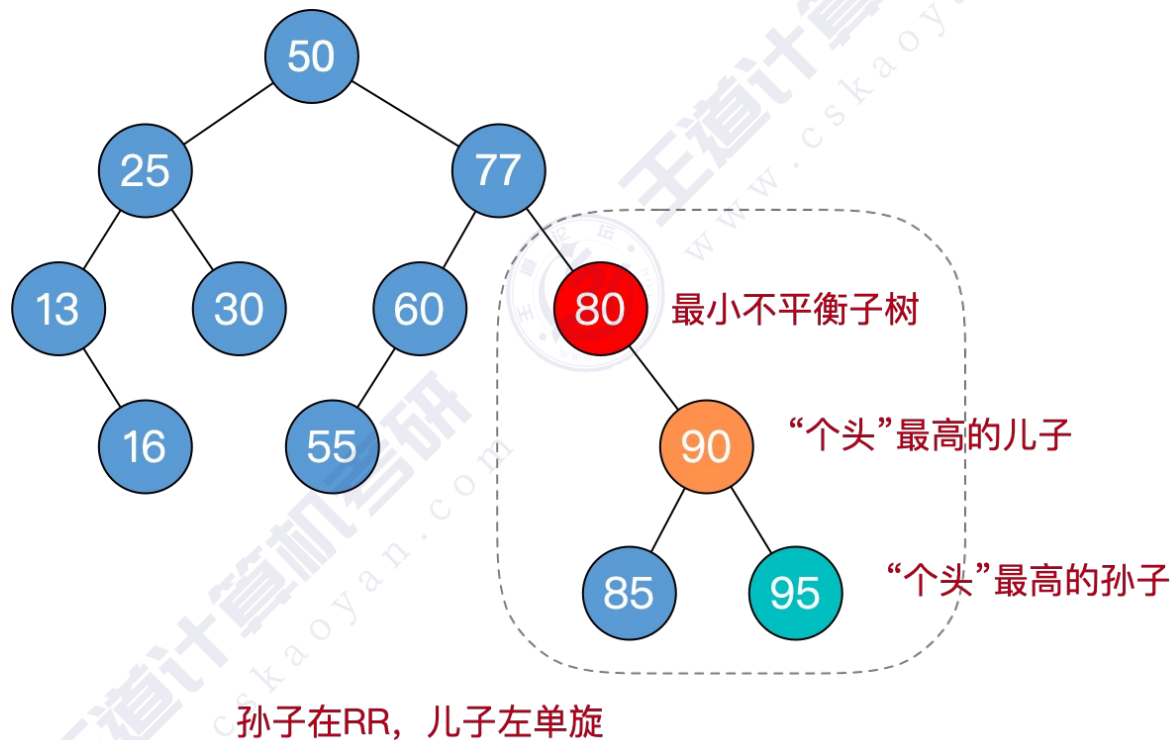
⑤如果不平衡向上传导，继续②



AVL树删除操作——例6

平衡二叉树的删除操作具体步骤：

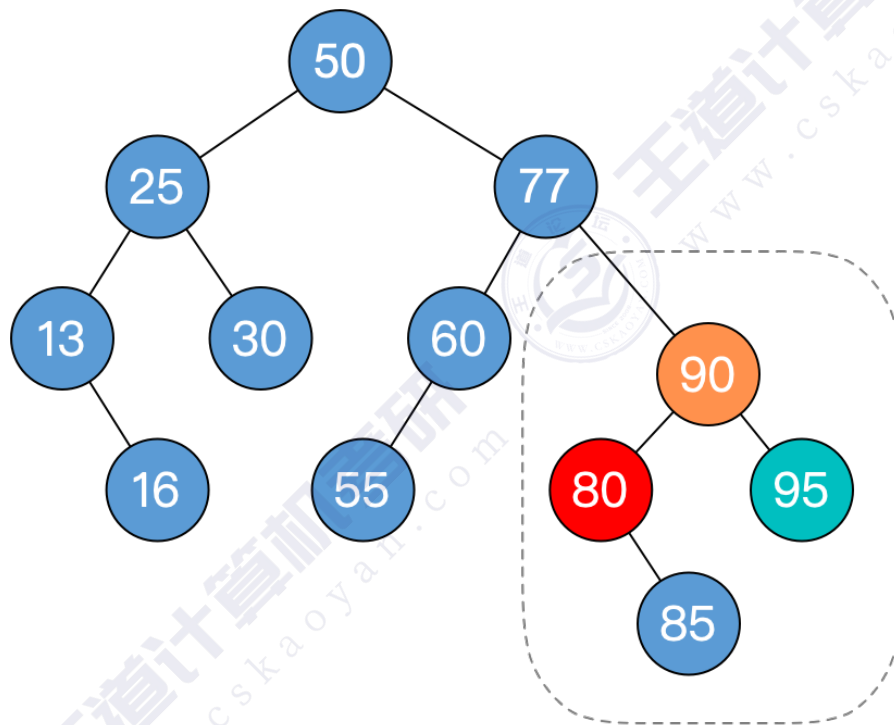
- ①删除结点（方法同“二叉排序树”）
- ②一路向北找到最小不平衡子树，找不到就完结撒花
- ③找最小不平衡子树下，“个头”最高的儿子、孙子
- ④根据孙子的位置，调整平衡（LL/RR/LR/RL）
- ⑤如果不平衡向上传导，继续②



AVL树删除操作——例6

平衡二叉树的删除操作具体步骤：

- ①删除结点（方法同“二叉排序树”）
- ②一路向北找到最小不平衡子树，找不到就完结撒花
- ③找最小不平衡子树下，“个头”最高的儿子、孙子
- ④根据孙子的位置，调整平衡（LL/RR/LR/RL）
 - 孙子在LL：儿子右单旋
 - 孙子在RR：儿子左单旋
 - 孙子在LR：孙子先左旋，再右旋
 - 孙子在RL：孙子先右旋，再左旋
- ⑤如果不平衡向上传导，继续②

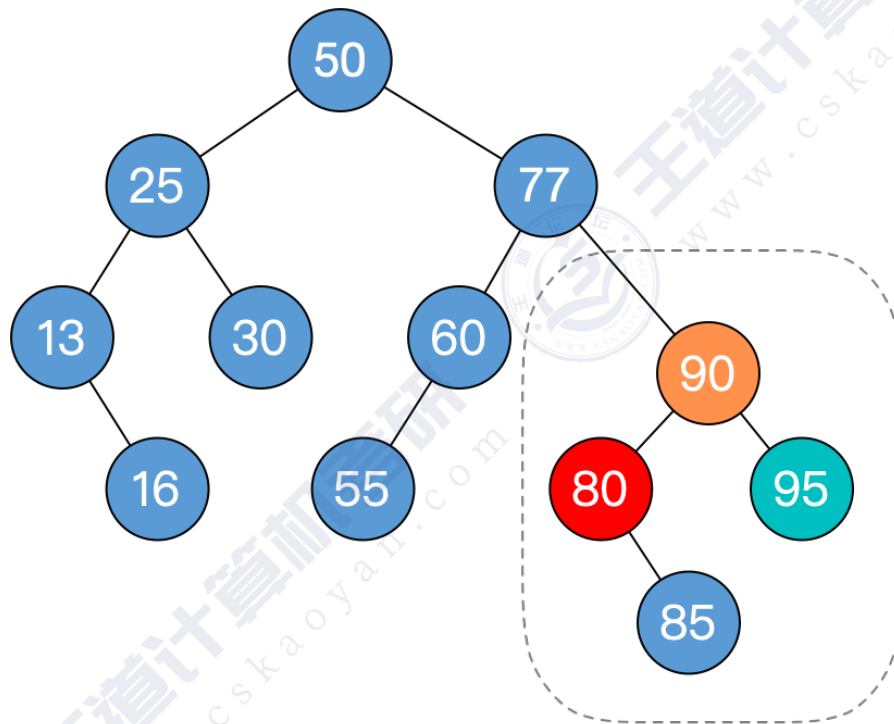


儿子左单旋

AVL树删除操作——例6

平衡二叉树的**删除**操作具体步骤：

- ①删除结点（方法同“二叉排序树”）
- ②一路向北找到最小不平衡子树，找不到就完结撒花
- ③找最小不平衡子树下，“个头”最高的儿子、孙子
- ④根据孙子的位置，调整平衡（LL/RR/LR/RL）
- ⑤如果不平衡向上传导，继续②
 - 对最小不平衡子树的旋转可能导致树变矮，从而导致上层祖先不平衡（不平衡向上传递）



Over!



AVL树删除操作——例6

平衡二叉树的删除操作具体步骤：

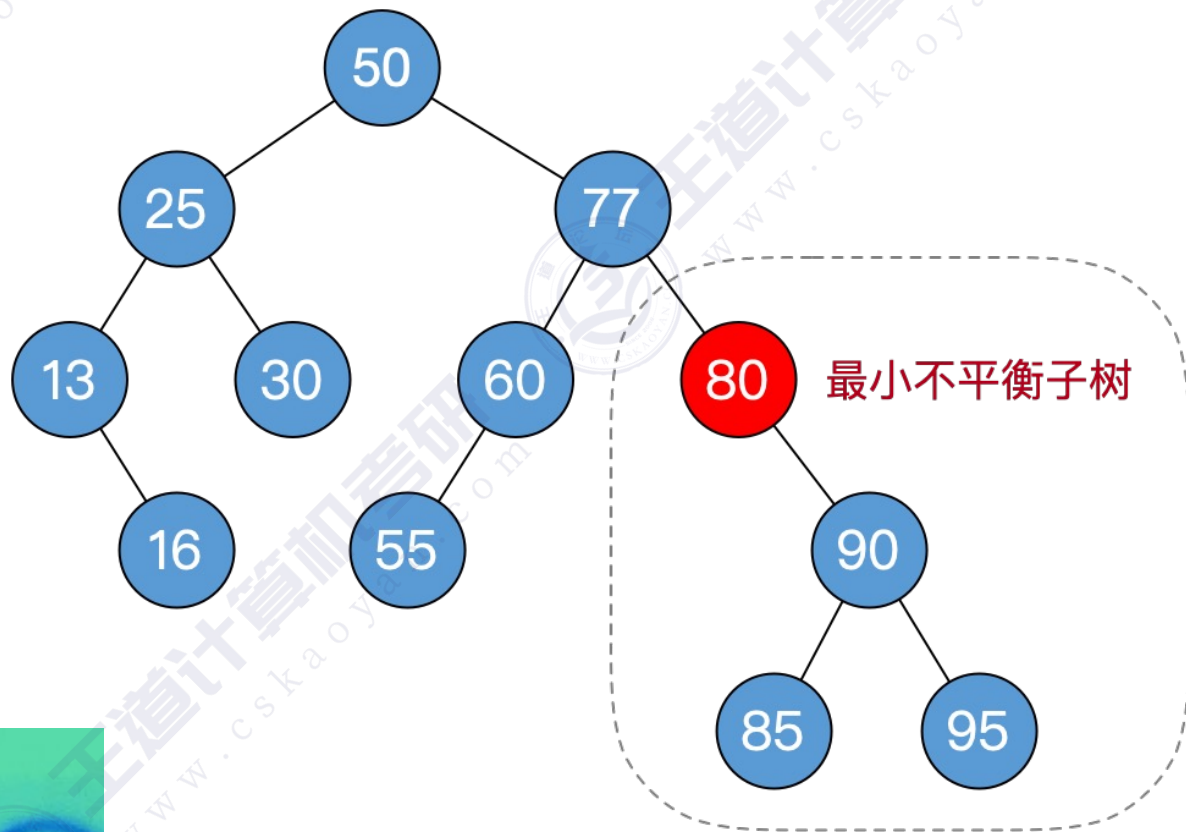
①删除结点（方法同“二叉排序树”）

②一路向北找到最小不平衡子树，找不到就完结撒花

③找最小不平衡子树下，“个头”最高的儿子、孙子

④根据孙子的位置，调整平衡（LL/RR/LR/RL）

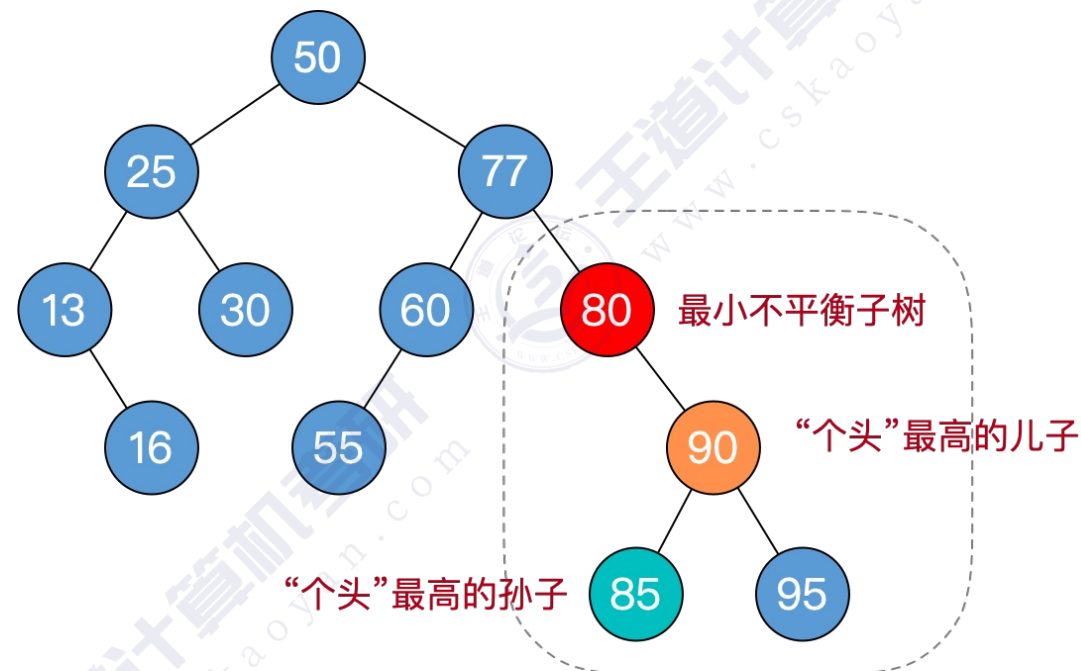
⑤如果不平衡向上传导，继续②



AVL树删除操作——例6

平衡二叉树的删除操作具体步骤：

- ①删除结点（方法同“二叉排序树”）
- ②一路向北找到最小不平衡子树，找不到就完结撒花
- ③找最小不平衡子树下，“个头”最高的儿子、孙子
- ④根据孙子的位置，调整平衡（LL/RR/LR/RL）
- ⑤如果不平衡向上传导，继续②



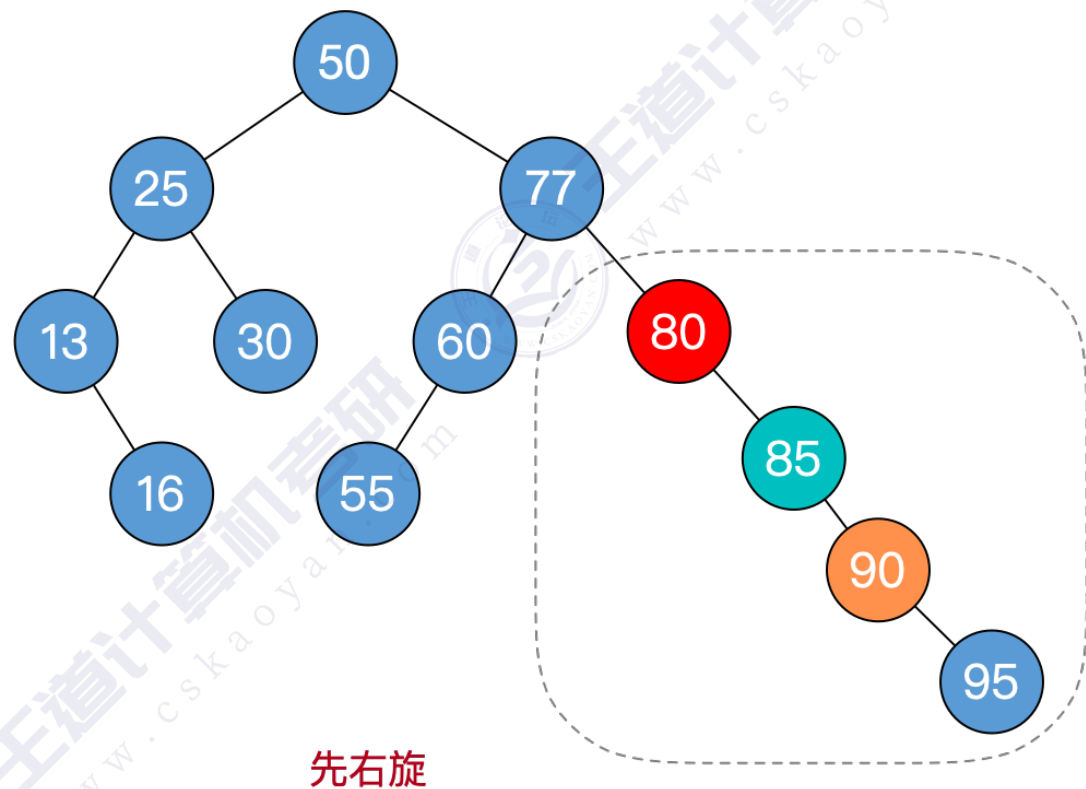
孙子在RL，孙子先右旋，再左旋



AVL树删除操作——例6

平衡二叉树的删除操作具体步骤：

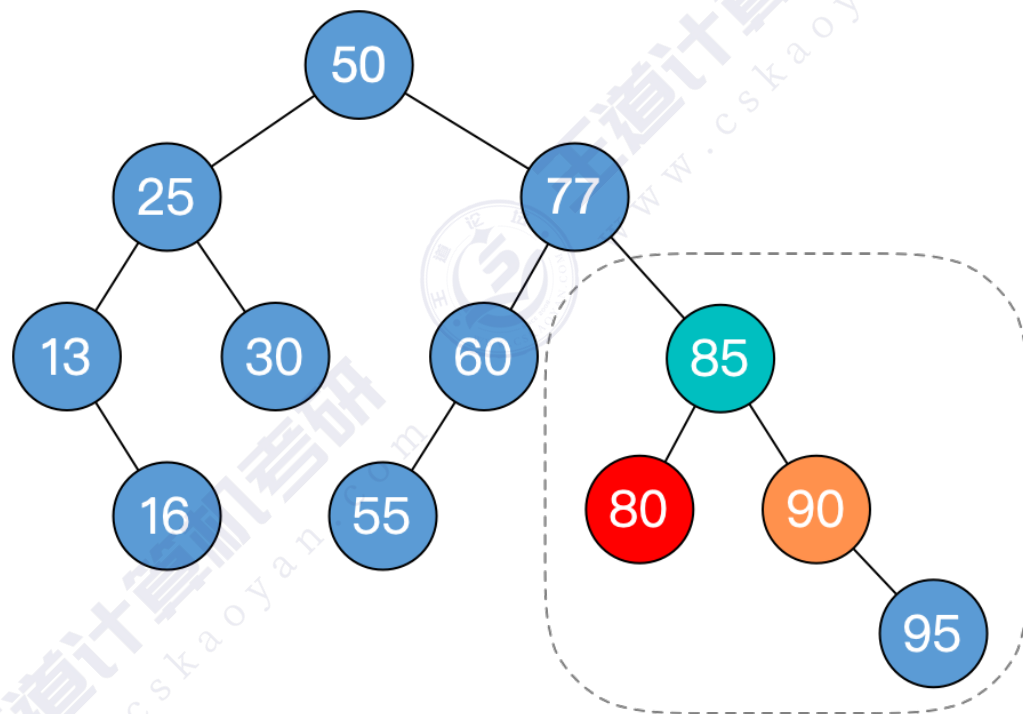
- ①删除结点（方法同“二叉排序树”）
- ②一路向北找到最小不平衡子树，找不到就完结撒花
- ③找最小不平衡子树下，“个头”最高的儿子、孙子
- ④根据孙子的位置，调整平衡（LL/RR/LR/RL）
 - 孙子在LL：儿子右单旋
 - 孙子在RR：儿子左单旋
 - 孙子在LR：孙子先左旋，再右旋
 - 孙子在RL：孙子先右旋，再左旋
- ⑤如果不平衡向上传导，继续②



AVL树删除操作——例6

平衡二叉树的删除操作具体步骤：

- ①删除结点（方法同“二叉排序树”）
- ②一路向北找到最小不平衡子树，找不到就完结撒花
- ③找最小不平衡子树下，“个头”最高的儿子、孙子
- ④根据孙子的位置，调整平衡（LL/RR/LR/RL）
 - 孙子在LL：儿子右单旋
 - 孙子在RR：儿子左单旋
 - 孙子在LR：孙子先左旋，再右旋
 - 孙子在RL：孙子先右旋，再左旋
- ⑤如果不平衡向上传导，继续②

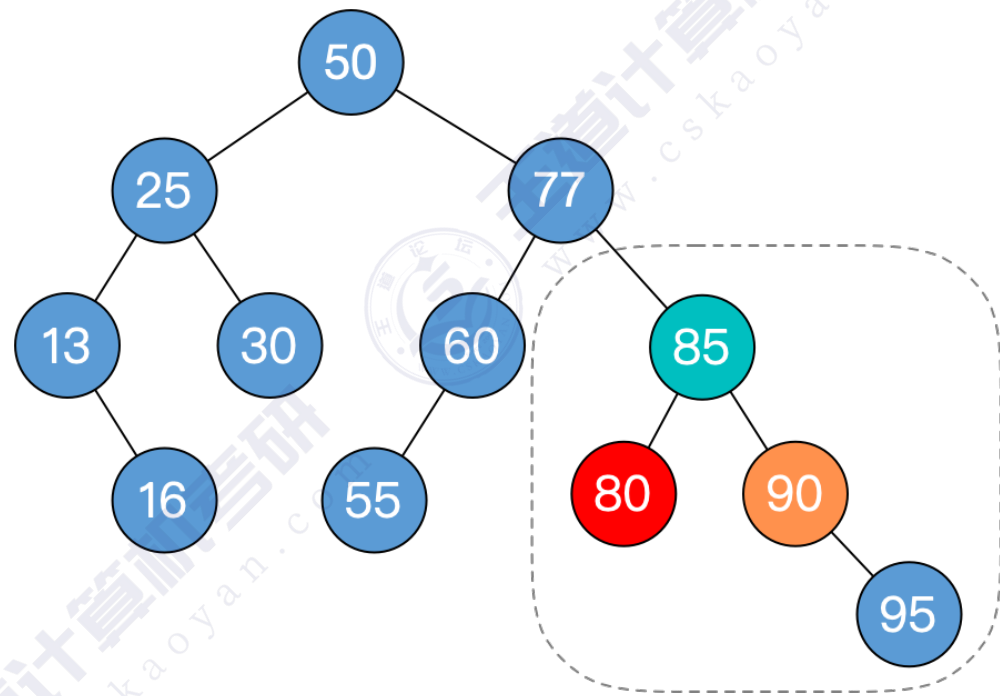


再左旋

AVL树删除操作——例6

平衡二叉树的**删除**操作具体步骤：

- ①删除结点（方法同“二叉排序树”）
- ②一路向北找到最小不平衡子树，找不到就完结撒花
- ③找最小不平衡子树下，“个头”最高的儿子、孙子
- ④根据孙子的位置，调整平衡（LL/RR/LR/RL）
- ⑤如果不平衡向上传导，继续②
 - 对最小不平衡子树的旋转可能导致树变矮，从而导致上层祖先不平衡（不平衡向上传递）



Over!



不可能考这种有多种处理方式的题目！

知识回顾与重要考点

平衡二叉树的**删除**操作具体步骤：

①删除结点（方法同“二叉排序树”）

- 若删除的结点是叶子，直接删。
- 若删除的结点只有一个子树，用子树顶替删除位置
- 若删除的结点有两棵子树，用前驱（或后继）结点顶替，并转换为对前驱（或后继）结点的删除。

②一路向北找到最小不平衡子树，找不到就完结撒花

③找最小不平衡子树下，“**个头**”最高的儿子、孙子

④根据孙子的位置，调整平衡（LL/RR/LR/RL）

- 孙子在LL：儿子右单旋
- 孙子在RR：儿子左单旋
- 孙子在LR：孙子先左旋，再右旋
- 孙子在RL：孙子先右旋，再左旋

⑤如果不平衡向上传导，继续②

- 对最小不平衡子树的**旋转可能导致树变矮**，从而导致上层祖先不平衡（不平衡向上传递）

平衡二叉树删除操作时间复杂度= $O(\log_2 n)$

本节内容

红黑树

(Red-Black Tree)
RBT

为什么要发明 红黑树?

		BST	AVL Tree	Red-Black Tree
生日		1960	1962	1972
时间复杂度	Search (查)	$O(n)$	$O(\log_2 n)$	$O(\log_2 n)$
	Insert (插)	$O(n)$	$O(\log_2 n)$	$O(\log_2 n)$
	Delete (删)	$O(n)$	$O(\log_2 n)$	$O(\log_2 n)$



平衡二叉树 AVL: 插入/删除 很容易破坏“平衡”特性，需要频繁调整树的形态。如：插入操作导致不平衡，则需要先计算平衡因子，找到最小不平衡子树（时间开销大），再进行 LL/RR/LR/RL 调整

红黑树 RBT: 插入/删除 很多时候不会破坏“红黑”特性，无需频繁调整树的形态。即便需要调整，一般都可以在常数级时间内完成

平衡二叉树: 适用于以查为主、很少插入/删除的场景

红黑树: 适用于频繁插入、删除的场景，实用性更强

红黑树大概会怎么考?

红黑树的定义、性质——选择题

红黑树的插入/删除——要能手绘插入过程（不太可能考代码，略复杂），删除操作也比较麻烦，也许不考

4. 现有一棵无重复关键字的平衡二叉树（AVL 树），对其进行中序遍历可得到一个降序序列。下列关于该平衡二叉树的叙述中，正确的是_____。↵

A. 根结点的度一定为 2

B. 树中最小元素一定是叶结点↵

C. 最后插入的元素一定是叶结点

D. 树中最大元素一定是无左子树↵

2015真题

3. 若将关键字 1, 2, 3, 4, 5, 6, 7 依次插入到初始为空的平衡二叉树 T 中，则 T 中平衡因子为 0 的分支结点的个数是_____。↵

A. 0

B. 1

C. 2

D. 3↵

2013真题



知识总览

红黑树

定义、性质



插入



删除

红黑树的定义

红黑树是二叉排序树



左子树结点值 $\leq$ 根结点值 $\leq$ 右子树结点值

与普通BST相比，有什么要求



- ①每个结点或是红色，或是黑色的
- ②根节点是黑色的
- ③叶结点（外部结点、NULL结点、失败结点）均是黑色的
- ④不存在两个相邻的红结点（即红结点的父节点和孩子结点均是黑色）
- ⑤对每个结点，从该节点到任一叶结点的简单路径上，所含黑结点的数目相同

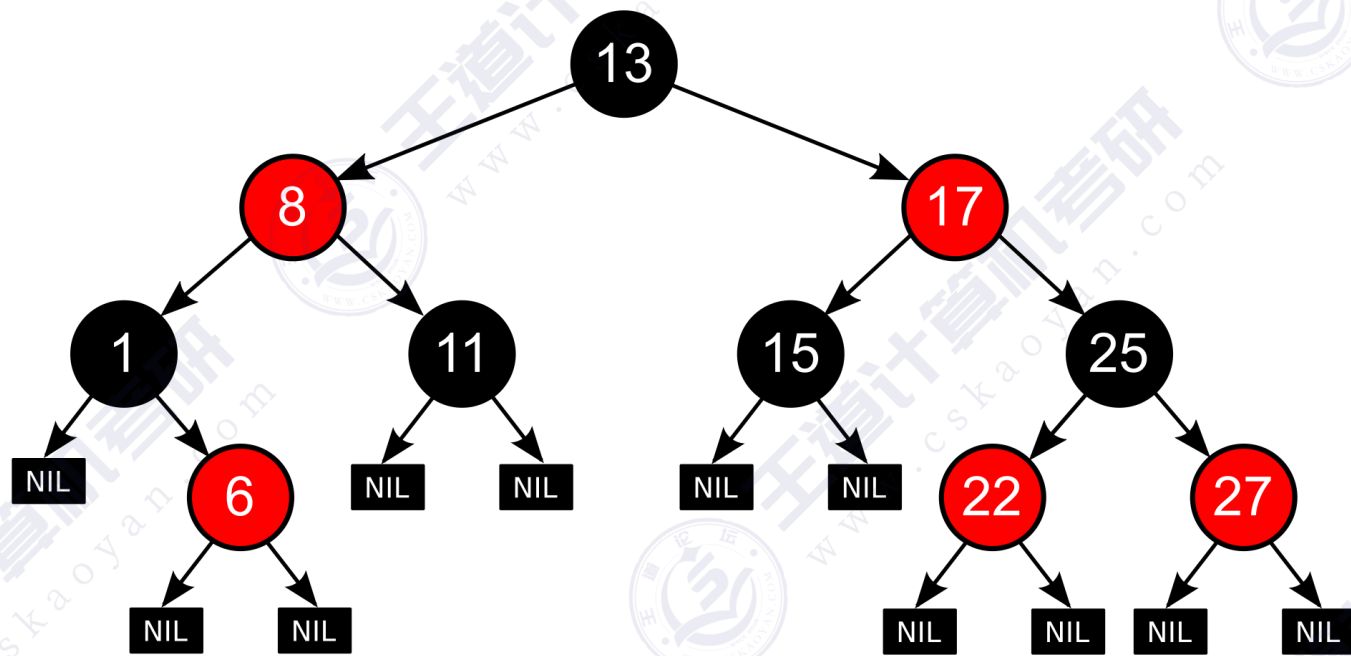
左根右，根叶黑
不红红，黑路同



张口就是freestyle

```
struct RBnode {           // 红黑树的结点定义
    int key;                // 关键字的值
    RBnode* parent;         // 父节点指针
    RBnode* lChild;         // 左孩子指针
    RBnode* rChild;         // 右孩子指针
    int color;              // 结点颜色，如：可用 0/1 表示 黑/红，也可使用枚举型enum表示颜色
};
```

实例：一棵红黑树

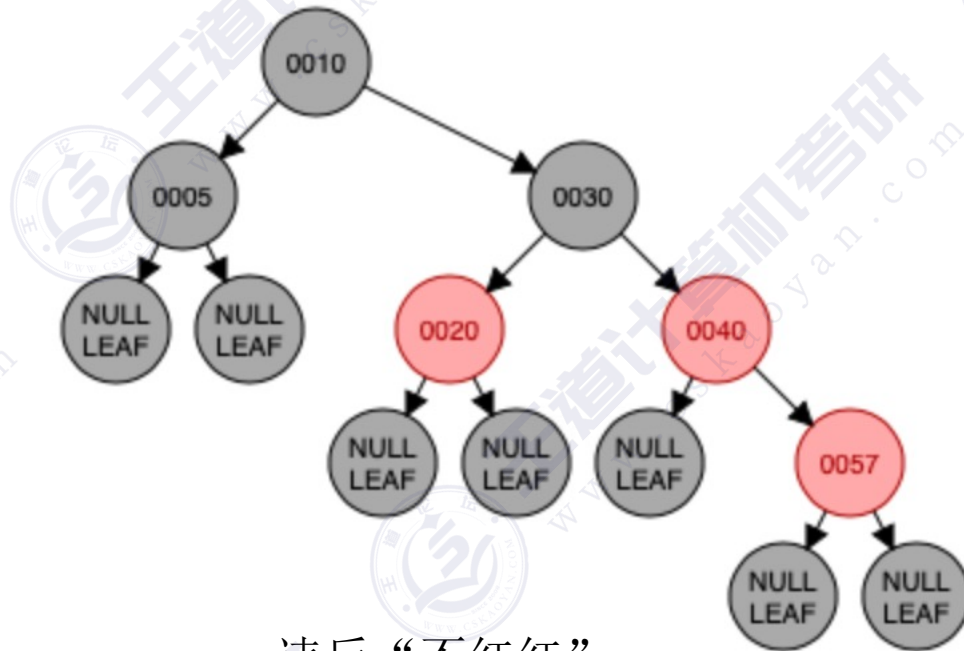


练习：是否符合红黑树要求？

左根右，根叶黑
不红红，黑路同



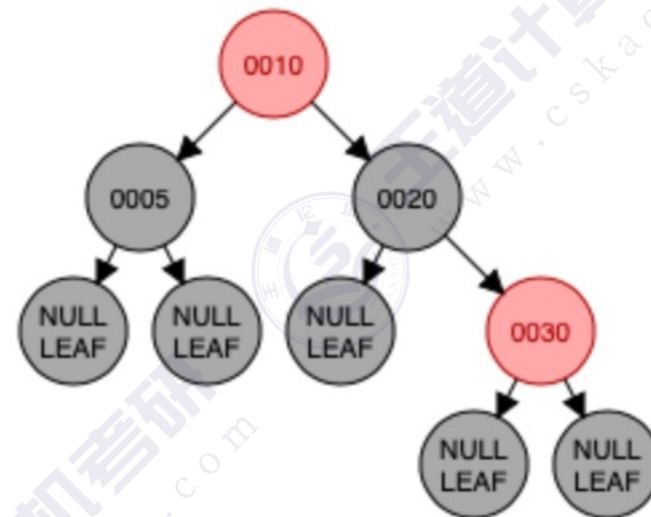
张口就是freestyle



违反“不红红”



极速否认



违反“根叶黑”



极速否认

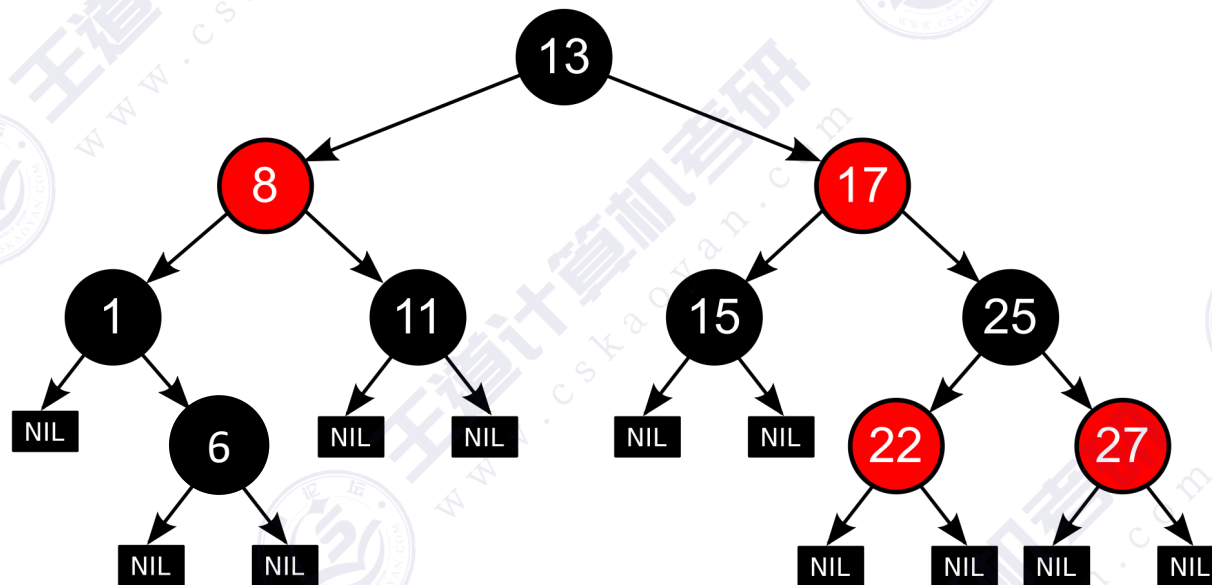
练习：是否符合红黑树要求？



左根右，根叶黑
不红红，黑路同



张口就是freestyle



违反“黑路同”

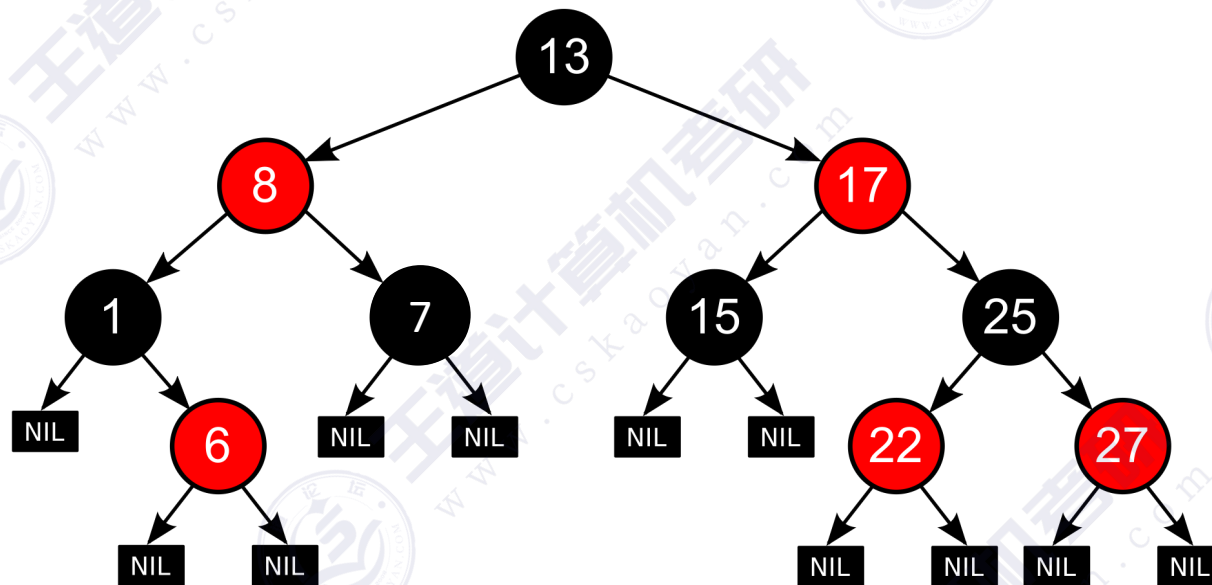
练习：是否符合红黑树要求？



左根右，根叶黑
不红红，黑路同



张口就是freestyle



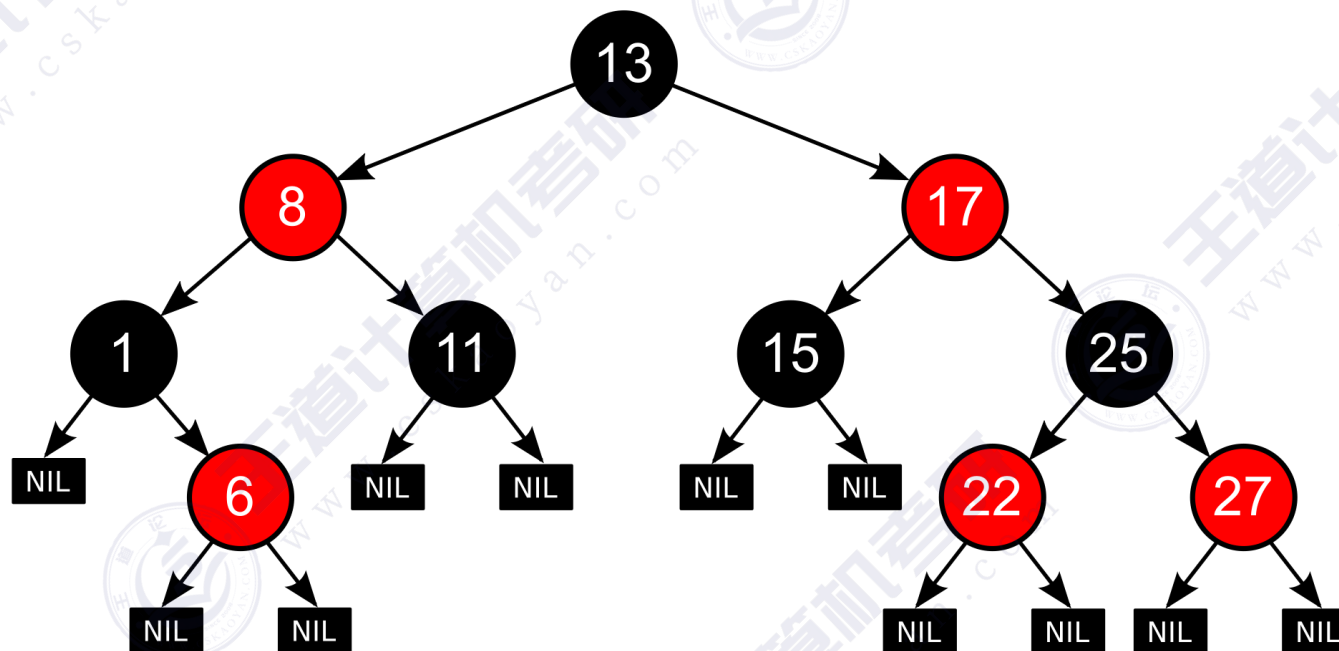
违反“左根右”

练习：是否符合红黑树要求？

左根右，根叶黑
不红红，黑路同



张口就是freestyle



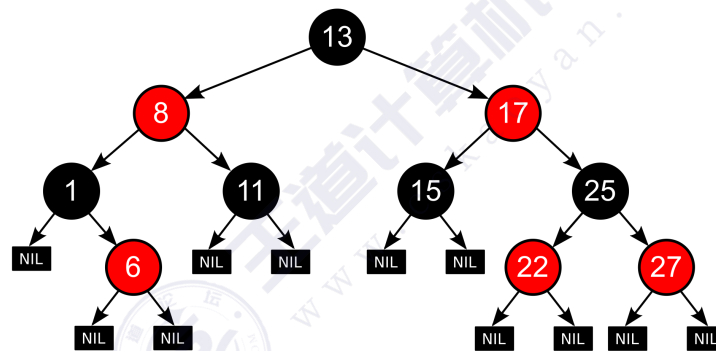
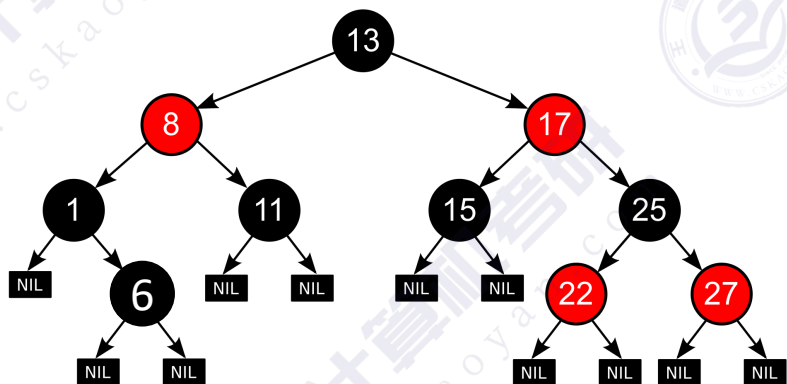
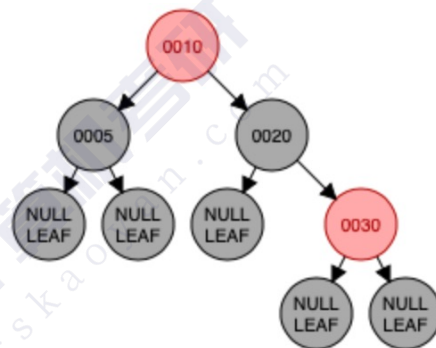
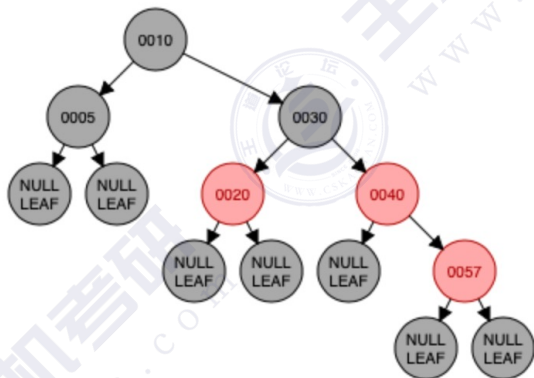
我想到了



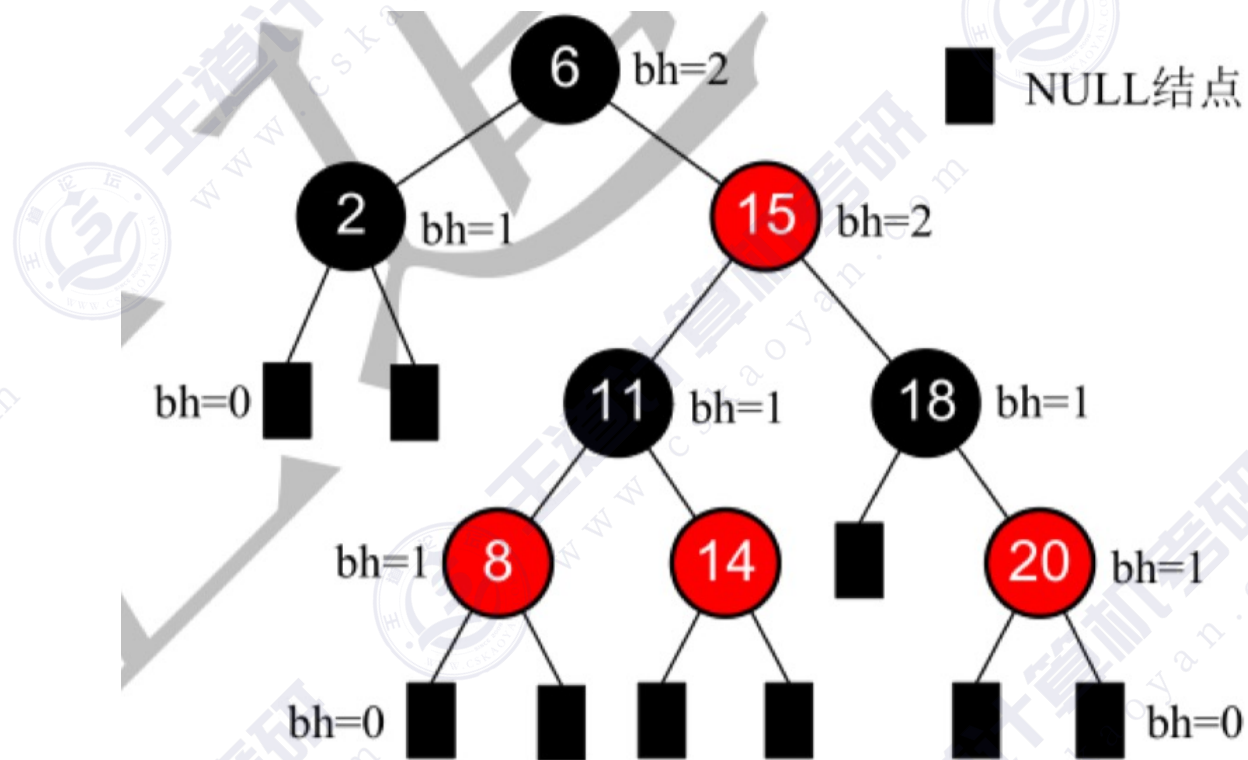
花里胡哨的肯定

一种可能的出题思路

下面这些选项中，满足“红黑树”定义的是（ ）



补充概念：结点的“黑高”



结点的**黑高** bh —— 从某结点出发（不含该结点）到达任一空叶结点的路径上黑结点总数

红黑树的定义→性质

红黑树是二叉排序树



左子树结点值 $\leq$ 根结点值 $\leq$ 右子树结点值

与普通BST相比，有什么要求



- ①每个结点或是红色，或是黑色的
- ②根节点是黑色的
- ③叶结点（外部结点、NULL结点、失败结点）均是黑色的
- ④不存在两个相邻的红结点（即红结点的父节点和孩子结点均是黑色）
- ⑤对每个结点，从该节点到任一叶结点的简单路径上，所含黑结点的数目相同

左根右，根叶黑
不红红，黑路同



张口就是freestyle

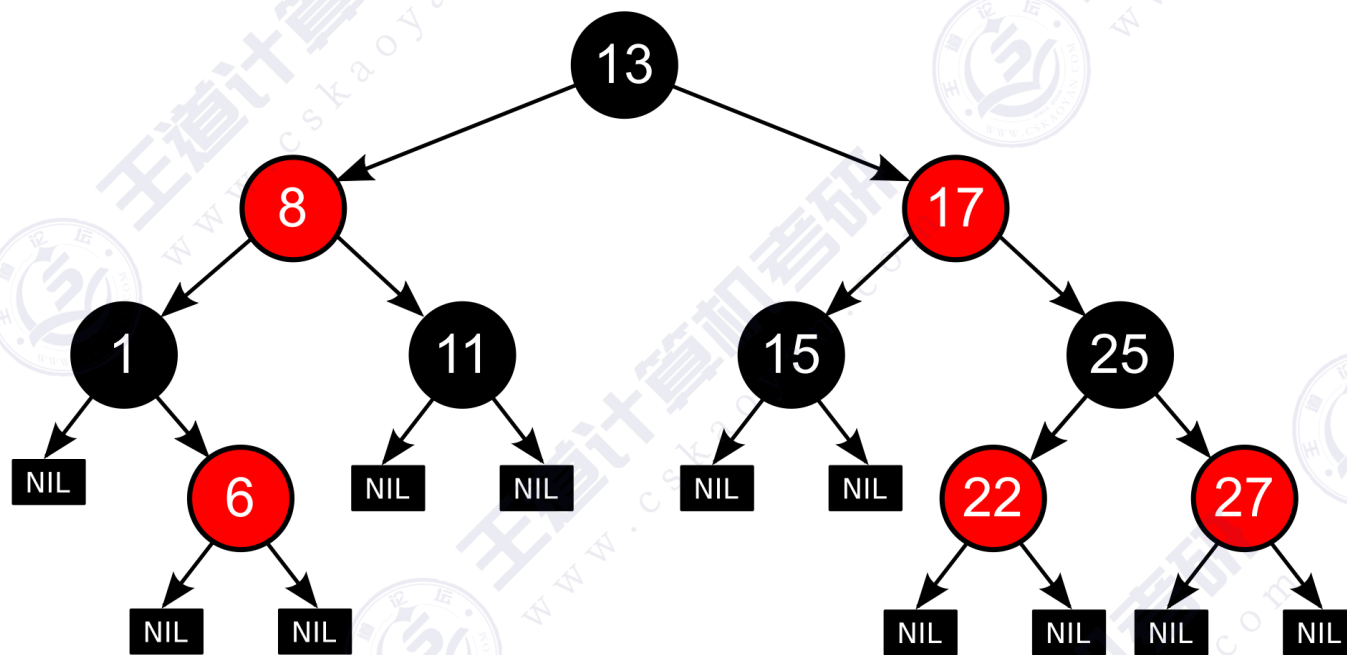


性质1：从根节点到叶结点的最长路径不大于最短路径的2倍
性质2：有n个内部节点的红黑树高度 $h \leq 2\log_2(n + 1)$

→ 红黑树查找操作时间复杂度 = $O(\log_2 n)$

查找效率与AVL
树同等数量级

红黑树的查找



与 BST、AVL 相同，从根出发，左小右大，若查找到一个空叶节点，则查找失败

复习：平均查找长度 ASL，查找成功、查找失败时分别怎么计算？

本节内容

红黑树

插入操作

红黑树的插入

从一棵空的红黑树开始，插入：20, 10, 5, 30, 40, 57, 3, 2, 4, 35, 25, 18, 22, 23, 24, 19, 18

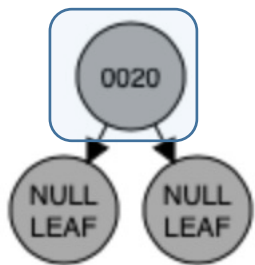
- 先查找，确定插入位置（原理同二叉排序树），插入新结点
- 新结点是根——染为黑色
- 新结点非根——染为红色
 - 若插入新结点后依然满足红黑树定义，则插入结束
 - 若插入新结点后不满足红黑树定义，需要调整，使其重新满足红黑树定义
 - 黑叔：旋转+染色
 - LL型：右单旋，父换爷+染色
 - RR型：左单旋，父换爷+染色
 - LR型：左、右双旋，儿换爷+染色
 - RL型：右、左双旋，儿换爷+染色
 - 红叔：染色+变新
 - 叔父爷染色，爷变为新结点

如何调整：看新结点叔叔的脸色

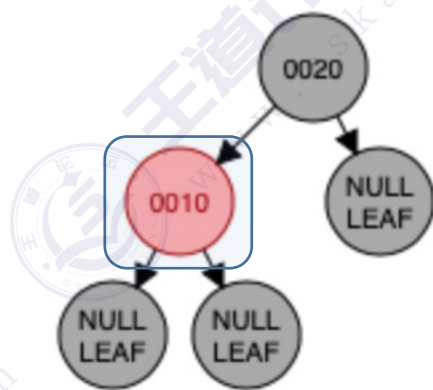


红黑树的插入

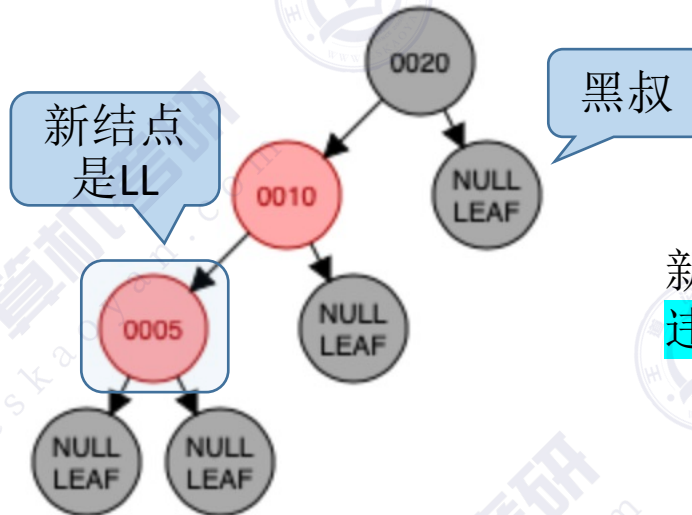
从一棵空的红黑树开始，插入：20, 10, 5, 30, 40, 57, 3, 2, 4, 35, 25, 18, 22, 23, 24, 19, 18



新结点是根，染黑

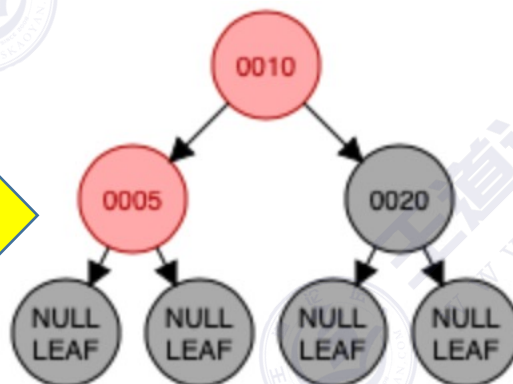


新结点非根，染红
依然满足红黑树

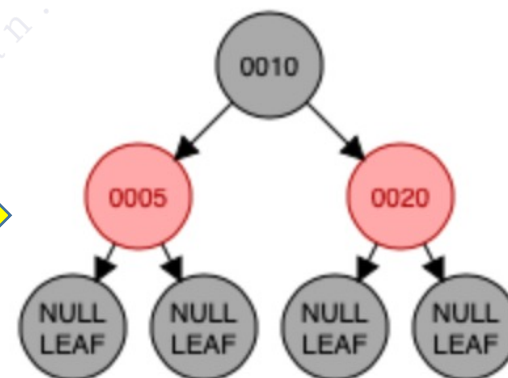


新结点非根，染红
违反“不红红”

右单旋
父换爷



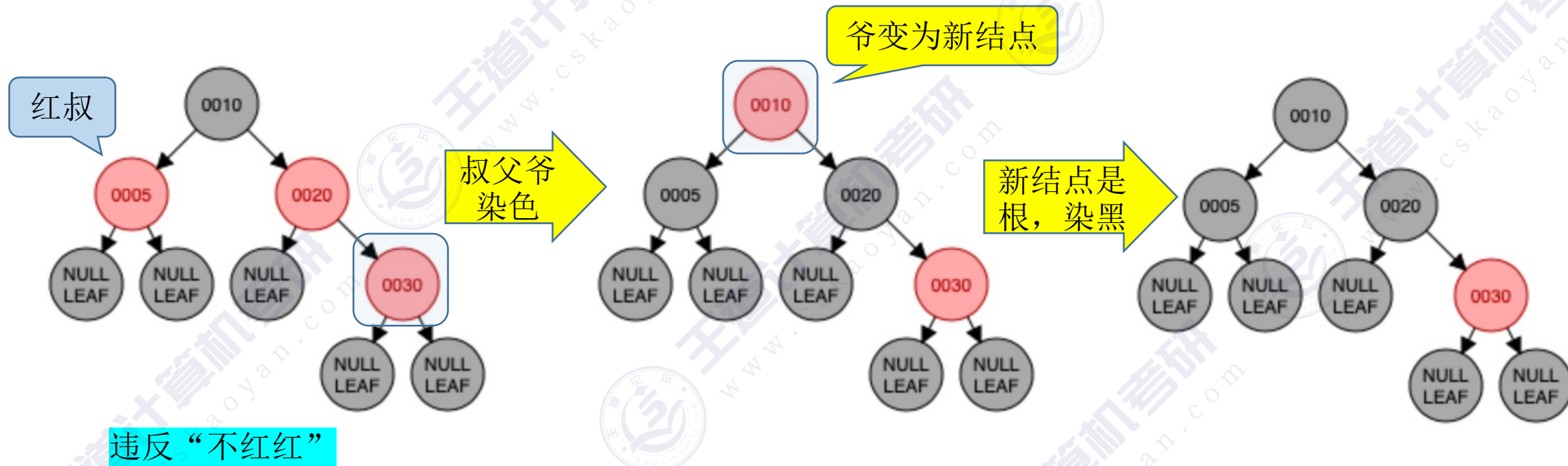
+染色



- 黑叔：旋转+染色
 - LL型：右单旋，父换爷+染色

红黑树的插入

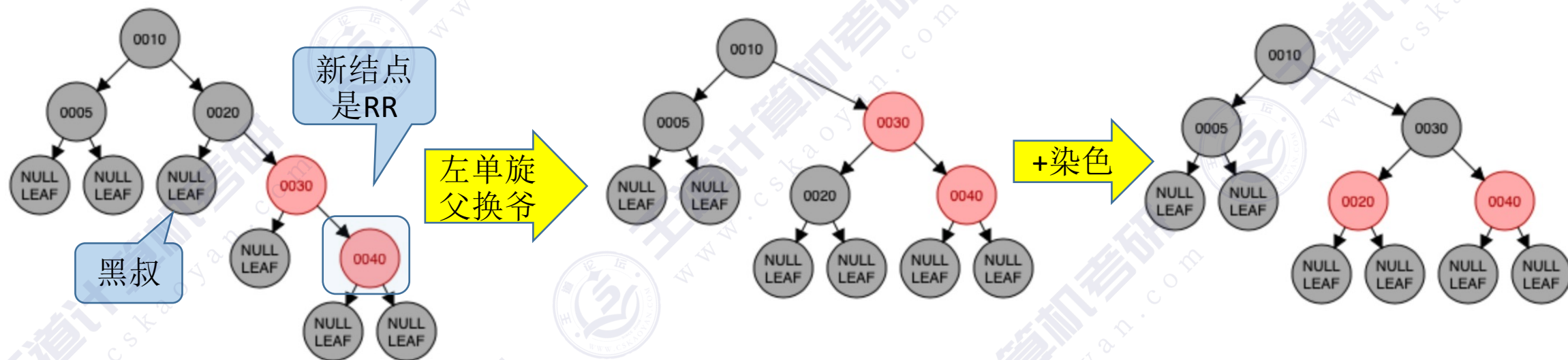
从一棵空的红黑树开始，插入：20, 10, 5, 30, 40, 57, 3, 2, 4, 35, 25, 18, 22, 23, 24, 19, 18



- 红叔：染色+变新
 - 叔父爷染色，爷变为新结点

红黑树的插入

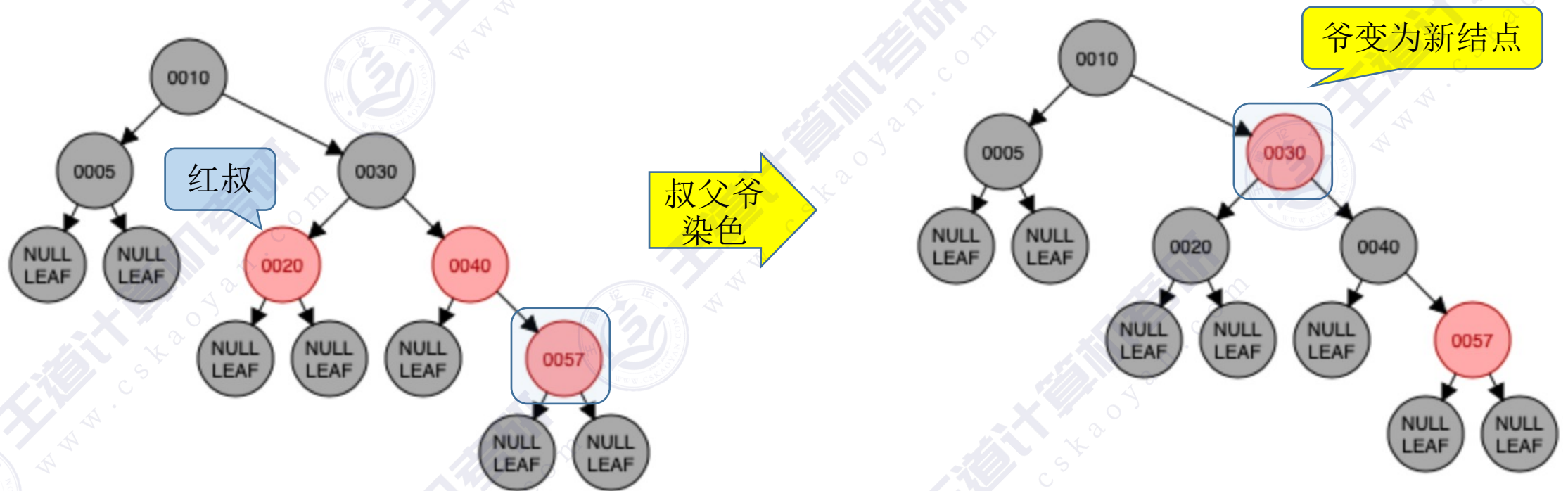
从一棵空的红黑树开始，插入：20, 10, 5, 30, 40, 57, 3, 2, 4, 35, 25, 18, 22, 23, 24, 19, 18



- 黑叔：旋转+染色
 - RR型：左单旋，父换爷+染色

红黑树的插入

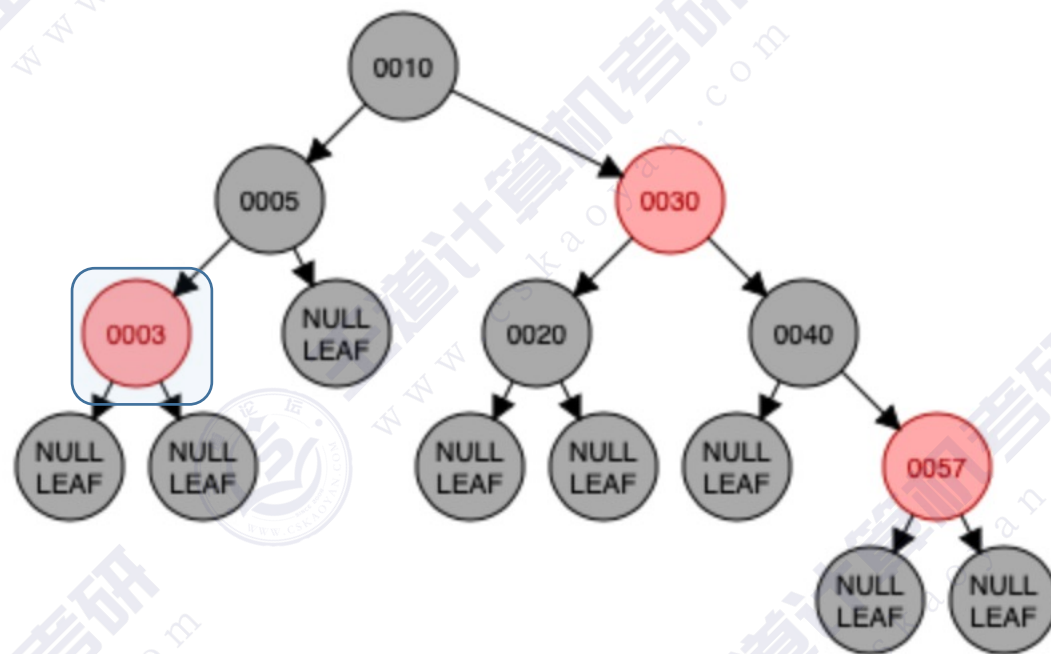
从一棵空的红黑树开始，插入：20, 10, 5, 30, 40, 57, 3, 2, 4, 35, 25, 18, 22, 23, 24, 19, 18



- 红叔: 染色+变新
 - 叔父爷染色, 爷变为新结点

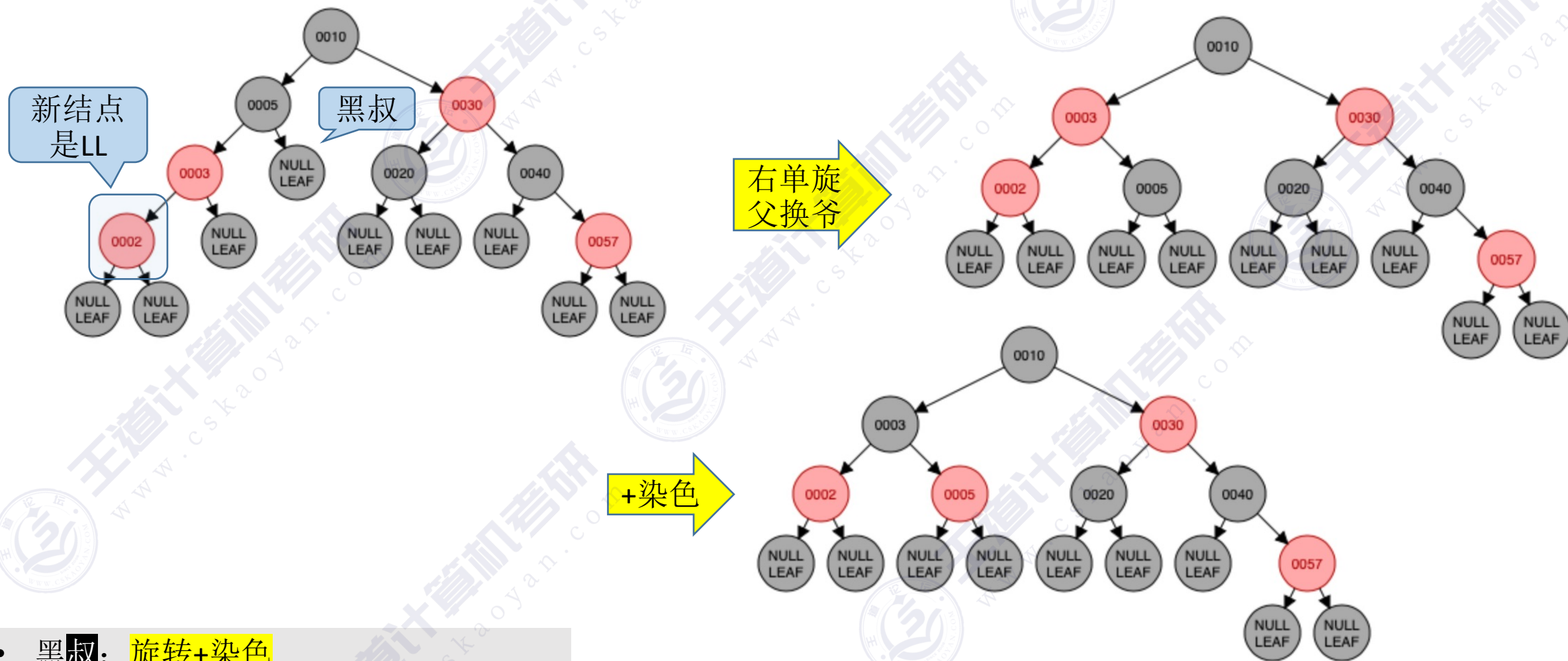
红黑树的插入

从一棵空的红黑树开始，插入：20, 10, 5, 30, 40, 57, 3, 2, 4, 35, 25, 18, 22, 23, 24, 19, 18



红黑树的插入

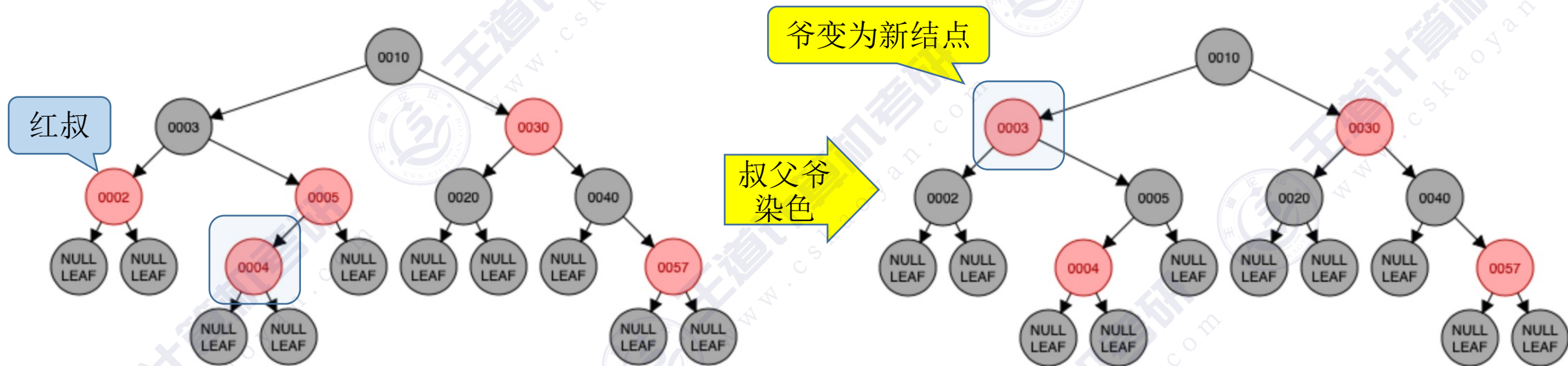
从一棵空的红黑树开始，插入：20, 10, 5, 30, 40, 57, 3, 2, 4, 35, 25, 18, 22, 23, 24, 19, 18



- 黑叔：旋转+染色
 - LL型：右单旋，父换爷+染色

红黑树的插入

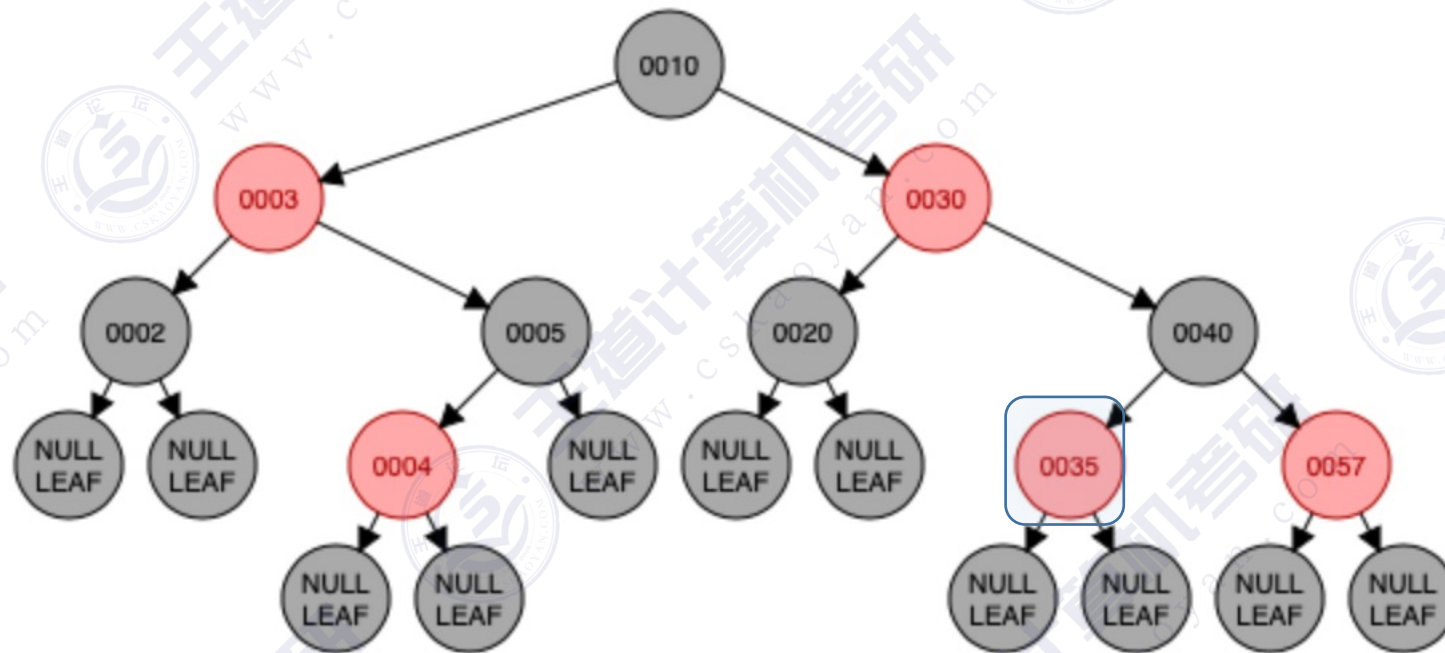
从一棵空的红黑树开始，插入：20, 10, 5, 30, 40, 57, 3, 2, 4, 35, 25, 18, 22, 23, 24, 19, 18



- 红叔: 染色+变新
 - 叔父爷染色, 爷变为新结点

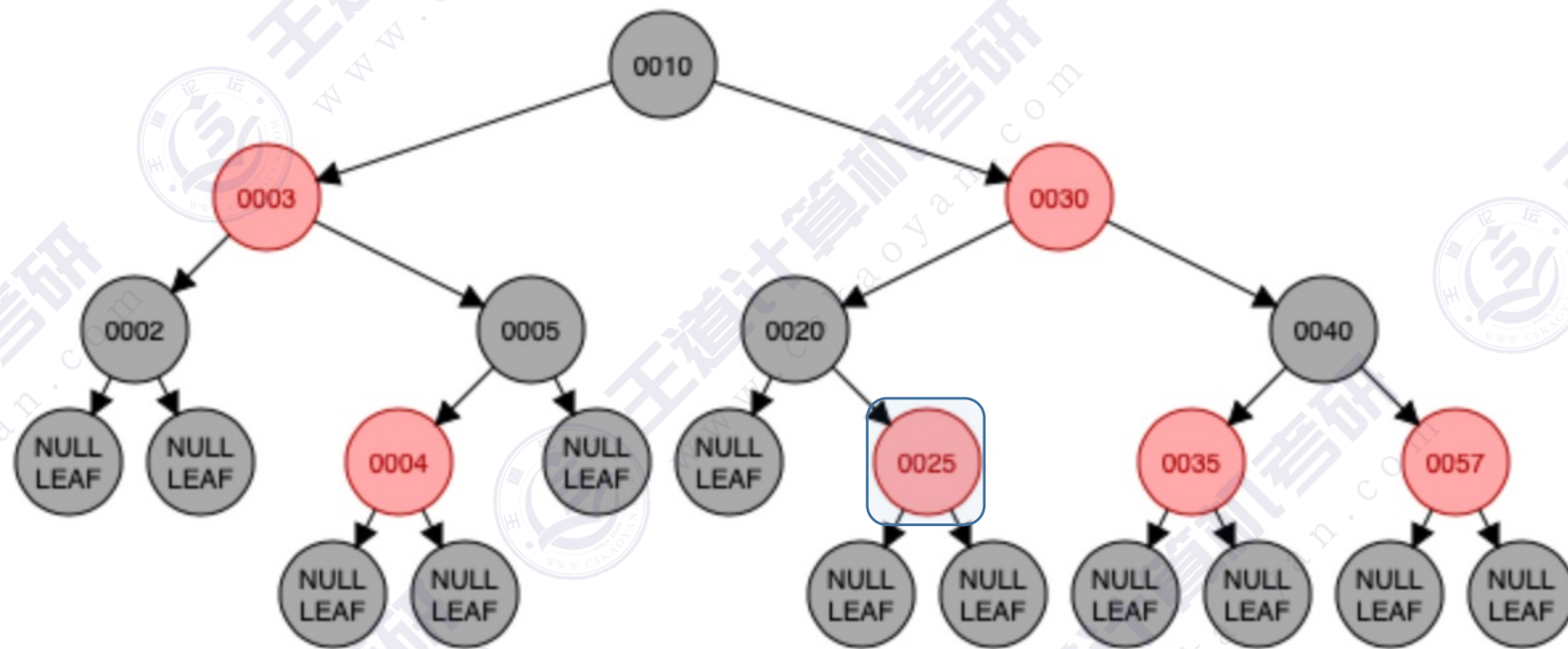
红黑树的插入

从一棵空的红黑树开始，插入：20, 10, 5, 30, 40, 57, 3, 2, 4, 35, 25, 18, 22, 23, 24, 19, 18



红黑树的插入

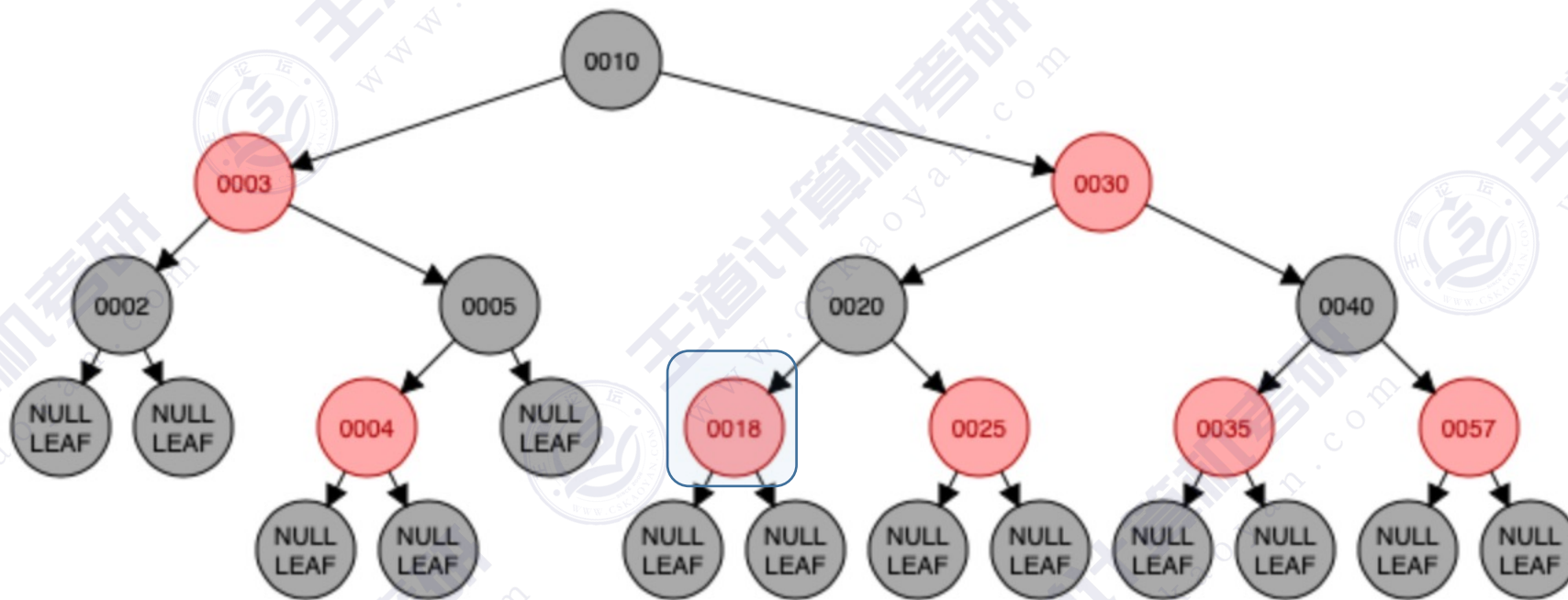
从一棵空的红黑树开始，插入：20, 10, 5, 30, 40, 57, 3, 2, 4, 35, 25, 18, 22, 23, 24, 19, 18



花里胡哨的肯定

红黑树的插入

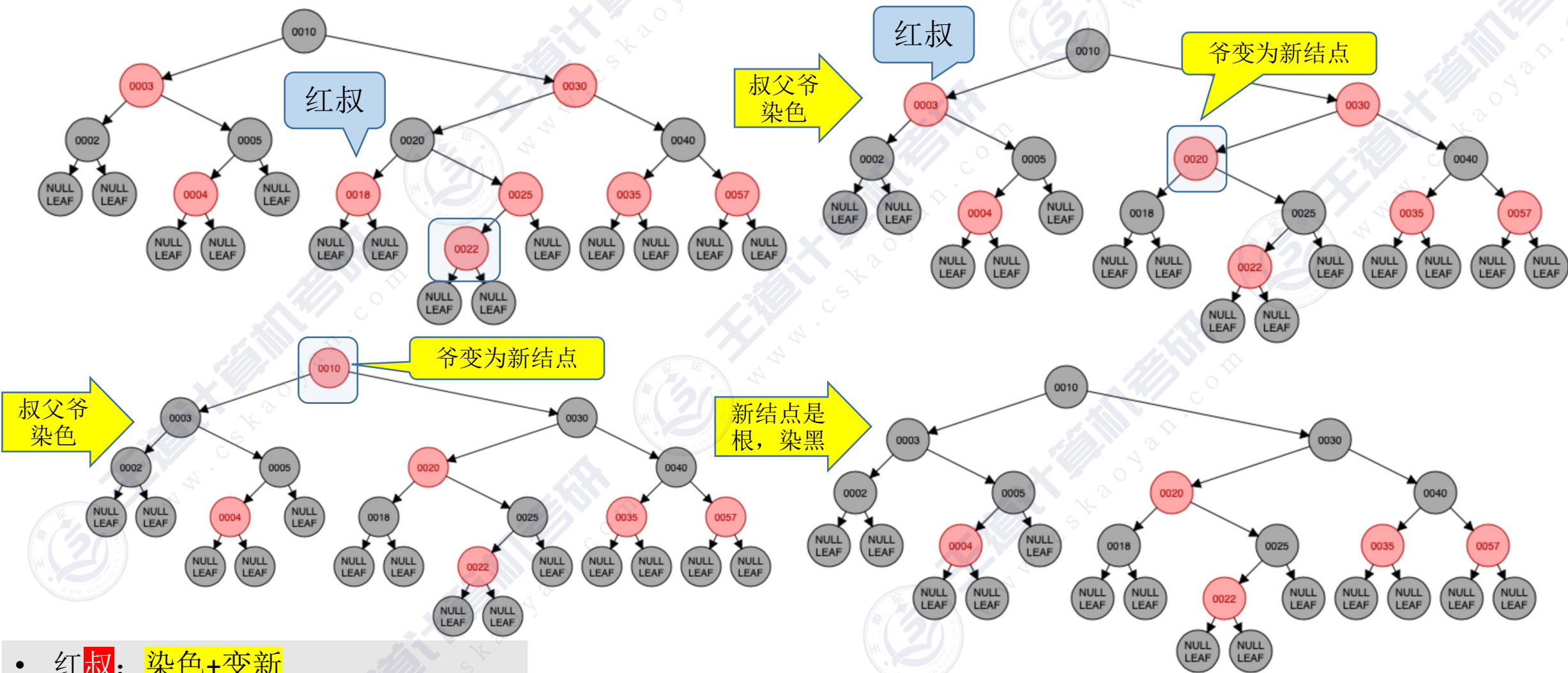
从一棵空的红黑树开始，插入：20, 10, 5, 30, 40, 57, 3, 2, 4, 35, 25, 18, 22, 23, 24, 19, 18



花里胡哨的肯定

红黑树的插入

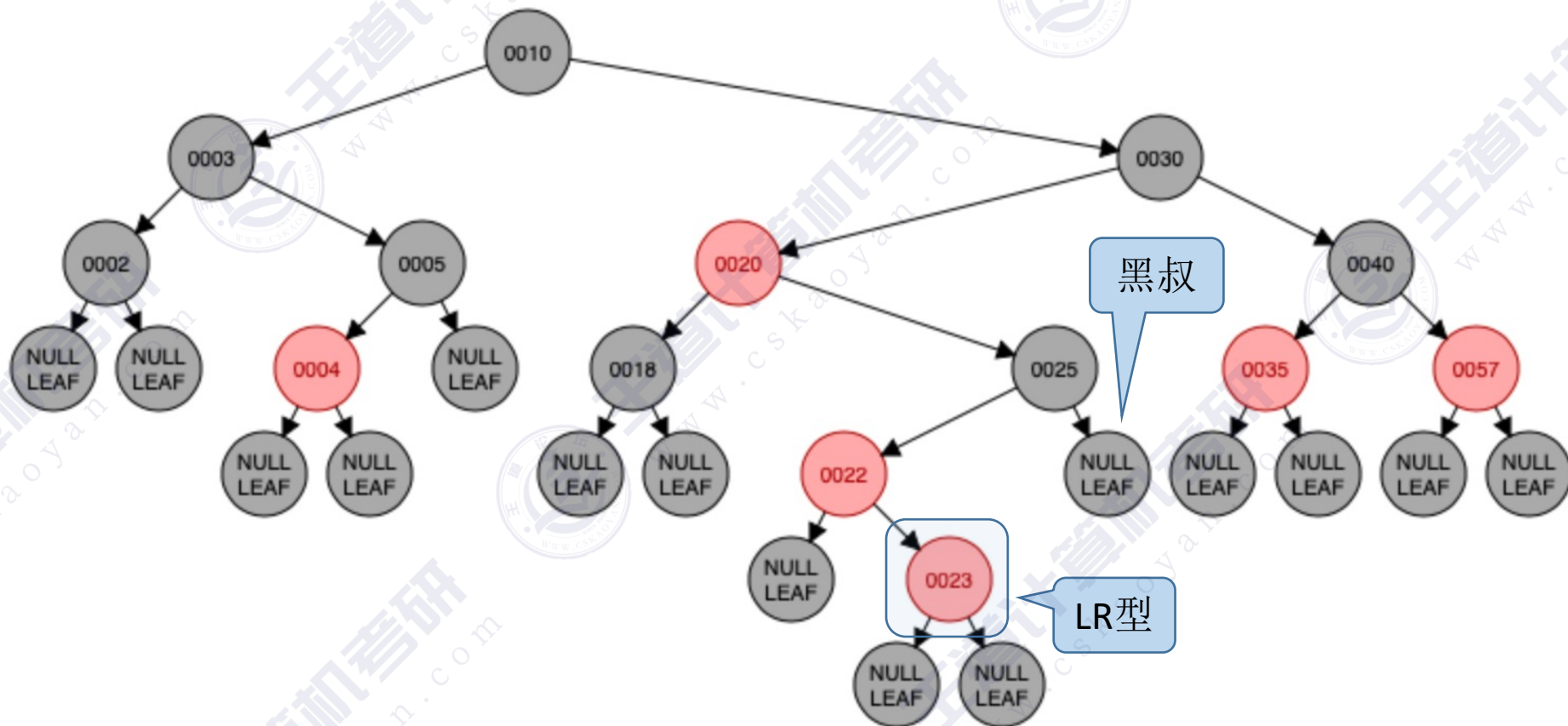
从一棵空的红黑树开始，插入：20, 10, 5, 30, 40, 57, 3, 2, 4, 35, 25, 18, 22, 23, 24, 19, 18



- 红叔：染色+变新
 - 叔父爷染色，爷变为新结点

红黑树的插入

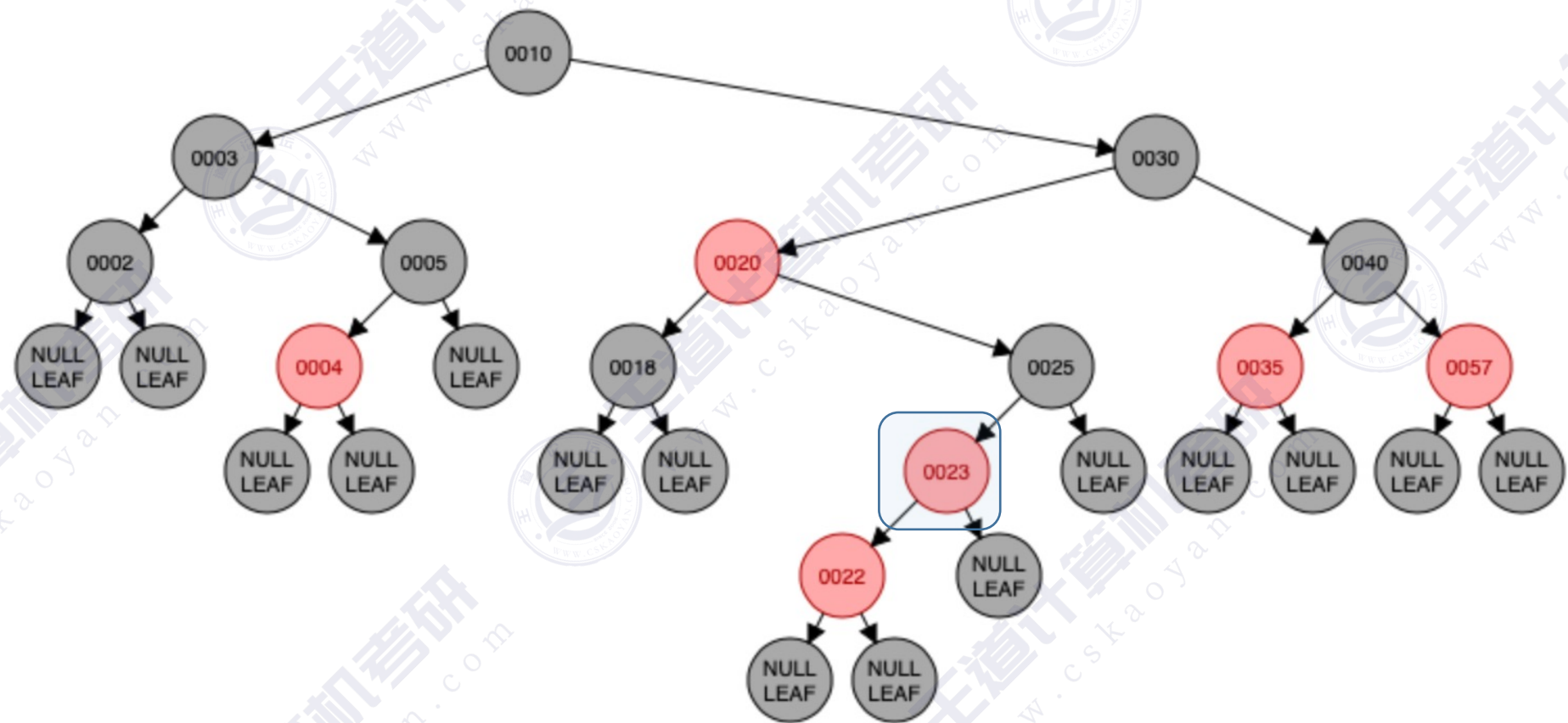
从一棵空的红黑树开始，插入：20, 10, 5, 30, 40, 57, 3, 2, 4, 35, 25, 18, 22, 23, 24, 19, 18



- 黑叔：旋转+染色
 - LR型：左、右双旋，儿换爷+染色

红黑树的插入

从一棵空的红黑树开始，插入：20, 10, 5, 30, 40, 57, 3, 2, 4, 35, 25, 18, 22, 23, 24, 19, 18

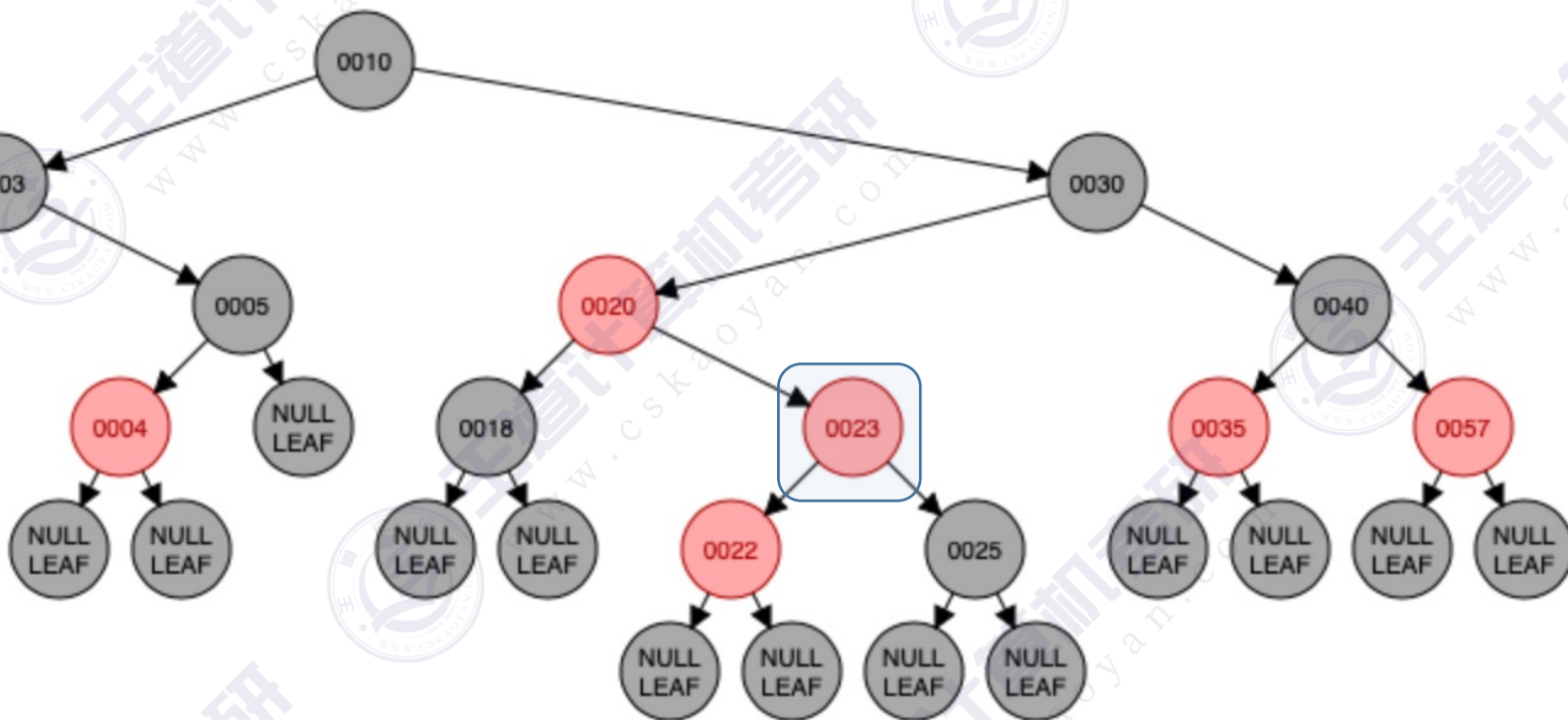


①左单旋的结果：儿子辈分升高

- 黑叔：旋转+染色
 - LR型：左、右双旋，儿换爷+染色

红黑树的插入

从一棵空的红黑树开始，插入：20, 10, 5, 30, 40, 57, 3, 2, 4, 35, 25, 18, 22, 23, 24, 19, 18

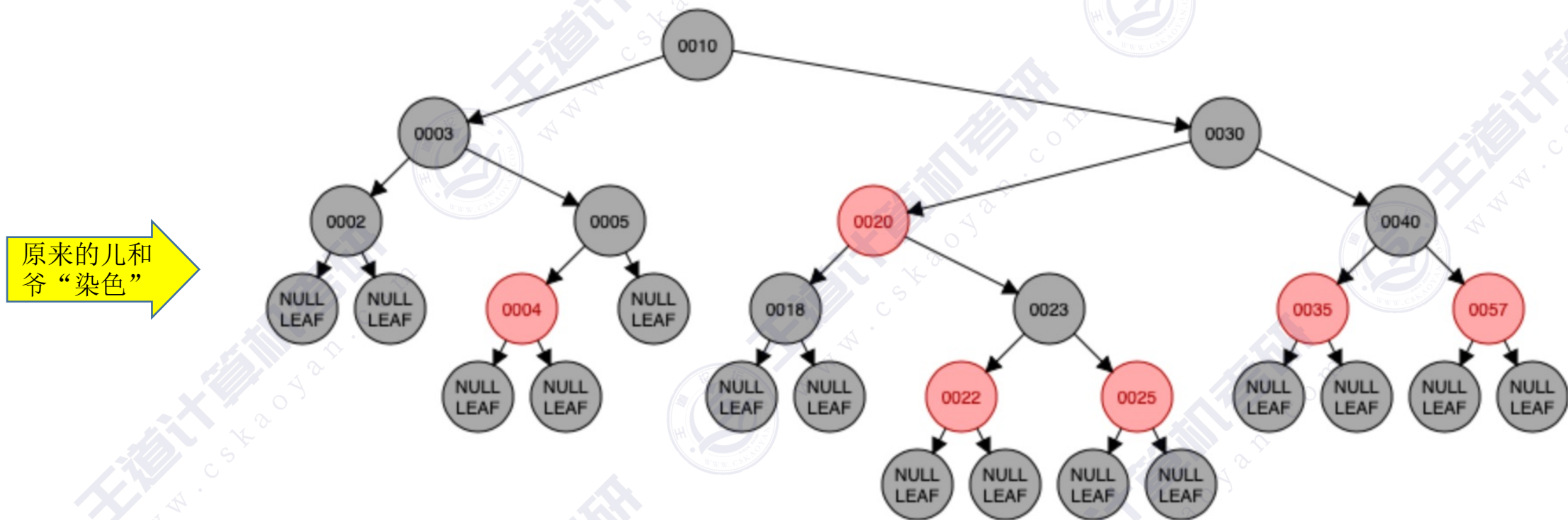


②右单旋的结果：儿子辈分再升高

- 黑叔：旋转+染色
 - LR型：左、右双旋，儿换爷+染色

红黑树的插入

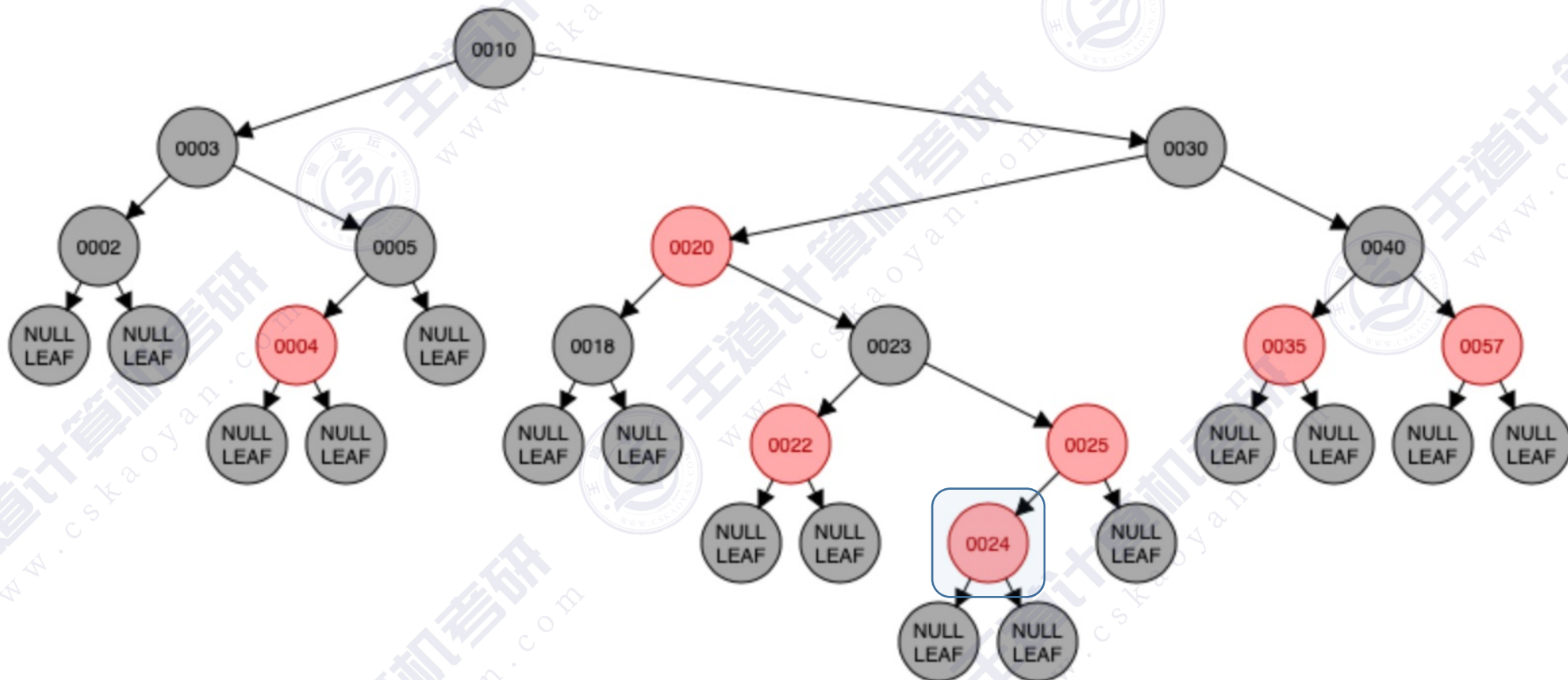
从一棵空的红黑树开始，插入：20, 10, 5, 30, 40, 57, 3, 2, 4, 35, 25, 18, 22, 23, 24, 19, 18



- 黑叔：旋转+染色
 - LR型：左、右双旋，儿换爷+染色

红黑树的插入

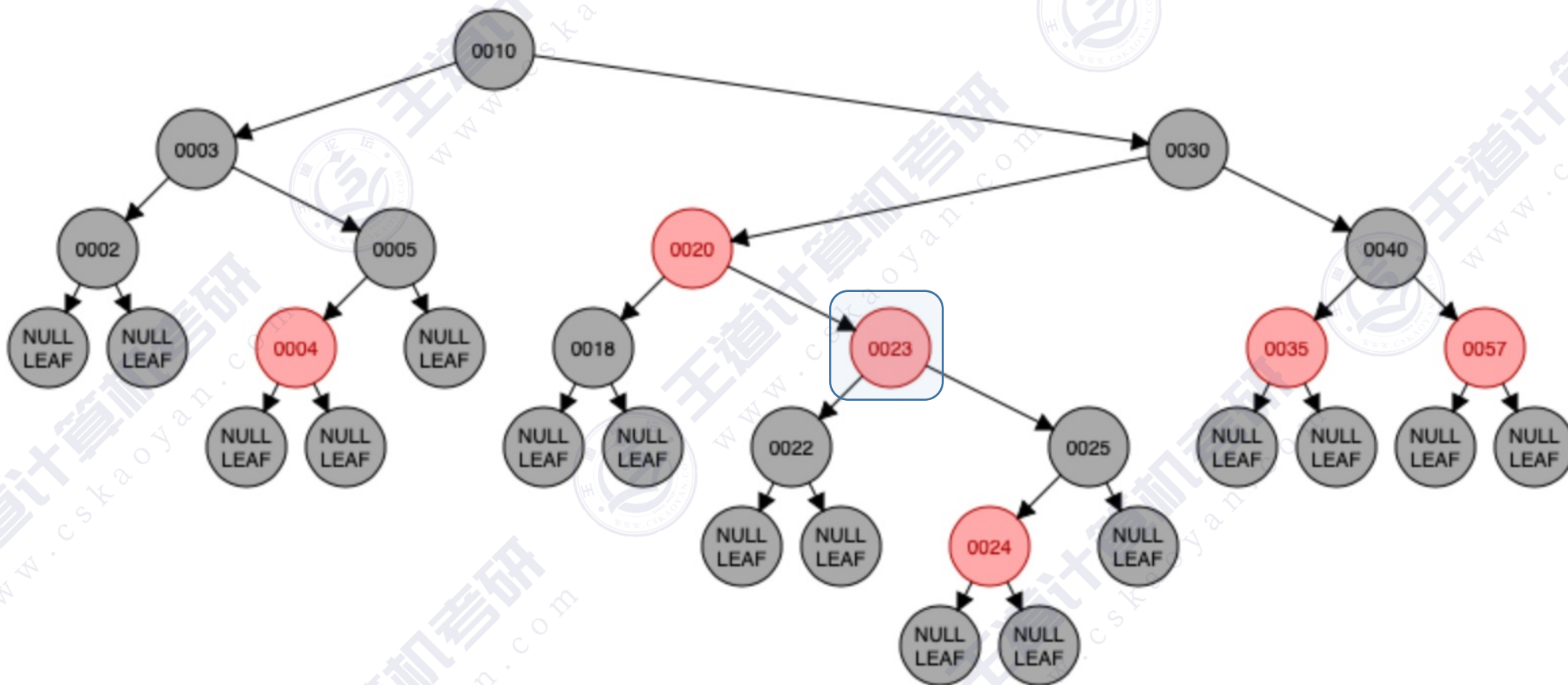
从一棵空的红黑树开始，插入：20, 10, 5, 30, 40, 57, 3, 2, 4, 35, 25, 18, 22, 23, 24, 19, 18



- 红叔：染色+变新
 - 叔父爷染色，爷变为新结点

红黑树的插入

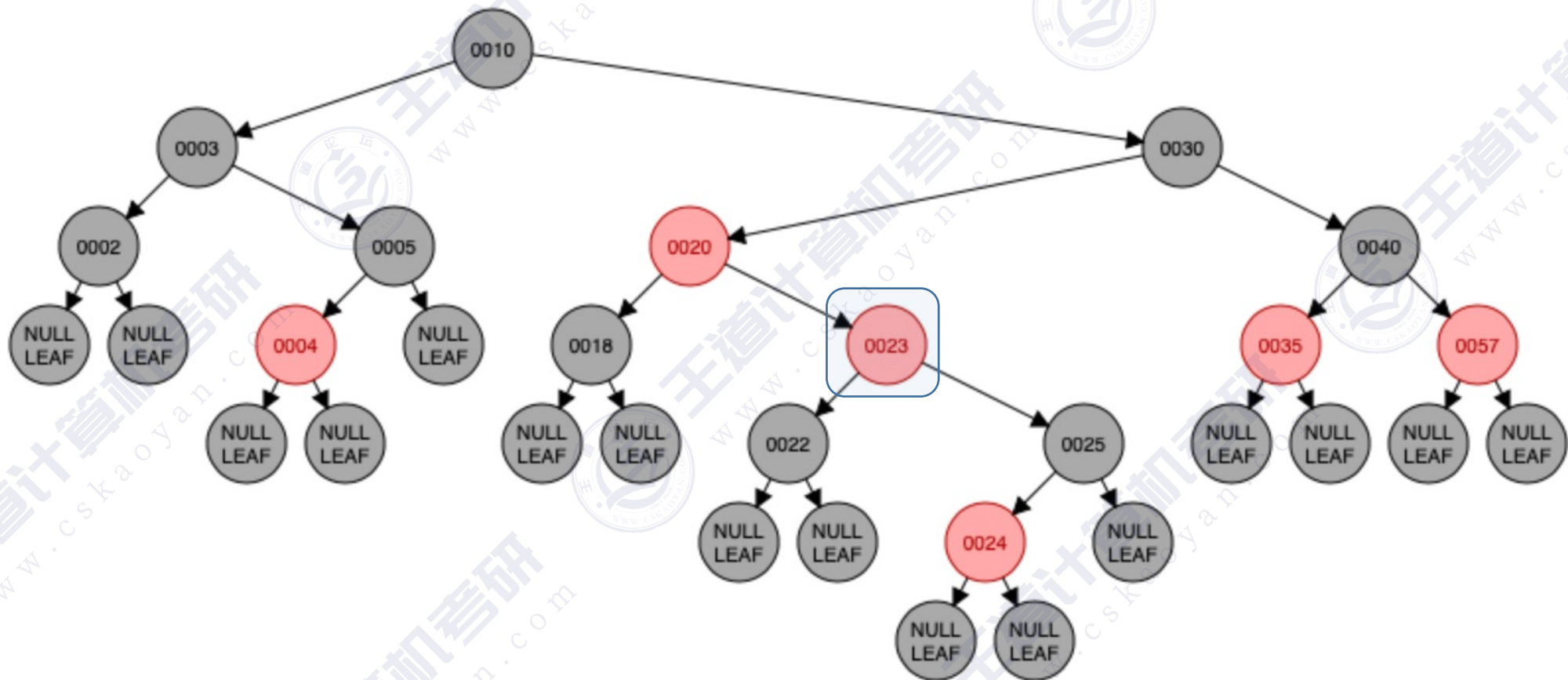
从一棵空的红黑树开始，插入：20, 10, 5, 30, 40, 57, 3, 2, 4, 35, 25, 18, 22, 23, 24, 19, 18



- 红叔：染色+变新
 - 叔父爷染色，爷变为新结点

红黑树的插入

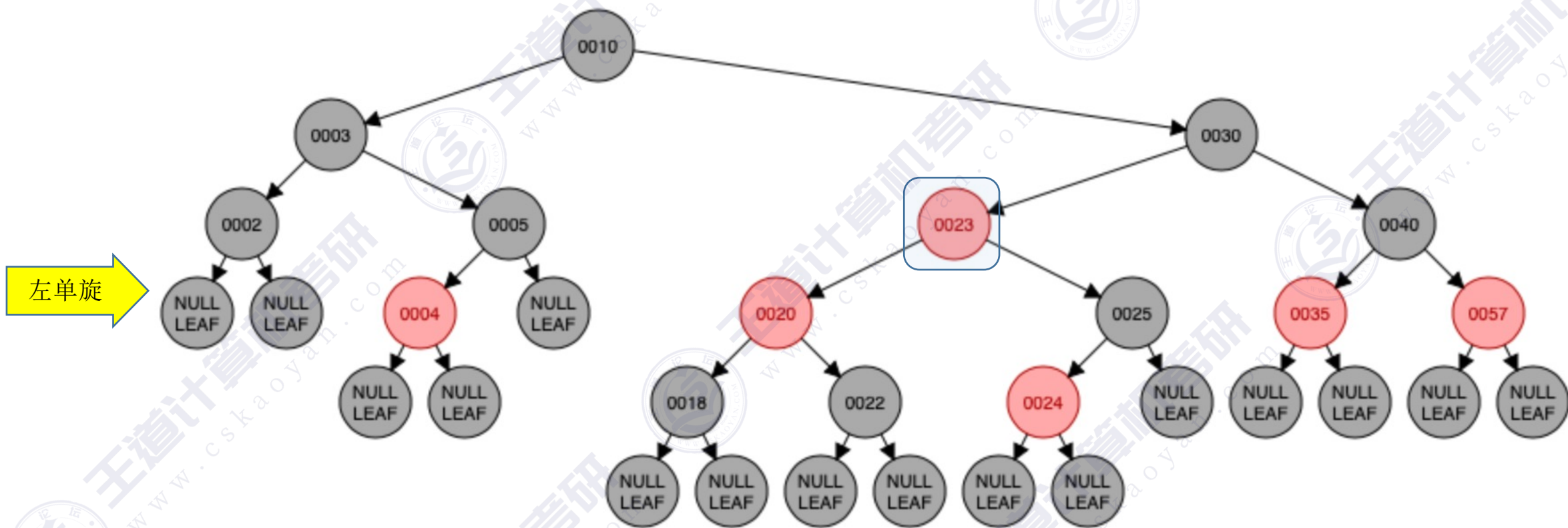
从一棵空的红黑树开始，插入：20, 10, 5, 30, 40, 57, 3, 2, 4, 35, 25, 18, 22, 23, 24, 19, 18



- 黑叔：旋转+染色
 - LR型：左、右双旋，儿换爷+染色

红黑树的插入

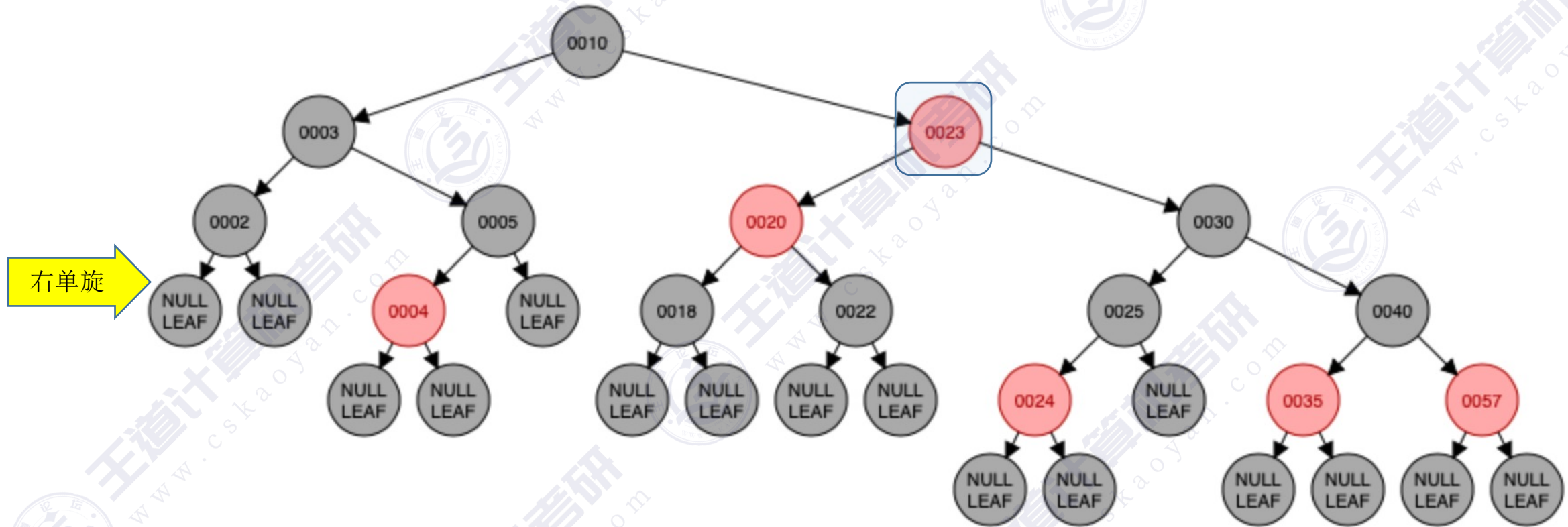
从一棵空的红黑树开始，插入：20, 10, 5, 30, 40, 57, 3, 2, 4, 35, 25, 18, 22, 23, 24, 19, 18



- 黑叔：旋转+染色
 - LR型：左、右双旋，儿换爷+染色

红黑树的插入

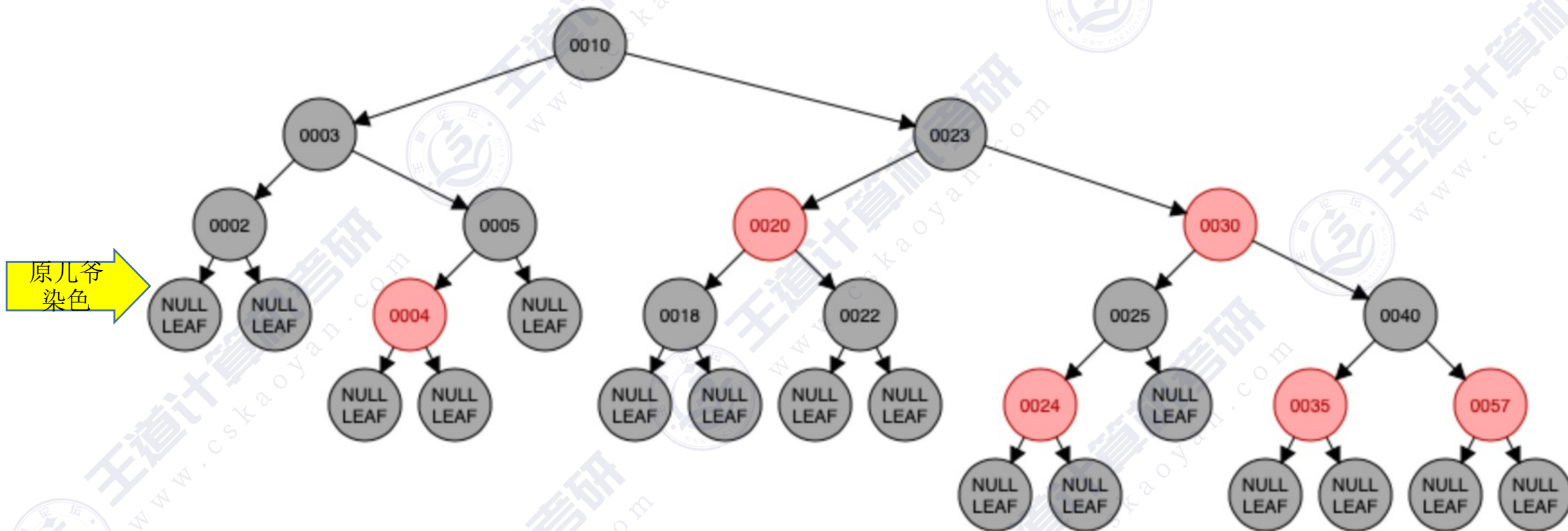
从一棵空的红黑树开始，插入：20, 10, 5, 30, 40, 57, 3, 2, 4, 35, 25, 18, 22, 23, 24, 19, 18



- 黑叔：旋转+染色
 - LR型：左、右双旋，儿换爷+染色

红黑树的插入

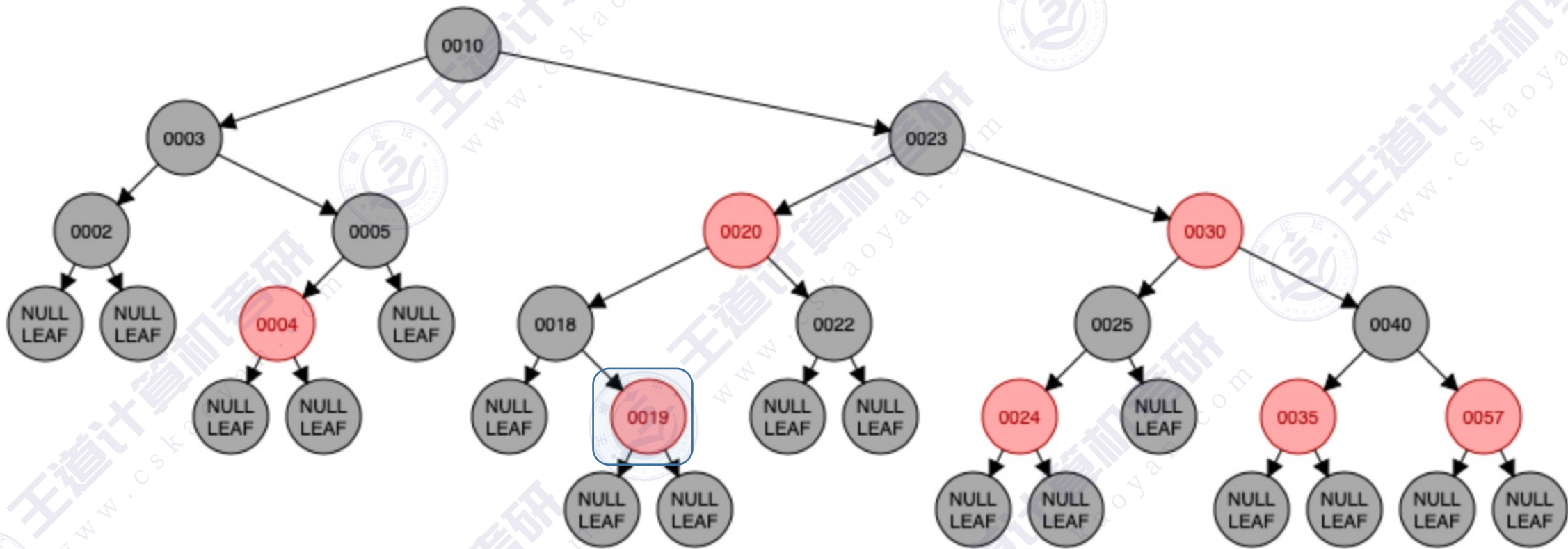
从一棵空的红黑树开始，插入：20, 10, 5, 30, 40, 57, 3, 2, 4, 35, 25, 18, 22, 23, **24**, 19, 18



- 黑叔：旋转+染色
 - LR型：左、右双旋，儿换爷+染色

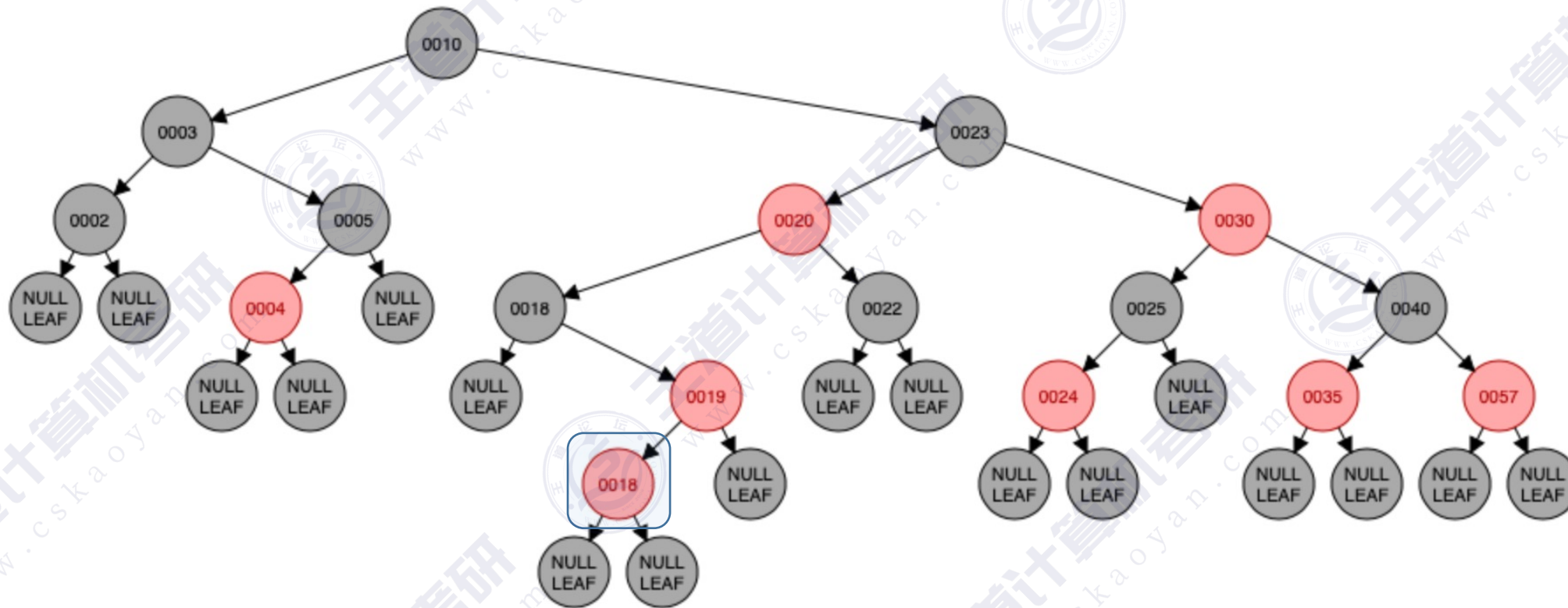
红黑树的插入

从一棵空的红黑树开始，插入：20, 10, 5, 30, 40, 57, 3, 2, 4, 35, 25, 18, 22, 23, 24, 19, 18



红黑树的插入

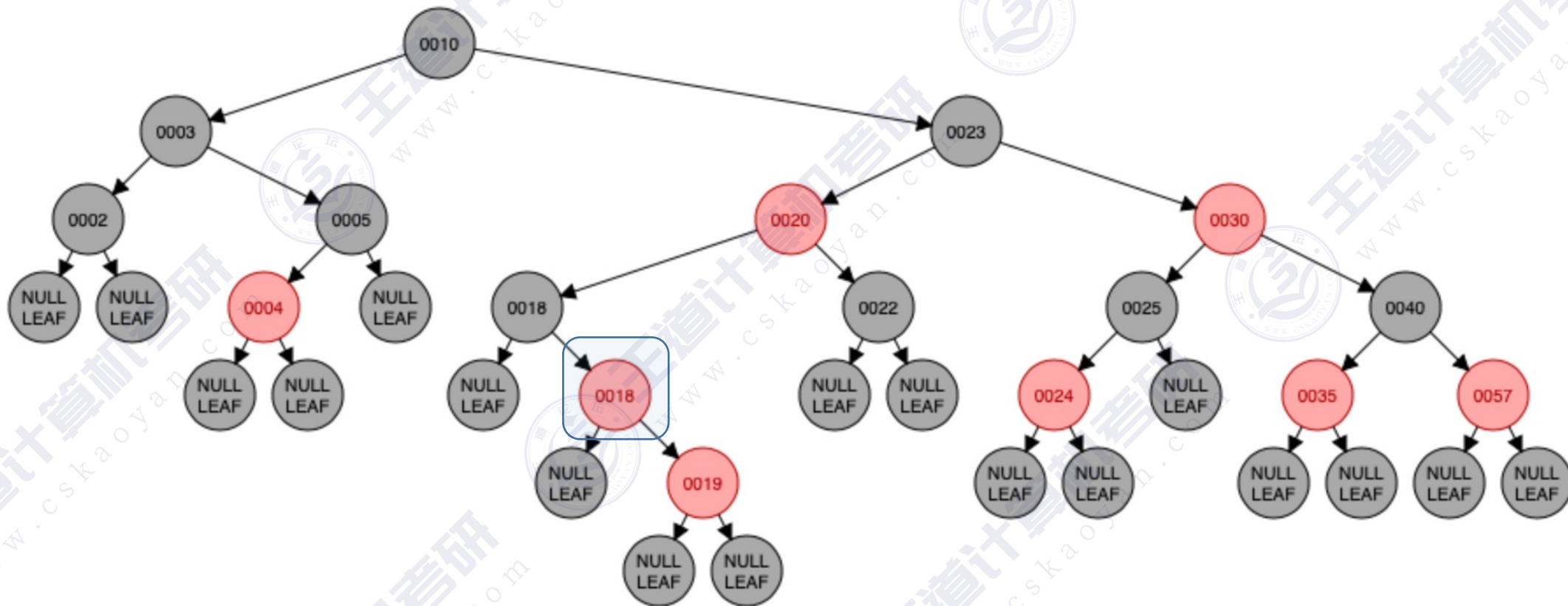
从一棵空的红黑树开始，插入：20, 10, 5, 30, 40, 57, 3, 2, 4, 35, 25, 18, 22, 23, 24, 19, 18



- 黑叔：旋转+染色
 - RL型：右、左双旋，儿换爷+染色

红黑树的插入

从一棵空的红黑树开始，插入：20, 10, 5, 30, 40, 57, 3, 2, 4, 35, 25, 18, 22, 23, 24, 19, 18



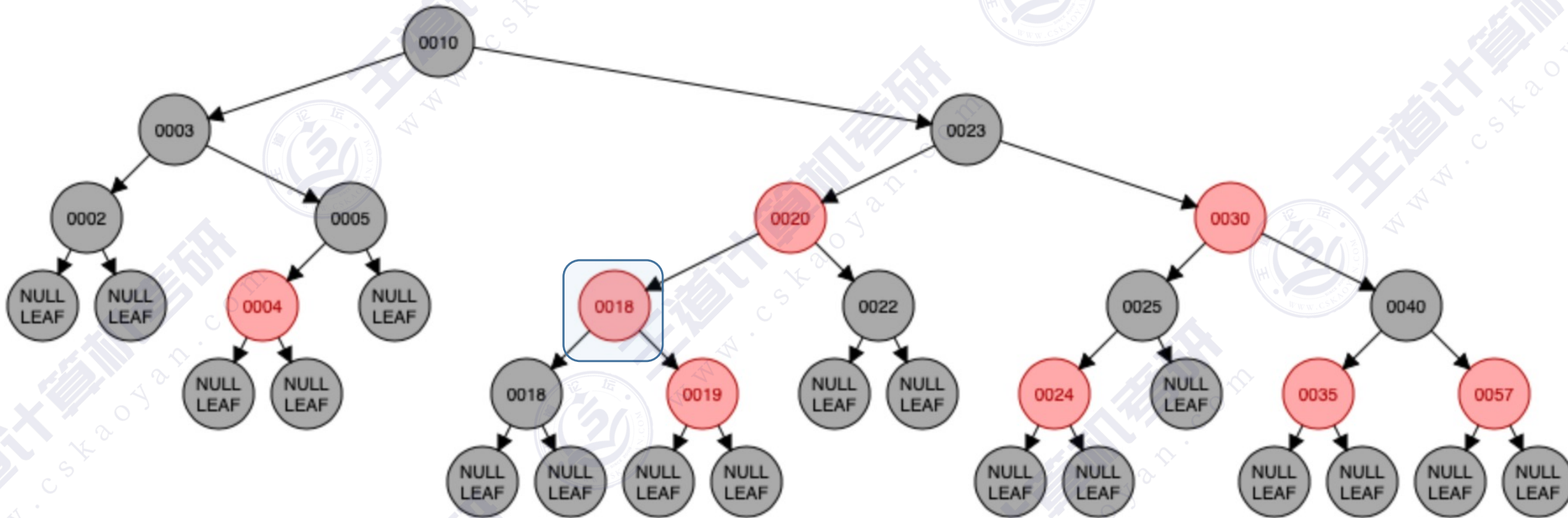
右单旋

- 黑叔：旋转+染色
 - RL型：右、左双旋，儿换爷+染色

红黑树的插入

从一棵空的红黑树开始，插入：20, 10, 5, 30, 40, 57, 3, 2, 4, 35, 25, 18, 22, 23, 24, 19, 18

左单旋

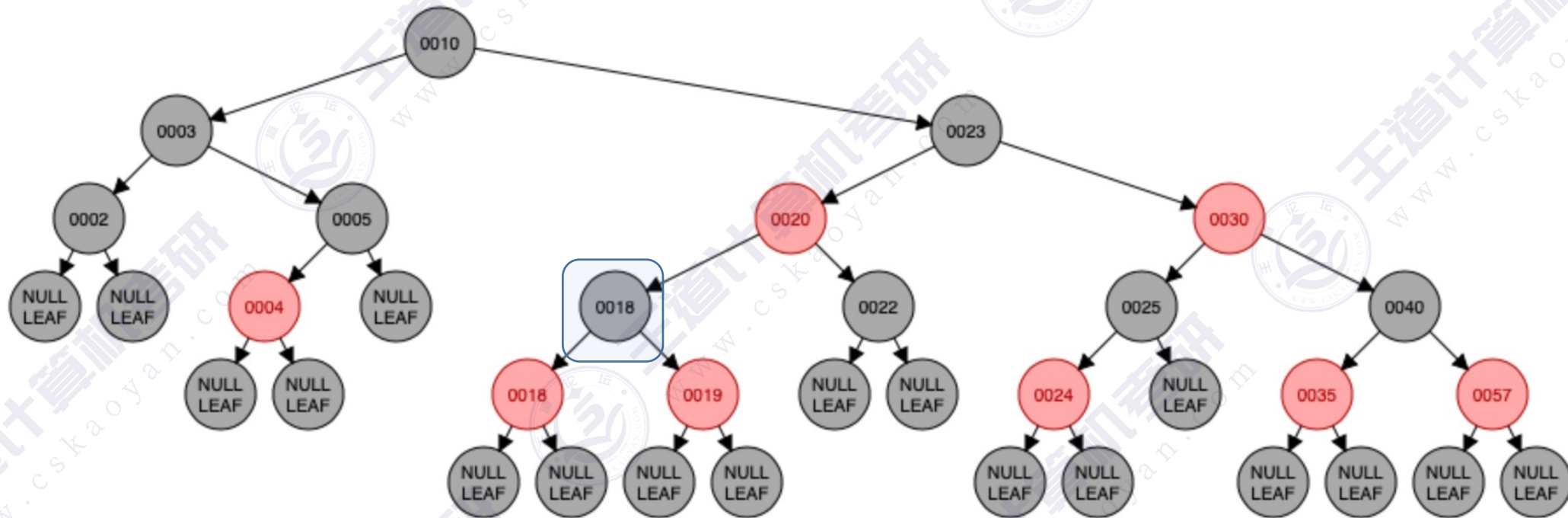


- 黑叔：旋转+染色
 - RL型：右、左双旋，儿换爷+染色

红黑树的插入

从一棵空的红黑树开始，插入：20, 10, 5, 30, 40, 57, 3, 2, 4, 35, 25, 18, 22, 23, 24, 19, 18

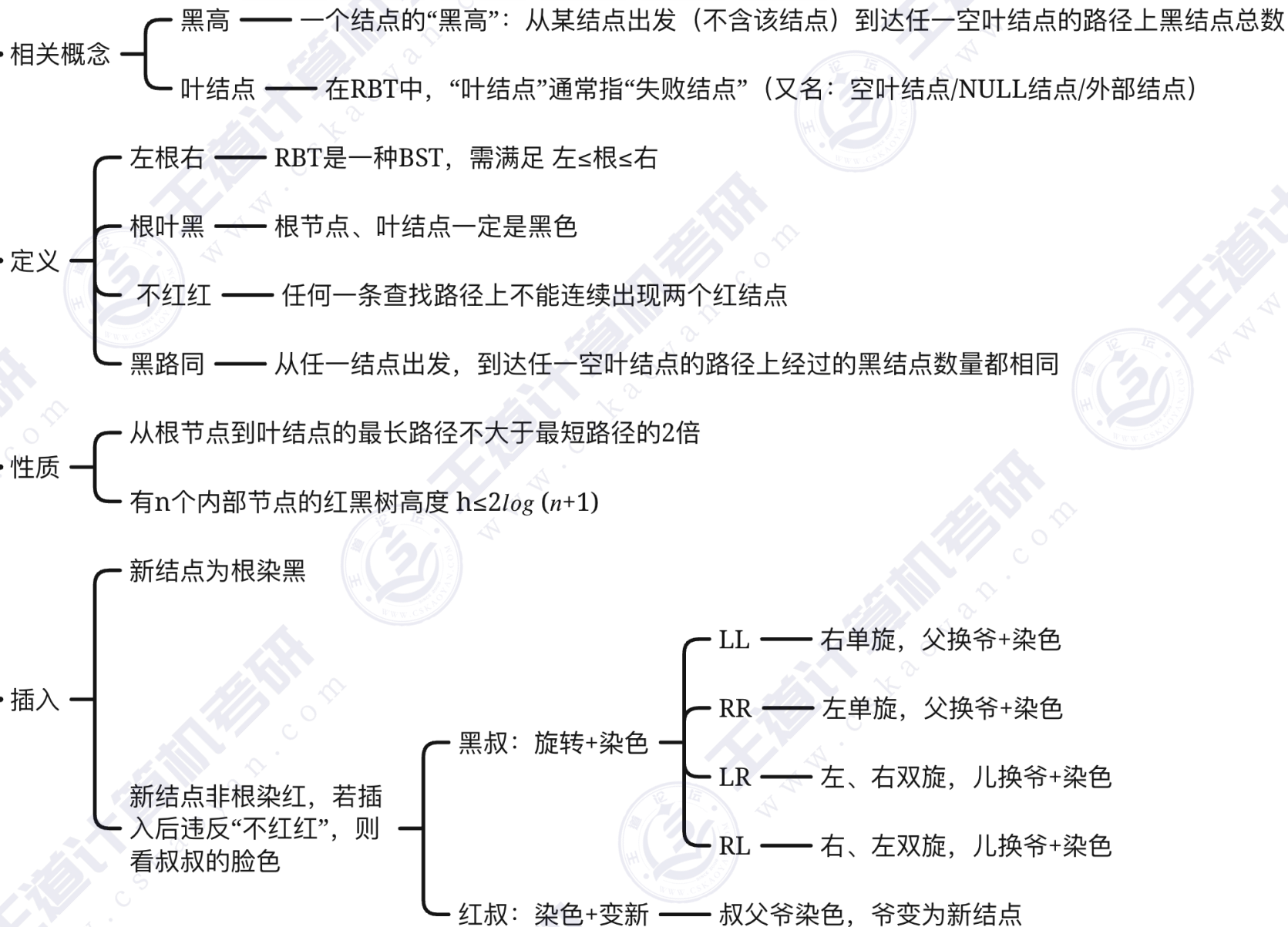
原儿爷
染色



- 黑叔：旋转+染色
 - RL型：右、左双旋，儿换爷+染色

知识回顾与重要考点

红黑树RBT



红黑树练习方法（插入操作）

课件中的例子，插入：20, 10, 5, 30, 40, 57, 3, 2, 4, 35, 25, 18, 22, 23, 24, 19, 18

Red/Black Tree

①打上勾，显示空叶结点

③插入关键字

②选择 pause，使用 Step Forward/Step Back 进行单步演示

Animation Completed

Skip Back Step Back pause Step Forward Skip Forward

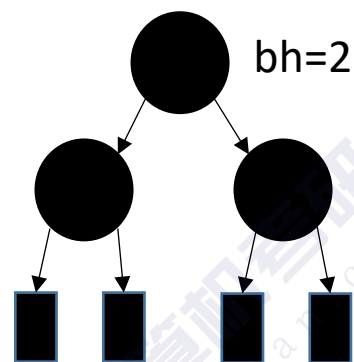
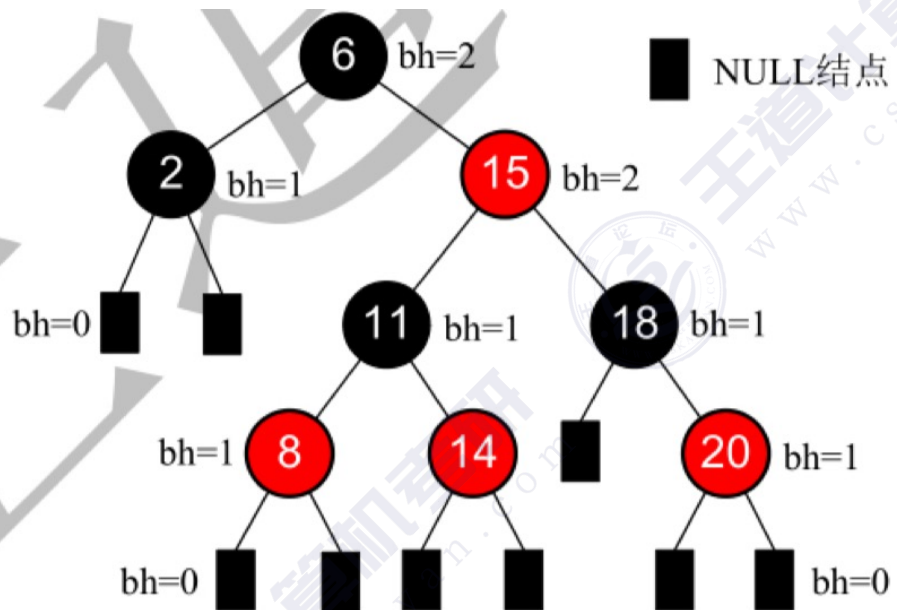
Animation Speed w: 800 h: 300 Change Canvas Size Move Controls

Algorithm Visualizations

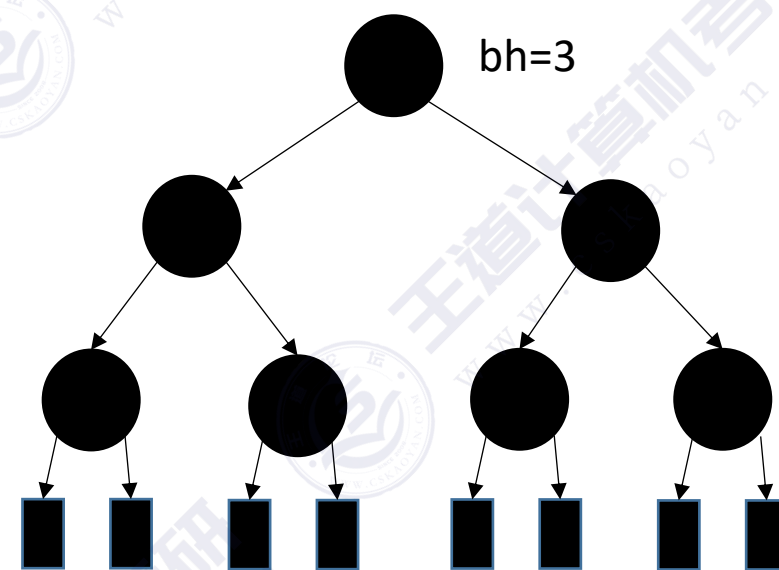
自我训练方法：你可以自己设计一些插入序列，每一次插入后，都先思考应该如何调整，然后在网站中演示验证。插入新元素时，尽可能覆盖 黑叔LL/RR/LR/RL、红叔 的情况。

<https://www.cs.usfca.edu/~galles/visualization/RedBlack.html>

与“黑高”相关的推论



根节点黑高=2，
内部结点数最少
的情况



根节点黑高=3，内部结点数最少
的情况

结点的**黑高** bh —— 从某结点出发（不含该结点）到达任一叶结点的路径上黑结点总数

思考：根节点黑高为 h 的红黑树，内部结点数（关键字）至少有多少个？

回答：内部结点数最少——总共 h 层黑结点的满树形态

结论：若根节点黑高为 h ，内部结点数（关键字）最少有 $2^h - 1$ 个

红黑树的定义→性质

红黑树是二叉排序树



左子树结点值 $\leq$ 根结点值 $\leq$ 右子树结点值

与普通BST相比，有什么要求



左根右，根叶黑
不红红，黑路同



张口就是freestyle

①每个结点或是红色，或是黑色的

②根节点是黑色的

③叶结点（外部结点、NULL结点、失败结点）均是黑色的

④不存在两个相邻的红结点（即红结点的父节点和孩子结点均是黑色）

⑤对每个结点，从该节点到任一叶结点的简单路径上，所含黑结点的数目相同



性质1：从根节点到叶结点的最长路径不大于最短路径的2倍

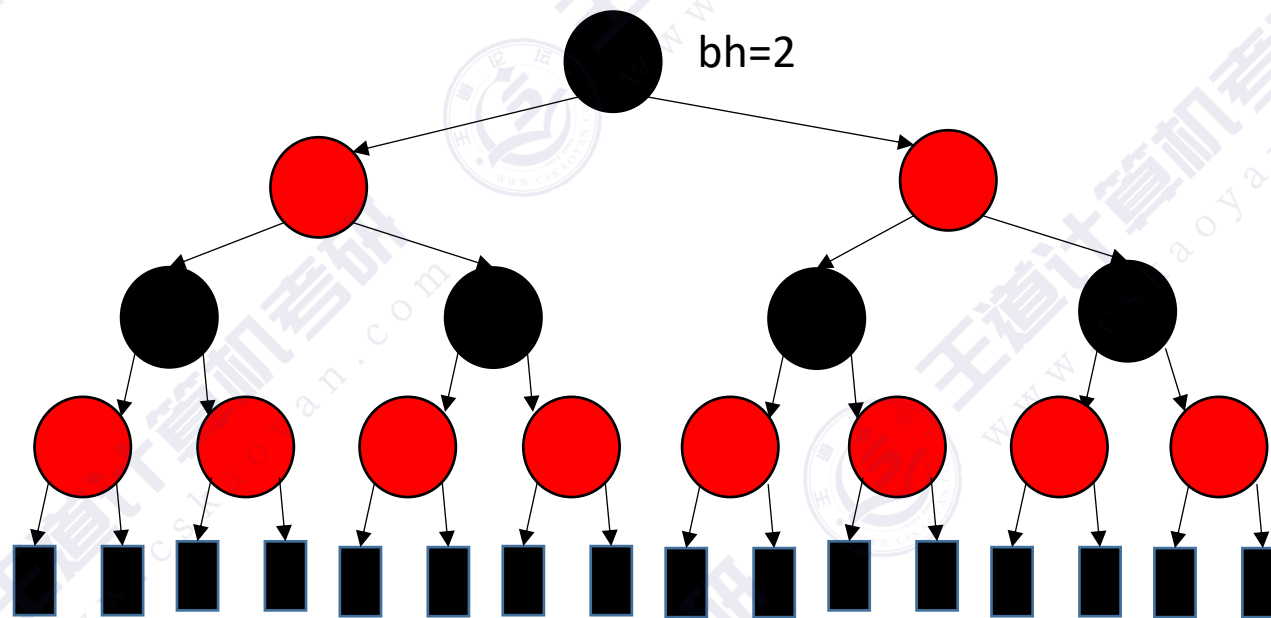
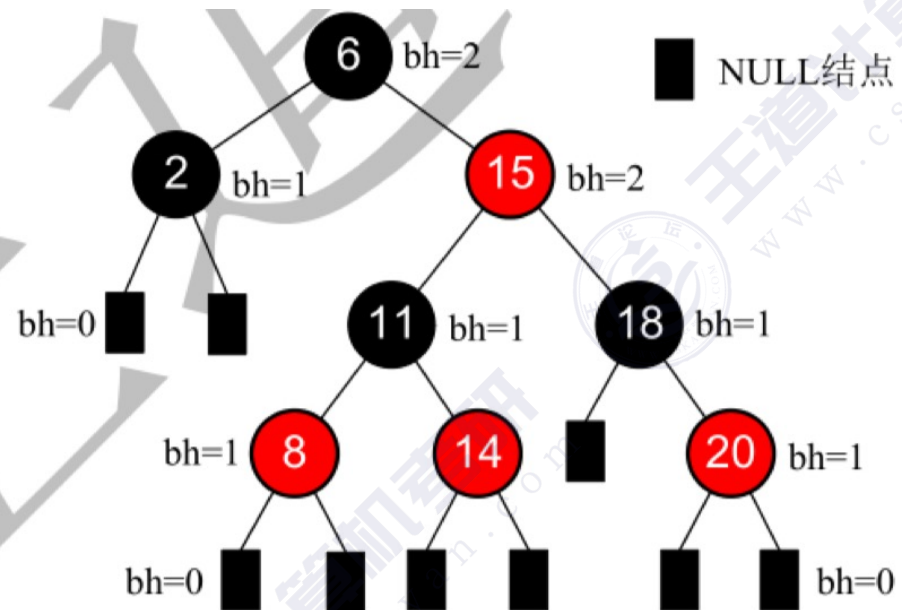
性质2：有 n 个内部节点的红黑树高度 $h \leq 2\log_2(n + 1)$

→ 红黑树查找操作时间复杂度 = $O(\log_2 n)$

性质1证明：任何一条查找失败路径上黑结点数量都相同，而路径上不能连续出现两个红结点，即红结点只能穿插在各个黑结点中间

性质2证明：若红黑树总高度= h ，则根节点黑高 $\geq h/2$ ，因此内部结点数 $n \geq 2^{h/2}-1$ ，由此推出 $h \leq 2\log_2(n + 1)$

与“黑高”相关的推论



根节点黑高=2，内部结点数最多的情况

结点的**黑高** bh —— 从某结点出发（不含该结点）到达任一叶结点的路径上黑结点总数

思考：根节点黑高为 h 的红黑树，内部结点数（关键字）至多有多少个？

回答：内部结点数**最多**的情况—— h 层黑结点，每一层黑结点下面都铺满一层红结点。共 $2h$ 层的满树形态

结论：若根节点黑高为 h ，内部结点数（关键字）最多有 $2^{2h}-1$ 个

本节内容

红黑树

删除操作

红黑树的删除操作



学完“红黑树的插入”后的你👉

警告⚠️ “红黑树的删除”比“红黑树的插入”难得多！

红黑树的删除操作

重要考点：

- ①红黑树删除操作的时间复杂度 = $O(\log_2 n)$
- ②在红黑树中删除结点的处理方式和“二叉排序树的删除”一样
- ③按②删除结点后，可能破坏“红黑树特性”，此时需要调整结点颜色、位置，使其再次满足“红黑树特性”。

知识回顾和重要考点

重要考点:

- ①红黑树删除操作的时间复杂度 = $O(\log_2 n)$
- ②在红黑树中删除结点的处理方式和“二叉排序树的删除”一样
- ③按②删除结点后，可能破坏“红黑树特性”，此时需要调整结点颜色、位置，使其再次满足“红黑树特性”。



提桶跑路

红黑树大概会怎么考?

红黑树的定义、性质——选择题

红黑树的插入/删除——要能手绘插入过程（不太可能考代码，略复杂），删除操作也比较麻烦，也许不考

4. 现有一棵无重复关键字的平衡二叉树（AVL 树），对其进行中序遍历可得到一个降序序列。下列关于该平衡二叉树的叙述中，正确的是_____。↵

A. 根结点的度一定为 2

B. 树中最小元素一定是叶结点↵

C. 最后插入的元素一定是叶结点

D. 树中最大元素一定是无左子树↵

2015真题

3. 若将关键字 1, 2, 3, 4, 5, 6, 7 依次插入到初始为空的平衡二叉树 T 中，则 T 中平衡因子为 0 的分支结点的个数是_____。↵

A. 0

B. 1

C. 2

D. 3↵

2013真题



知识总览

红黑树

定义、性质



插入



删除